Investigating the effect of prompting techniques on answer accuracy and runtime of Large Language Models in solving mathematical problems

Research Question: To what extent do prompting techniques affect the accuracy and runtime of Large Language Models in mathematical problem-solving?

Subject: Computer Science (Group 4) Extended Essay

Word Count: 3,744

Session: November 2025

TABLE OF CONTENTS

1. Introduction

1.1. Context and Scope:

Large Language Models (LLMs) such as ChatGPT and DeepSeek are becoming increasingly popular today for their abilities to assist us in various tasks. These models are a subset of Artificial Intelligence that utilize deep learning algorithms to generate new outputs based on the vast amounts of data they are trained on. They perform tasks such as summarization, text generation, and question answering. As they are naturally designed to perform Natural Language Processing (NLP) tasks, they struggle with reasoning and logic tasks (Singha Roy et al., 2025). Mathematics is a field that heavily relies on in-depth reasoning to solve problems. The capabilities of LLMs in solving mathematical problems are still dubious.

LLMs utilize a probabilistic sampling process to select the next token. They merely replicate reasoning based on patterns in the training data to predict the next word of a sentence (Masood, 2025). Prompts are the instructions given by a user to an LLM. They guide step-by-step reasoning, therefore improving accuracy and transparency.

1.2. Research Question:

The primary goal of this paper is to explore how the mathematical problem-solving capabilities of LLMs are affected by different prompting techniques. LLMs are evaluated for accuracy and responsiveness. In particular, this provides the opportunity to investigate the Research Question: **To what extent do prompting techniques affect the accuracy and runtime of Large Language Models in mathematical problem-solving?**

1.3. Related Work:

The limitations of mathematical reasoning in Large Language Models have been studied. Apple Inc. research has introduced an evaluation benchmark called the GSM-Symbolic to generate diverse variations of mathematical questions. A research journal published in Arxiv proposes the Mathematical Pitfalls and Logical Evaluation (MAPLE) score based on error rates and redundancy for holistic evaluation. Another research paper published in 2024 systematically analyzes 41 types of prompting techniques and presents them in the form of a taxonomy. This study evaluates both metrics to consider the trade-off between performance and speed when employing three key prompting techniques on LLMs. Prompting is a cost-effective way to improve mathematical reasoning. Through this study, I seek to contribute to cost-effective LLM optimizations for reasoning tasks in the future.

2. Background Study

2.1. Working of an LLM

2.1.1 Neural Networks

A neural network is a computational model widely used in artificial intelligence (AI) that mimics the human brain through layers of interconnected neurons or artificial nodes. It includes three layers: an input layer that receives the data as input for processing and computation, hidden layers that process the input data from the input layer, and an output layer that produces the final output.

The final output given by the neural network is compared with the actual output, and this difference assigns parameters such as weights and biases to the nodes. Weights adjust the

strength of input signals between neurons in different layers of a neural network to reduce this difference. Biases are constant parameters added to the weighted input used for learning patterns. Both are critical for mathematical reasoning as they determine how LLMs prioritise different input features from the prompts given.

2.1.2 Transformer Architecture

A transformer is a type of neural network architecture that has significantly contributed to the advancement of LLMs. It was introduced in a research paper titled "Attention is All You Need" in 2017. It uses different attention mechanisms to capture long-range relationships between data. Although this is primarily used to process complex text, it can also be used to improve multi-step reasoning.

The text entering the transformer is tokenized or broken down into smaller words. It is then represented as numerical vectors or vector embeddings. Positional encodings capture the logical sequence of the inputted text by adding positions (IBM, 2024). An encoder is used to process these embeddings and identify patterns. A decoder generates outputs in the form of individual tokens. LLMs must be able to capture similarities in mathematical structure to improve problem-solving capabilities (Datacamp, 2024).

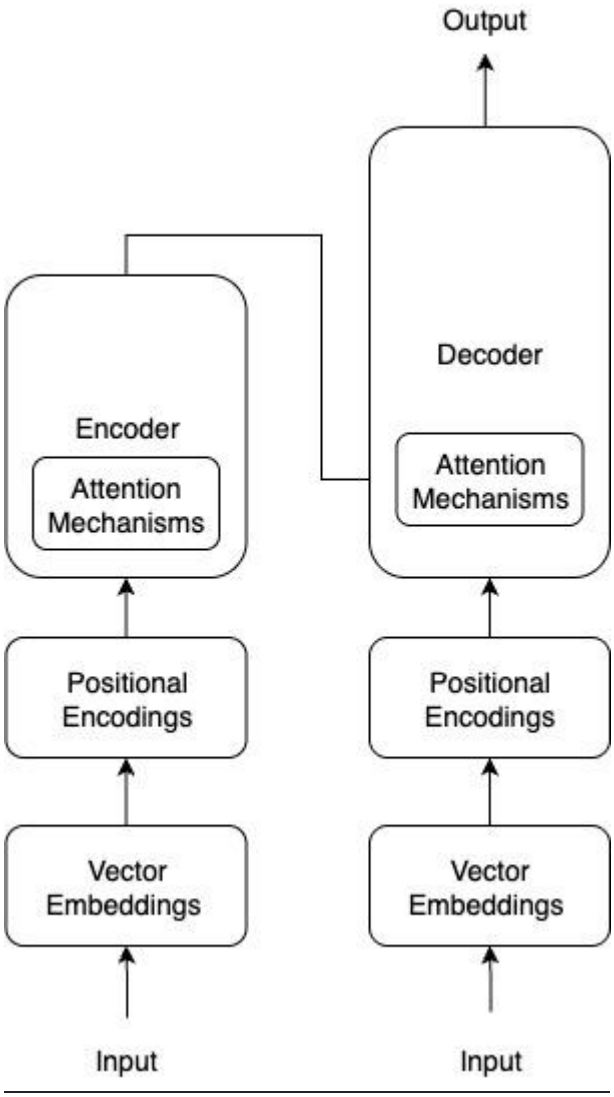


Figure 1: Transformer Architecture used in LLMs

Hallucination is an important challenge in mathematical reasoning, where the model generates meaningless outputs due to the lack of sufficient context. LLMs may generate steps that appear convincing but are incorrect. Thus, mathematical tasks must be evaluated for answer accuracy. LLMs are prone to generating different outputs for the same input (Bender, 2021).

2.2. Retrieval Augmented Generation:

Retrieval Augmented Generation, known as RAG, is an external framework that integrates

external sources with an LLM to reduce hallucinations and generate accurate outputs (IBM, 2024). Instead of solely relying on the training data, RAG uses a traditional retrieval system to fetch data relevant to a user's prompt. This additional data augments the LLM's existing knowledge with additional context to facilitate structured reasoning.

2.3. Challenges in Mathematical Reasoning:

Several research papers point to the limitations of using LLMs for mathematical reasoning. LLMs may be more sophisticated in terms of probabilistic pattern matching rather than formal reasoning. They are prone to making errors in calculations as they lack the functionalities of calculators. They have a fixed context window. Thus, they can remember only a limited number of tokens, creating the need for an external memory.

The transformer architecture treats mathematics like another language and copies patterns to predict the next token. Moreover, the training datasets of LLMs lack symbolic templates for LLMs to understand mathematical characters and symbols.

2.4. Prompting Techniques

Prompting helps optimize LLMs for specific use cases and retrieve accurate outputs. Poor prompting may lead to hallucinations and inaccuracies. There are multiple techniques used in prompting. The 3 most basic prompting techniques are:

- **Zero-shot prompting:** This is a prompting technique that uses direct instructions without context or examples. The LLM answers the prompt based on its training data.
- **Few-shot prompting:** Also known as in-context prompting, this prompting technique involves adding examples along with the instruction. The LLM may copy the logic in the examples without truly understanding the reasoning behind them.

- **Chain-of-Thought prompting (CoT):** The prompt is structured as a chain of subtasks in this technique. The instructions are broken down into a linear series of subtasks guiding the LLM to execute its algorithms. It supports sophisticated reasoning capabilities. (Prompting Guide, n.d.)

Prompt sensitivity is a model's responsiveness to minor changes in the prompts given by users. Rephrasing the same mathematical problem in different ways yields different outputs due to different reasoning paths (Mirzadeh et al., 2024). Our focus in this research is to investigate the impact of different promoting techniques on mathematical problem-solving.

3. Methodology

3.1. Overview

In the scope of the experiment, 4 LLM models (*OpenAI GPT 4, OpenAI o4-mini, OpenAI GPT 4.1, DeepSeek-V1*) are used with all-MiniLM-L6-v2 as the sentence-transformer model for processing the Math Word Problem (MWP) Multiple-equation alg514 datasets. The models were deployed in Google Colab, a cloud-based Jupyter notebook environment for easier experimentation (Google, n.d.). It allows users to create individual code cells that have to be executed in a sequential order.

This setup is utilized with three prompting techniques (zero-shot, few-shot, and CoT) evaluated for accuracy and runtime performance. The three prompting techniques used in the study are representative of other prompting techniques as they vary from providing minimal context (zero-shot) to examples (few-shot) and step-by-step reasoning (CoT).

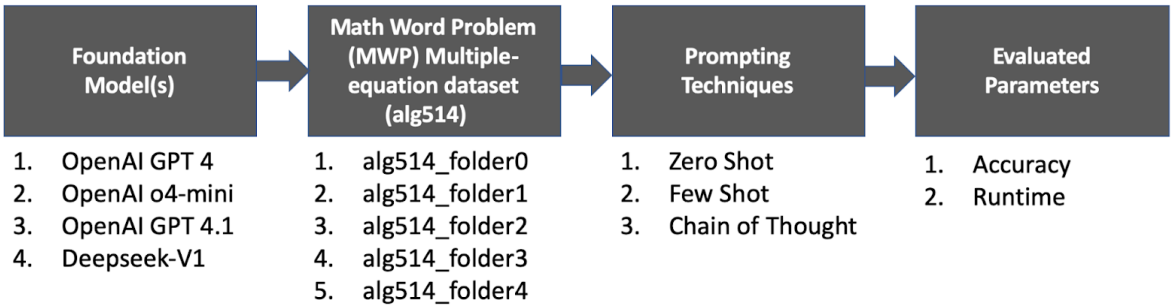


Figure 2: Experimental Overview

3.2 Dataset

MathEval is a benchmark widely used to evaluate the mathematical capacities of LLMs. Out of the 20 evaluation datasets available, the Dolphin1878 dataset was chosen as it contains 1878 user-generated math problems from algebra.com and other websites, organized into 187 sample datasets (Matheval.ai, n.d.). In order to ensure a fair and rigorous experiment, 5 Multi-Word Problem (MWP) alg514 evaluation datasets were randomly selected to prevent bias. The problems in these datasets are word problems that contain 1 to 4 unknowns. They are suited to the scope of the study as they require multi-step reasoning. The number of datasets was limited to 5, and the number of LLMs was limited to 4 due to the computational cost involved in running LLMs.

3.3 Variables

Independent Variable	Type of Prompting Technique used: - Zero-shot prompting - Few-shot prompting - Chain-of-Thought prompting

Dependent Variables	**Accuracy:** The number of correct answers that match the ground truth divided by the total number of answers. Accuracy is always expressed as a percentage.
	Runtime: The number of seconds taken by a specific LLM with a specific dataset inputted to answer questions through each of the prompting techniques. This helps gauge the speed and computational efficiency of different models.

While model latency refers to the time delay between the user inputting their query and the model generating an output, runtime measures the total time of execution for a program in milliseconds (ms) or seconds (s). In this experiment, runtime was measured per dataset. So, separate code cells were created for each prompting template and were calculated for each of the models.

Control Variable	Means of Controlling
Hardware Accelerator	The T4 GPU from Google Colab was used as

	the hardware accelerator throughout this experiment to ensure consistent runtime.
Temperature - It is a parameter that determines how random the generated outputs can be. A lower temperature produces focused results while a higher temperature produces creative outputs.	The temperature was constantly at 0.2 for three of the four models.
Max Length - It states the maximum number of tokens an LLM can generate as output.	The maximum number of tokens generated by the model was set to 1000 to ensure enough words for each solution generated.
Dataset Used	The same 5 datasets were used for each prompting model against each LLM without any change.

- Preliminary testing using a temperature of 1.0 produced an accuracy of 0.00% multiple times. Through trial and error, 0.2 was determined as the optimal temperature. This value reduces randomness or hallucinations. Mathematical problem solving does not require creativity, although 0.2 enables the model to explore other ways to solve the given problem. The only exception where the temperature was set to 1.0 was the o4-mini model, which does not support

temperature as its parameter.

- During the preliminary testing, a limit of 100 words truncated the output generated by CoT prompts and consistently led to CoT accuracies of 0.00%. Therefore, 1000 was chosen as the maximum token limit to allow sufficient space for reasoning steps, especially in CoT. While this may introduce bias towards models that elaborate more, the evaluation metric only considers the final answer.

3.4. Overview of LLMs Used

The following models have been used:

- *OpenAI GPT 4*: It's an older model described to exhibit medium speed and average intelligence. Primarily developed to complete chat-based tasks, it helps understand how well a chat-based model can reason with different prompting techniques in the experiment.
- *OpenAI o4-mini:* It is an OpenAI model specifically optimized for complex, multi-step tasks. It was chosen as it is specifically optimized for reasoning and coding tasks.
- *OpenAI GPT 4.1:* This is one of the flagship models developed by OpenAI for businesses and developers. It was selected due to its long context window (maximum number of tokens that an LLM can input) and instruction tuning. It exhibits medium speed and higher intelligence.
- *DeepSeek-V1:* This is the initial model released by DeepSeek. This was selected as it uses reinforcement learning and a CoT-based approach to reasoning tasks. It also helps distinguish performance from the OpenAI models used. However, its speed is known to be slow as it uses a large number of tokens.

3.5. Baseline Testing

Before starting the experiment, baseline testing was performed in OpenAI's playground using the *GPT-4.1 model* to gauge LLM performance before deploying it in Google Colab.

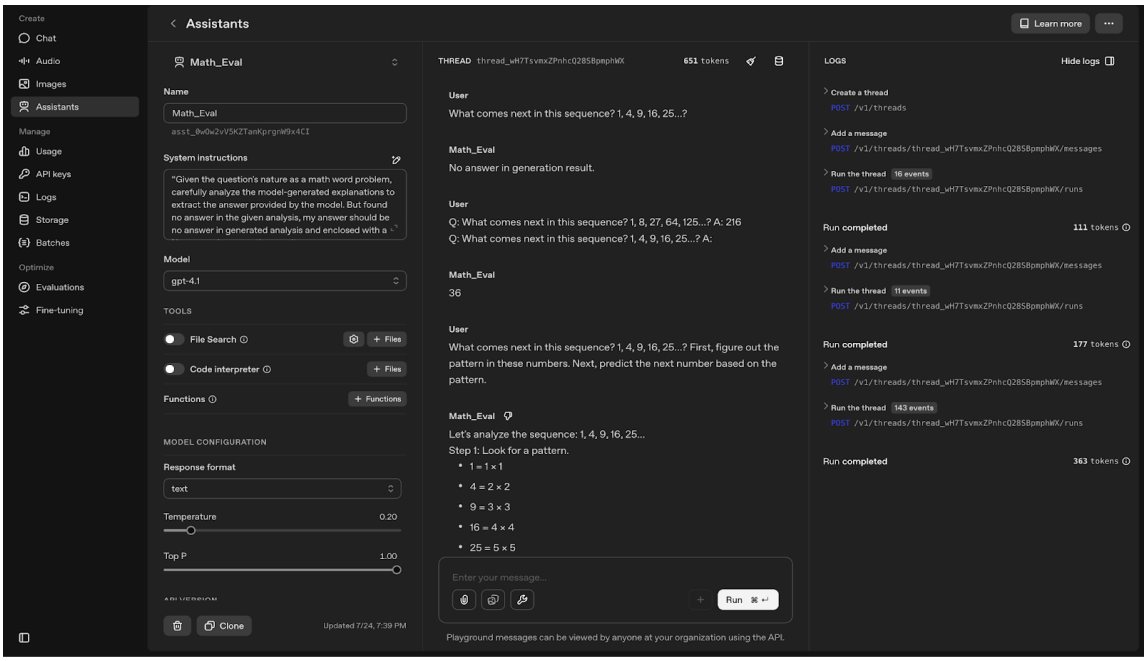


Figure 3: Baseline Testing in OpenAI Playground for GPT-4.1

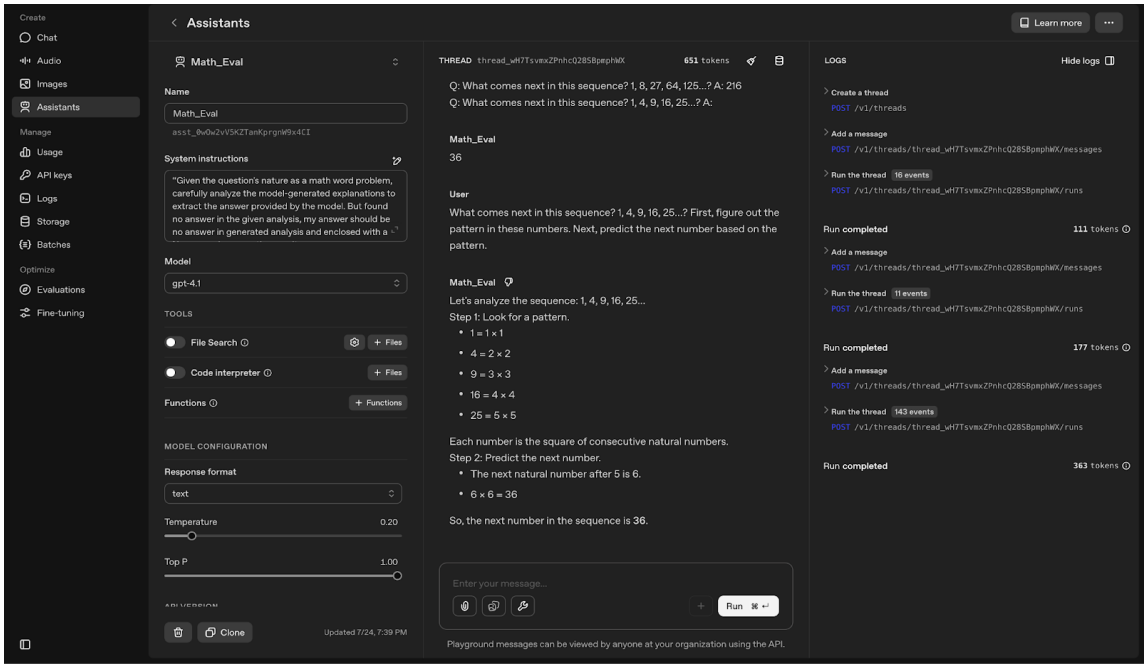


Figure 4: Baseline in OpenAI Playground for GPT-4.1 (Continued)

The following table summarizes the results:

Temperature	0.2
System Instruction	Given the question's nature as a math word problem, carefully analyze the model-generated explanations to extract the answer provided by the model. But found no answer in the given analysis, my answer should be no answer in the generated analysis and enclosed with a . No answer in the generation result.
Question	What comes next in this sequence? 1, 4, 9, 16, 25…?
Ground Truth	36
Zero-shot answer	No answer in the generation result
Few-shot answer	36
CoT answer	36

While few-shot and CoT prompting resulted in accurate answers, zero-shot prompting resulted in an accuracy of 0, as no answer was generated. The results of OpenAI's playground experiment were representative of the results retrieved by the other LLMs.

3.6. Experimental Procedure:

The following flowchart illustrates the experimental setup using Google Colab.

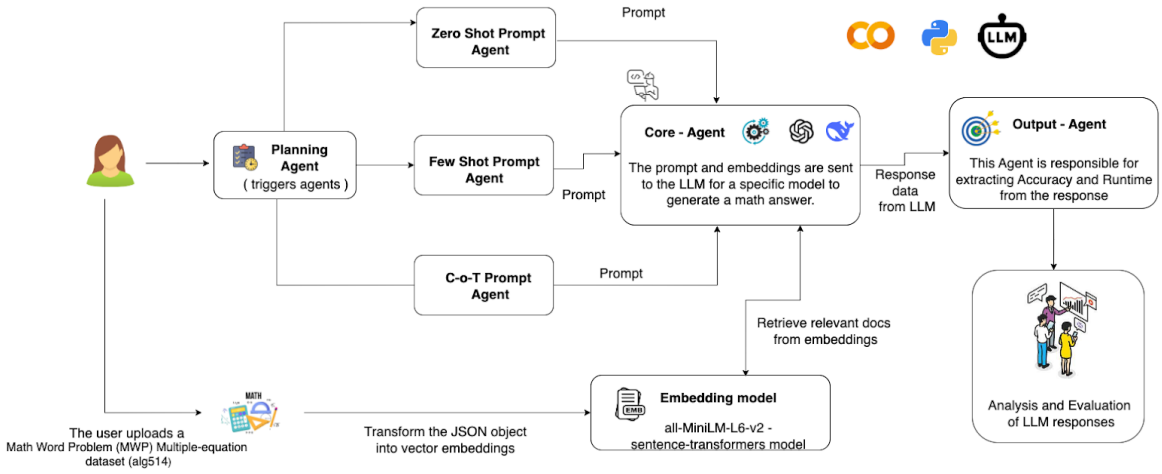


Figure 5: Flowchart for the Experimental Procedure

Setting up the environment for Deployment in Google Colab

1. Sign in to Google Colab.
2. Navigate to the runtime on the toolbar and click on change runtime type. By default, the program uses the computer's CPU. Switch it to the T4 GPU (Graphics Processing Unit). It has a memory of 16GB and accelerates the experiment.
3. Import and install necessary libraries such as openai or deepseek, transformers, and torch.
4. Import an embedding model for vector embeddings.
5. Set up the API key to access the LLMs. Create a function that will serve as an agent and interact with the LLM.

Evaluating the models

6. Upload each evaluation dataset and split the dataset into a training dataset and a testing

dataset. Use an agent to create a FAISS (Facebook AI Similarity Search) index that stores vector embeddings of different questions. It helps LLMs retrieve examples of similar questions for few-shot prompting.

7. Once the vector index is created, create prompting templates for zero-shot, few-shot, and CoT prompting.
8. Create separate functions to use each of these prompting templates that will execute the respective prompting templates as agents for each dataset
9. Create a function that can extract the final answer produced by the LLM and convert it into a structured numerical output.
10. Compare the final answers extracted with the ground truth and compute the accuracy of the LLM for each prompting technique using the evaluate_accuracy function.
11. Hover the mouse over the time displayed at the bottom of the program to note the runtime.
12. Use this evaluation pipeline to test all five datasets for an LLM.
13. Once all the datasets have been run through one LLM, repeat steps 5-12 for the remaining ones. The source code must be altered to deploy DeepSeek after the OpenAI models.

Data Analysis and Interpretation:

14. Record the accuracy from the program and the runtime from Google Colab in a spreadsheet. Organize them by dataset.
15. Compare accuracy and runtime for different prompting techniques across datasets.

After observing that the runtime couldn't be executed using the CPU, the free T4 GPU (Graphics Processing Unit) provided by Google Colab was used. LLMs require intensive computational power and memory. The T4 GPU can efficiently deploy LLMs due to its high

scalability and parallel processing capabilities (NVIDIA, n.d.).

JSON (JavaScript Object Notation) was imported for data interchange during API requests. The accuracy score was imported from the scikit-learn library. The train_test_split function was also imported to split the dataset into training and testing records. The transformer, 'all-MiniLM-L6-v2', a free open-source Sentence transformer from HuggingFace, was used for vector embeddings.

The requests are sent to the AI models through an API, Application Programming Interface. The following function was used to deploy OpenAI models in Google Colab:

```
def get_openai_response(prompt, model="gpt-4", temperature=0.2, max_tokens=1000):
    try:
        response = openai.ChatCompletion.create(
            model=model,
            messages=[{"role": "user", "content": prompt}],
            max_tokens=max_tokens,
            temperature=temperature
        )
        return response.choices[0].message.content.strip()
    except Exception as e:
        print("OpenAI error:", e)
        return ""
```

The function requires a prompt, model name, temperature, and max_tokens value as its arguments. The model name (*gpt-4*) can be replaced by the names of other OpenAI models. It uses the openai.ChatCompletion.create()method to send messages to the model

and receive outputs. It then returns the first response generated by the model as text after removing white spaces. The function also displays an error message if there are network issues or rate limits.

Unlike OpenAI, DeepSeek does not use Python to call APIs directly. Instead, it uses HTTP requests to generate outputs. Hence, the code was modified by importing requests and using the DeepSeek REST (Representational State) API. The source code for both experiments is attached in the appendix.

The results produced by each prompting technique are compared against labelled solutions in the dataset, referred to as the ground truth. An evaluate_accuracy metric was created to evaluate if the ground truth produced by the dataset is equal to the solutions predicted by each prompting technique. A tolerance of 1% was used to allow a difference of 0.01 between the prediction and the ground truth. This helps account for rounding errors and precision measurements. The code was improved to make sure that the answers of zero-shot and CoT prompting matched the format of the ground truth.

3.7. Hypothesis

The hypothesis is that CoT prompting will have the highest accuracy and runtime, while Zero-shot prompting will have the lowest accuracy and runtime. This is because CoT prompting breaks down the method of solving the problem and offers high clarity, whereas zero-shot prompting works based on simple logic. Thus, zero-shot prompting may struggle with multi-step word problems.

4. Experimental Results

4.1. Tabular Presentation of Results:

The results of the experiment have been presented in tabular form. The tables are organized based on the dataset.

Model	**Zero-shot Prompting**		**Few-shot Prompting**		**CoT Prompting**	
	Accuracy	*Runtime*	*Accuracy*	*Runtime*	*Accuracy*	*Runtime*
GPT-4	14.29	23.568	14.29	24.479	95.24	228.943
DeepSeek-V1	76.19	552.122	85.71	407.521	85.71	141.914
GPT-o4-mini	90.48	101.223	95.24	120.866	90.48	167.438
GPT-4.1	85.71	53.151s	52.38	33.403	80.95	98.233

Table 1: Accuracy and runtime for different LLMs using the 3 prompting techniques for Dataset0

Model	**Zero-shot Prompting**		**Few-shot Prompting**		**CoT Prompting**	
	Accuracy	*Runtime*	*Accuracy*	*Runtime*	*Accuracy*	*Runtime*
GPT-4	19.05	27.414	19.05	27.414	80.95	210.992
DeepSeek-V1	71.43	633.81	66.67	567.558	80.95	458.783
GPT-o4-mini	76.19	132.589	85.71	148.376	76.19	162.315
GPT-4.1	66.67	41.216	52.38	29.833	61.9	131.369

Table 2: Accuracy and runtime for different LLMs using the 3 prompting techniques for Dataset 1

Model	**Zero-shot Prompting**		**Few-shot Prompting**		**CoT Prompting**	
	Accuracy	*Runtime*	*Accuracy*	*Runtime*	*Accuracy*	*Runtime*
GPT-4	33.33	25.551	42.86	25.082	61.9	191.148
DeepSeek-V1	71.43	505.991	80.95	381.486	80.95	361.285
GPT-o4-mini	85.71	103.525	76.19	157.261	90.48	190.917
GPT-4.1	61.9	46.619	71.43	35.848	76.19	99.863

Table 3: Accuracy and runtime for different LLMs using the 3 prompting techniques for Dataset 2

Model	**Zero-shot Prompting**		**Few-shot Prompting**		**CoT Prompting**	
	Accuracy	*Runtime*	*Accuracy*	*Runtime*	*Accuracy*	*Runtime*
GPT-4	28.57	0.923	52.38	24.064	95.24	177.662
DeepSeek-V1	80.95	516.577	85.71	428.46	85.71	323.722
GPT-o4-mini	80.95	122.373	76.19	131.256	66.67	197.161
GPT-4.1	61.9	36.255	57.14	57.384	76.19	78.629

Table 4: Accuracy and runtime for different LLMs using the 3 prompting techniques for Dataset 3

Model	**Zero-shot Prompting**		**Few-shot Prompting**		**CoT Prompting**	
	Accuracy	*Runtime*	*Accuracy*	*Runtime*	*Accuracy*	*Runtime*
GPT-4	36.36	21.985	31.82	24.878	90.91	203.694
DeepSeek-V1	81.82	712.287	95.45	471.428	95.45	405.43
GPT-o4-mini	95.45	127.235	76.19	12.951	100	189.317
GPT-4.1	86.36	103.308	45.45	18.118	95.45	148.541

Table 5: Accuracy and runtime for different LLMs using the 3 prompting techniques for Dataset 4

4.2. Graphical Presentation of the Data:

The results of the experiment have been presented as graphs to enable visual comparison. The graphs are organized based on the dataset.

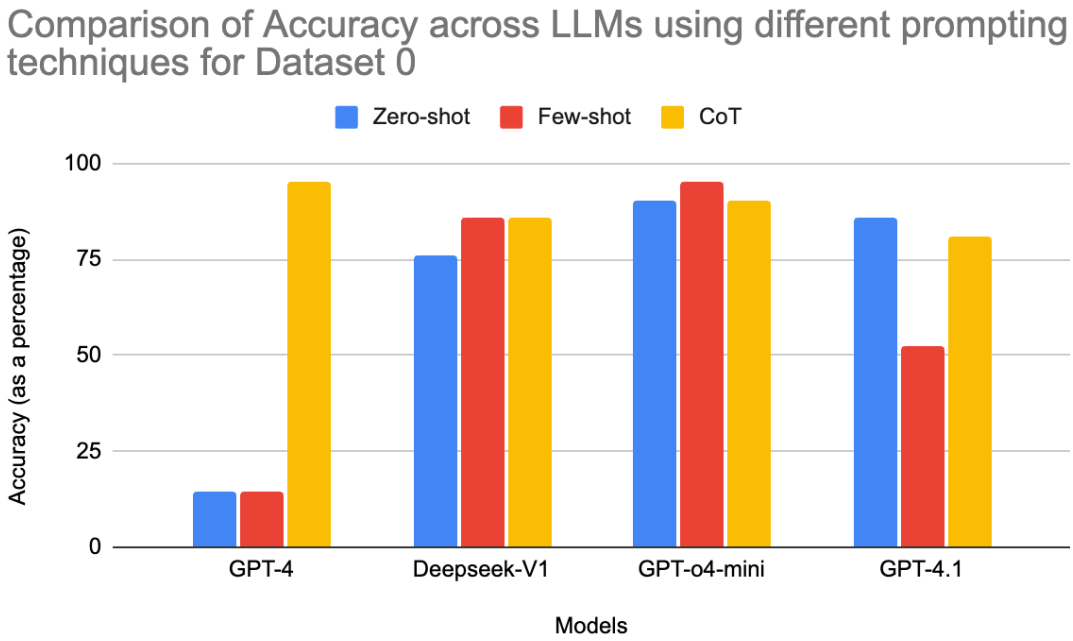


Figure 6: Comparison of accuracy across LLMs using different prompting techniques for Dataset 0

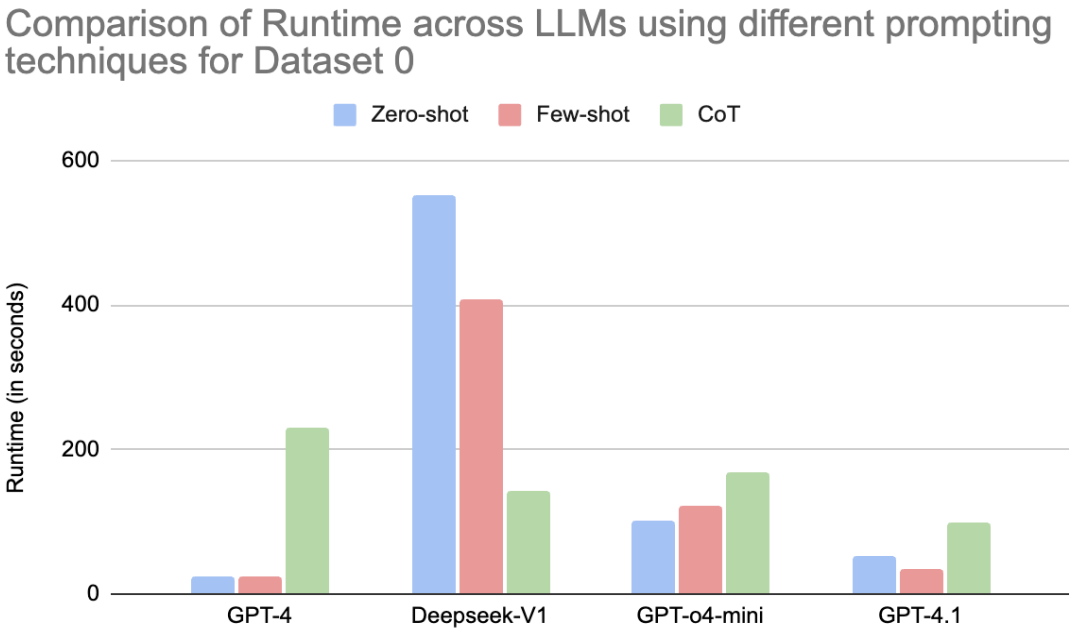


Figure 7: Comparison of runtime across LLMs using different prompting techniques for Dataset 0

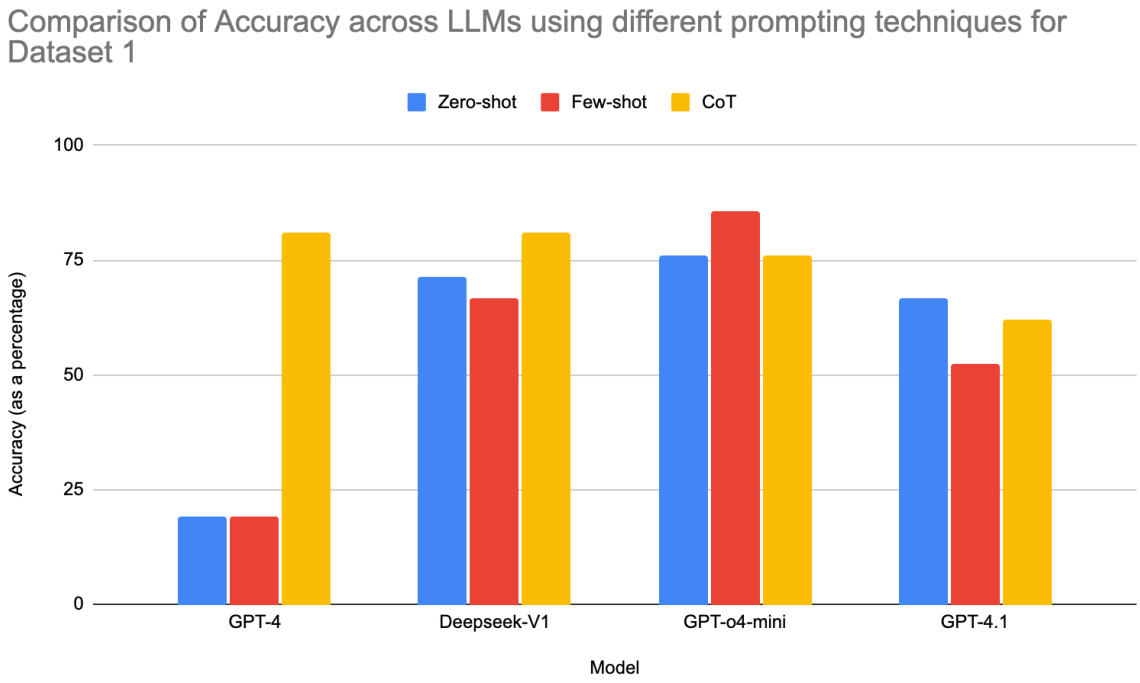


Figure 8: Comparison of accuracy across LLMs using different prompting techniques for Dataset 1

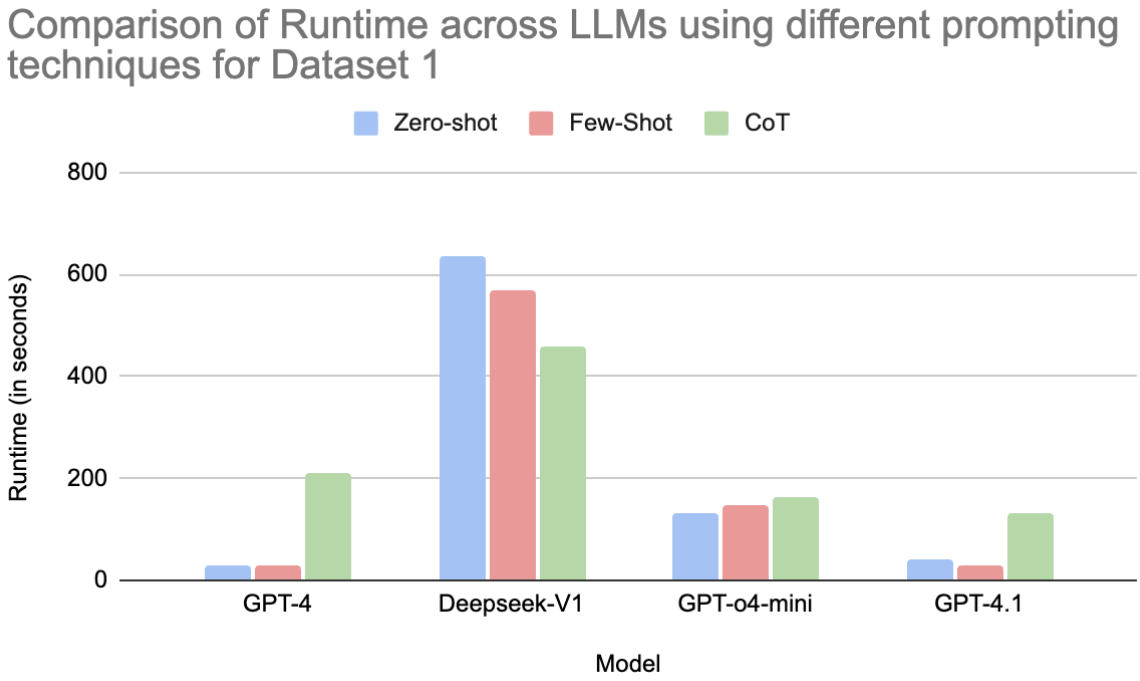


Figure 9: Comparison of runtime across LLMs using different prompting techniques for Dataset 1

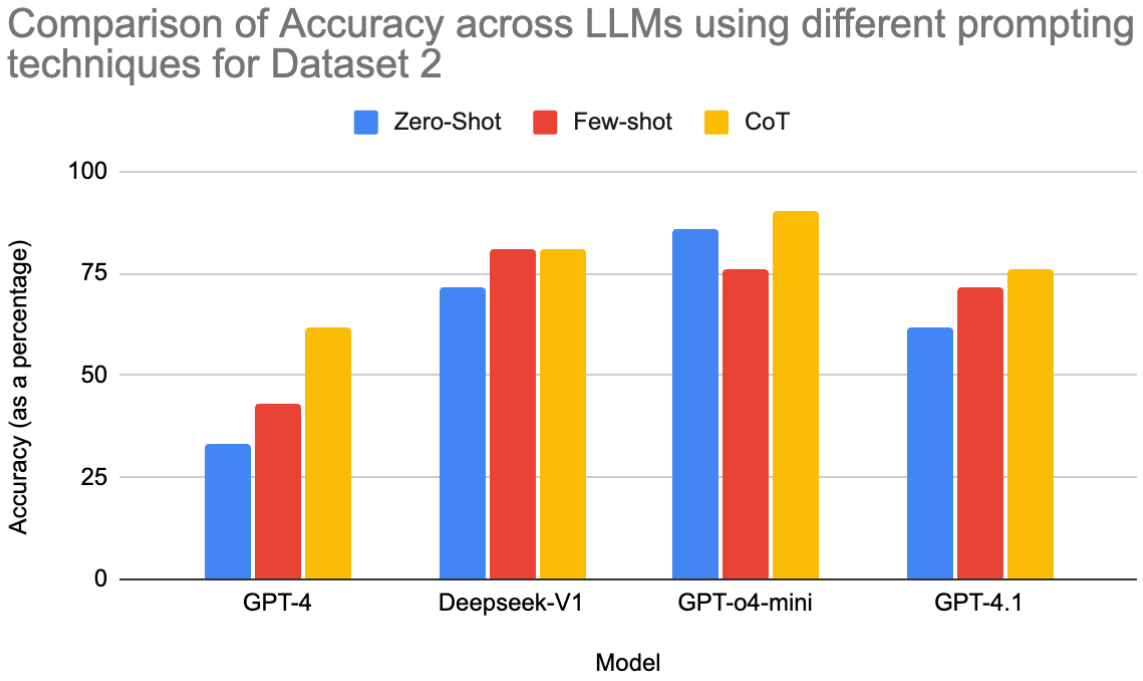


Figure 10: Comparison of accuracy across LLMs using different prompting techniques for Dataset 2

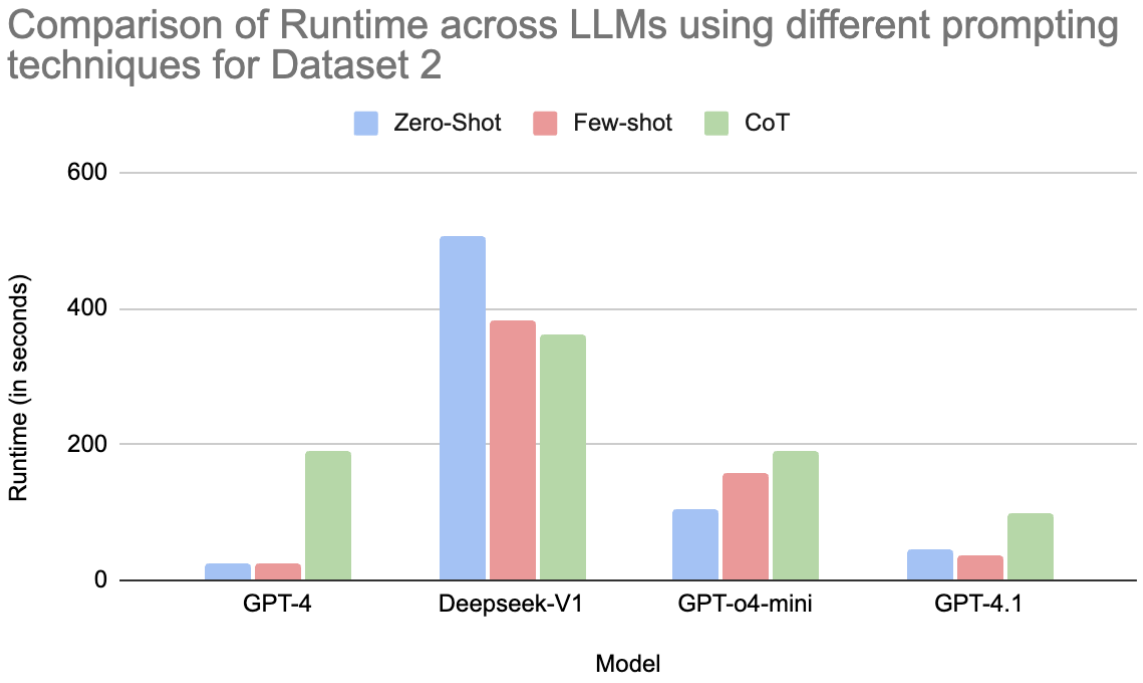


Figure 11: Comparison of runtime across LLMs using different prompting techniques for Dataset 2

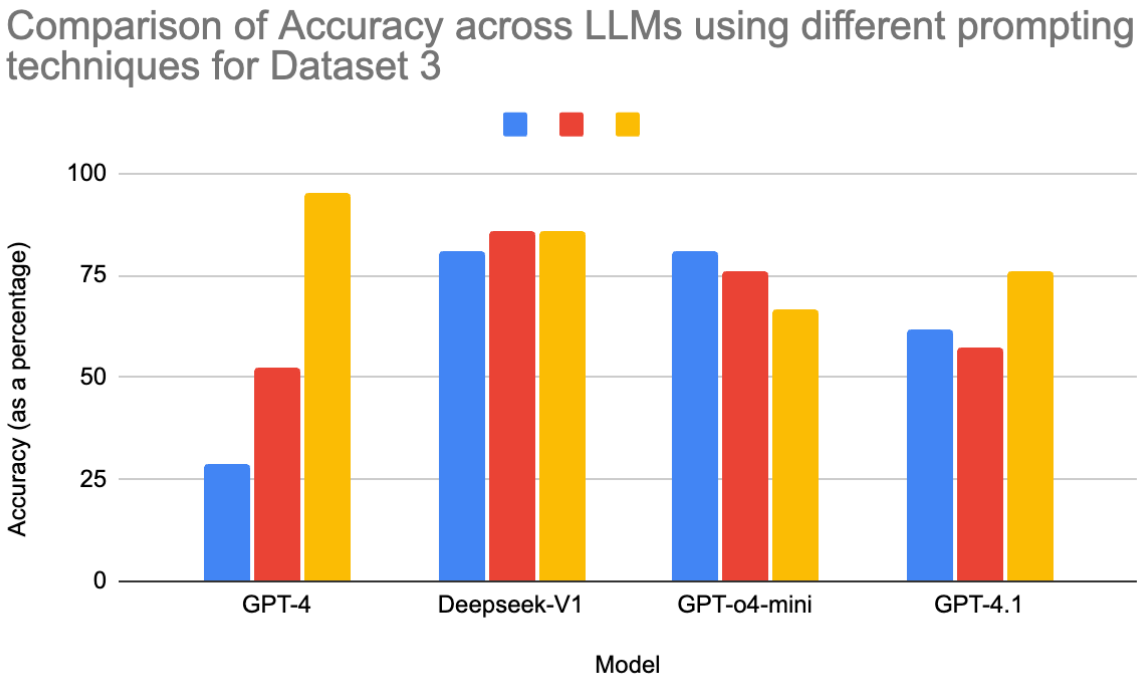


Figure 12: Comparison of accuracy across LLMs using different prompting techniques for Dataset 3

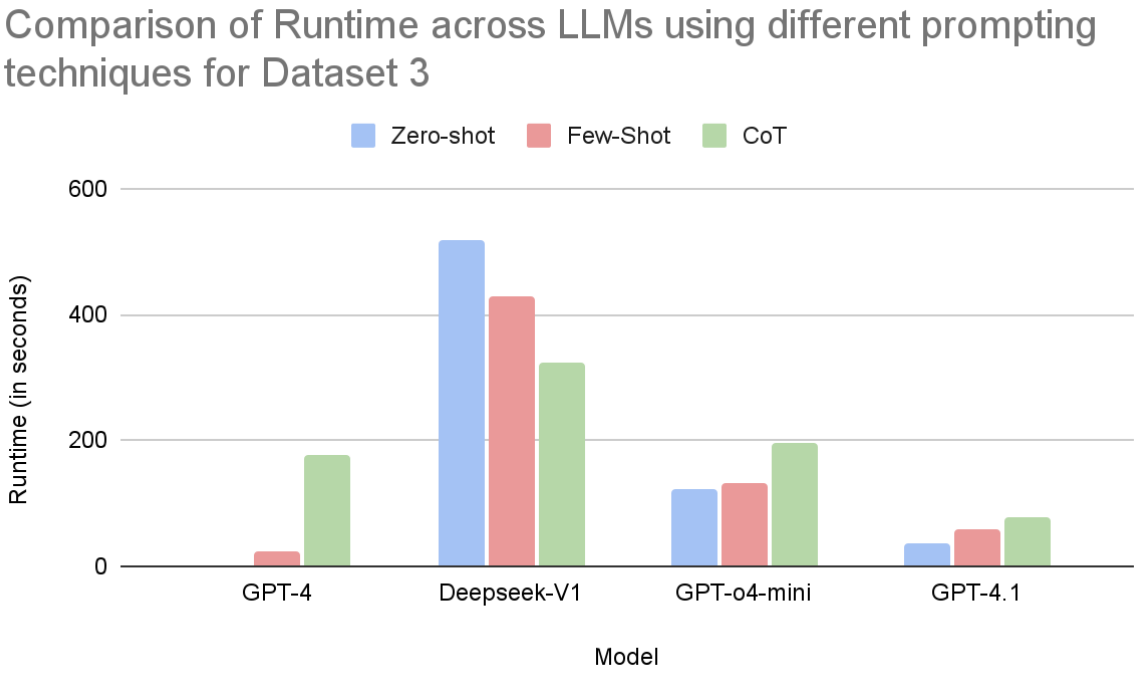


Figure 13: Comparison of runtime across LLMs using different prompting techniques for Dataset 3

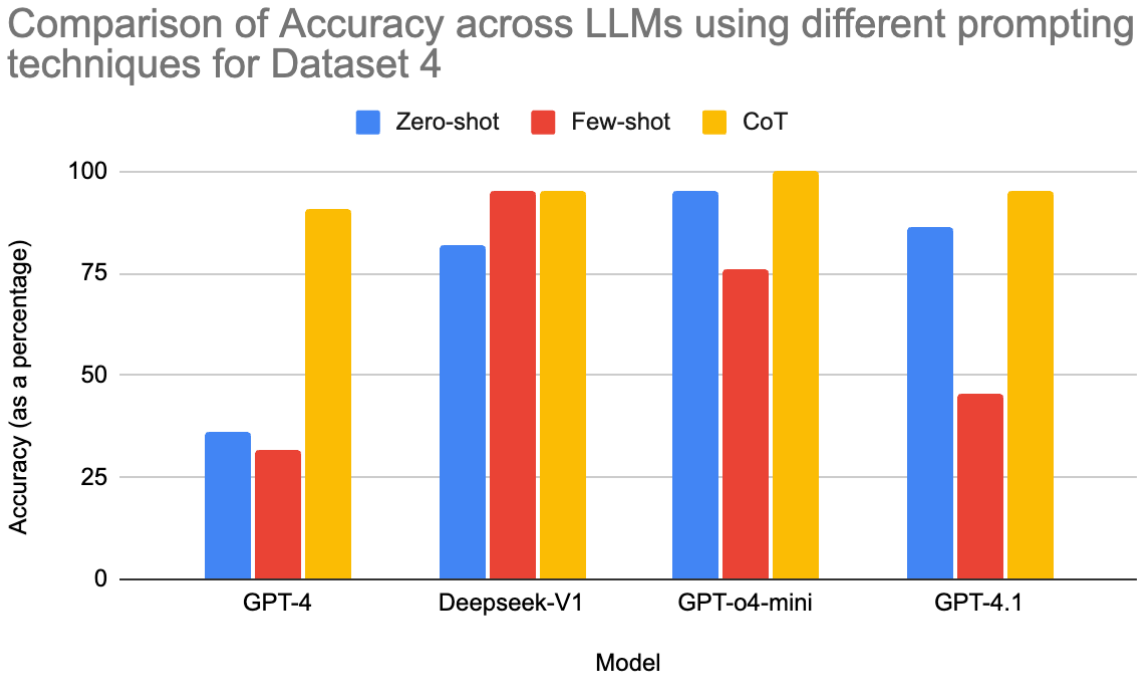


Figure 14: Comparison of accuracy across LLMs using different prompting techniques for Dataset 4

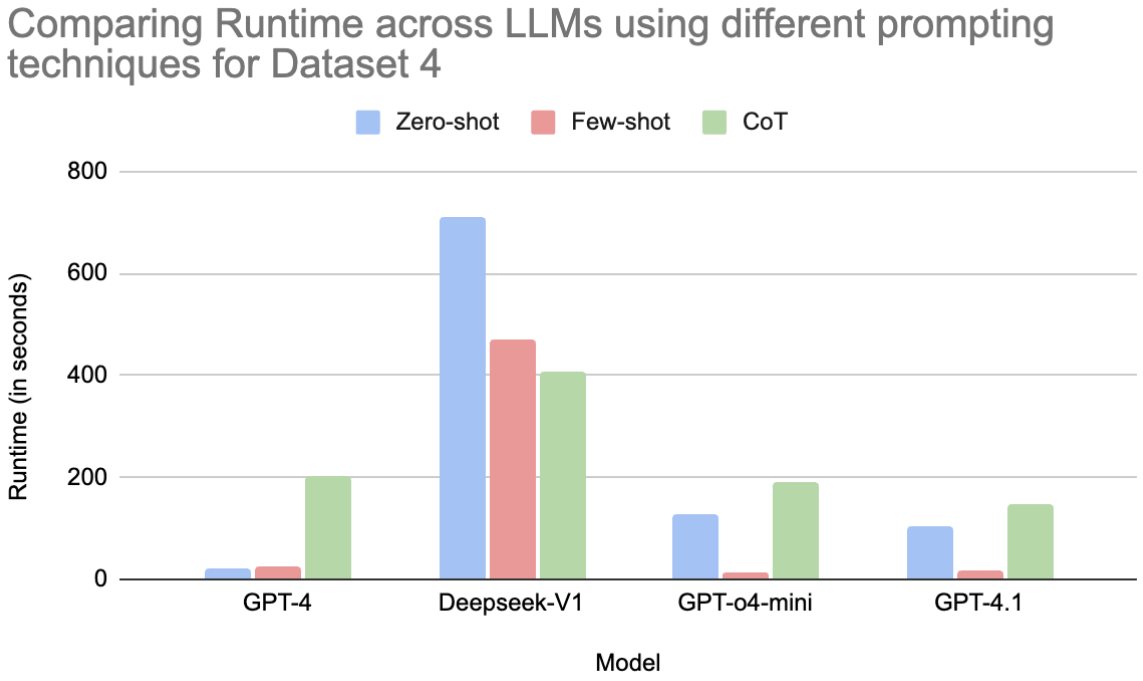


Figure 15: Comparison of runtime across LLMs using different prompting techniques for Dataset 4

4.3. Analysis

This analysis examines 3 prompting techniques across 4 LLMs. The average confidence interval was calculated to be between 57.2% and 84%, indicating a higher variability of 26.8 points due to different model features.

In *GPT-4*, CoT prompting consistently has the highest accuracy as well as the highest runtime, while Zero-shot prompting has the lowest accuracy and lowest runtime. Notably, when dataset 3 was used, the runtime was as low as 0.92 seconds, as documented in Table 4. It consistently performs well with CoT as it is a conversational model with average intelligence compared to newer models.

An analysis of the runtime graphs reveals that the runtime of the DeepSeek V1 model is significantly higher than the OpenAI models, as it processes each word for a longer time, incurring more computational costs. Interestingly, DeepSeek-V1 exhibits inverted runtime patterns where zero-shot prompting consistently demands the highest runtime, while CoT has the lowest runtime. This is likely due to the lack of context, forcing additional computations per question as zero-shot prompting correlates with low accuracy. CoT, on the other hand, may be more computationally efficient due to optimized reasoning pathways and DeepSeek's inherent CoT architecture. Furthermore, few-shot prompting achieves accuracy equivalent to CoT in 4 out of 5 datasets for the V1 model, validating the model's optimization for mathematical problem-solving.

When Dataset 3 was evaluated for *o4-mini*, CoT produced the lowest accuracy (66.67%), while zero-shot achieved the highest (80.95%). This counterintuitive result may be a result of *o4-mini's* architectural optimization for resource efficiency, as specified in the OpenAI model

docs. CoT prompts could have introduced unnecessary verbosity, reducing accuracy.

Supporting this interpretation, the *o4-mini* model exhibits an accuracy of 95.24% for few-shot prompting using Dataset 0. This number was higher than the accuracy achieved by zero-shot or CoT prompting. This may particularly happen when the problem is not complex and can be solved with generalized rules. During such cases, using CoT might lead to unnecessary wastage of runtime without adequate benefits.

Similarly, the *GPT-4.1* model demonstrates a zero-shot accuracy 4.76% higher than CoT accuracy on dataset 0 and a zero-shot accuracy 4.77% higher on dataset 1. This reinforces the model's optimization to follow instructions (instruction-tuning) as the model can respond to direct prompts better. The higher context window, however, does not provide any measurable improvements.

A holistic analysis of the runtimes recorded shows that CoT prompting consistently demands higher compute costs across OpenAI models. For instance, CoT prompting achieves a runtime of 228.943 seconds when GPT-4 was evaluated against dataset 0. However, these measurements in Google Colab are system-dependent and may not fully capture model performance due to changes in memory availability or system noise.

Zero-shot prompting may work for simple arithmetic problems and offers rapid execution, though its accuracy is lower and poses challenges in debugging errors. Few-shot prompting provides enhanced context compared to zero-shot prompting, but risks pattern matching where models replicate the logic in the examples given without understanding the reasoning behind them.

CoT prompting supports transparency and facilitates easier human evaluation of the steps used. Additionally, it reduces hallucinations by promoting structured reasoning over random pattern-matching. This represents a significant advantage of CoT prompting compared to few-shot and zero-shot prompting, despite the increased computational cost.

5. Conclusion

Based on the graphs and tables discussed above, CoT offers high accuracy in LLMs solving math problems as it breaks the problem down into smaller steps and aids in systematic planning. However, it increases computational cost and runtime due to the excessively high number of output tokens involved in its thinking and reasoning. On the other hand, zero-shot prompting produces results with lower runtime that are often lower in accuracy. Few-shot prompting acts as a balanced choice between the two, balancing the trade-offs between accuracy and speed. Surface-level reasoning fails in mathematics. So, the depth of reasoning is a key indicator in evaluating mathematical performance.

Model accuracy is also influenced by model architecture and training. Models such as *o4-mini* and *DeepSeek-V1* have been optimized with improved reasoning capacities. GPT-4.1 is optimized to respond to prompts better, although its increased context window does not affect results. The temperature ensures that the outputs are focused. The runtimes may be biased due to system limitations. The findings must be interpreted within the constraints of model differences and runtime bias.

6. Limitations

The analysis is subject to the following limitations:

- Since accuracy only evaluates the final answer, there is no quantitative metric in the experiment to check if the intermediate reasoning steps are correct.
- The dataset may not generalize well to rigorous mathematical reasoning in other mathematical domains, as it primarily focuses on algebra.
- The runtime is not normalized and varies from device to device. Thus, it is not fully indicative of prompt responsiveness.
- The retrieval of examples from the FAISS index cannot be evaluated to assess the efficiency of few-shot prompting templates. The tokenization of mathematical symbols and expressions is also unknown. Thus, the retrieval quality cannot be assessed.
- Since the OpenAI models used in the experiment are proprietary, the architecture cannot be fully evaluated.

7. Future Research

Future extensions of the experiment can use additional metrics such as step consistency or execution accuracy. These will help verify if a model's intermediate reasoning is correct or if it is being verbose due to a higher token limit. Datasets with undergraduate-level math problems and more complex domains, such as linear algebra or multivariable calculus, can be used to evaluate abstract reasoning. It helps determine if prompting

enables the LLMs in the experiment to generalize well to other branches of mathematics. However, this will require more computational resources. A fixed baseline task can be used to compare runtime across different systems and, therefore, normalize runtime. Embedding similarity and retrieval quality can be assessed by conducting a cosine similarity test of the vector embeddings. By assessing how similar two vectors are, this test also detects tokenization issues. An emerging field known as Reinforcement Learning from Human Feedback (RLHF) allows for research in leveraging human feedback iteratively to improve model performance.

Bibliography

---

1. Bender, E. M. (2021). On the dangers of stochastic parrots: Can language models be too big? Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency. https://dl.acm.org/doi/pdf/10.1145/3442188.3445922. Accessed on June 21, 2025.

2. DataCamp. (2024, April 26). Attention mechanism in LLMs: An intuitive explanation. https://www.datacamp.com/blog/attention-mechanism-in-llms-intuition. Accessed on November 11, 2024.

3. Google. (n.d.). Google Colaboratory. https://colab.research.google.com/. Accessed on November 11, 2024.

4. Gou, Z. (2024). Math-evaluation-harness: A simple toolkit for benchmarking LLMs on mathematical reasoning tasks. GitHub. https://github.com/ZubinGou/math-evaluation-harness. Accessed on June 21, 2025.

5. IBM. (2024, 12 June). What is vector embedding? IBM Think Topics. https://www.ibm.com/think/topics/vector-embedding. Accessed on November 11, 2024.

6. IBM. (n.d.). What is RAG (retrieval augmented generation)? IBM Think Topics. https://www.ibm.com/think/topics/retrieval-augmented-generation. Accessed on September 11, 2024.

7. Jégou, H., Douze, M., & Johnson, J. (2017, March 29). Faiss: A library for efficient similarity search. Engineering at Meta. https://engineering.fb.com/2017/03/29/data-infrastructure/faiss-a-library-for-efficient-similarity-search/. Accessed on November 12, 2024.

8. Li, X. (2024). A survey on LLM-based agentic workflows and LLM-profiled components. arXiv. https://arxiv.org/html/2406.05804v1. Accessed on December 1, 2024.

9. Masood, A. (2025, April 16). Why large language models struggle with mathematical reasoning? Medium. https://medium.com/@adnanmasood/why-large-language-models-struggle-with-mathematical-reasoning-3dc8e9f964ae. Accessed on July 11, 2025.

10. MathEval.ai. (n.d.). Dolphin1878 dataset. https://matheval.ai/dataset/dolphin1878/. Accessed on November 12, 2024.

11. Mirzadeh, I., Alizadeh, K., Shahrokhi, H., Tuzel, O., Bengio, S., & Farajtabar, M. (2024). GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models. arXiv. https://arxiv.org/pdf/2410.05229. Accessed on January 1, 2025.

12. NVIDIA. (n.d.). NVIDIA T4 tensor core GPUs for accelerating AI inference. https://www.nvidia.com/en-us/data-center/tesla-t4/. Accessed on November 20, 2024.

13. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (n.d.). Accuracy_score. Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html. Accessed on November 29, 2024.

14. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (n.d.). Train_test_split. Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html. Accessed on December 11, 2024.

15. Prompting Guide. (n.d.). Prompt engineering guide. https://www.promptingguide.ai/. Accessed on December 11, 2024.

16. Singha Roy, T., Baral, A., Jhaveri, A. R., & Baig, Y. (2025). Can LLMs understand math? Exploring the pitfalls in mathematical reasoning. arXiv. https://arxiv.org/html/2505.15623v1. Accessed on June 12, 2025.

17. University of Cambridge. (2025). New open-source platform allows users to evaluate performance of AI-powered chatbots [Press release]. https://www.cam.ac.uk/research/news/new-open-source-platform-allows-users-to-evaluate-performance-of-ai-powered-chatbots. Accessed on August 2, 2025.

18. White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., & Schmidt, D. C. (2024). The prompt report: A systematic survey of prompting techniques. arXiv. https://arxiv.org/html/2406.06608v1#Ch1.S1. Accessed on February 15, 2025.

Appendix

Source Code:

Versioning:

The following versions were used in the experiment.

openai==0.28

sentence-transformers==2.2.2

faiss-cpu==1.7.2

OpenAI:

```
# Install OpenAI to experiment with GPT-4, GPT-4.1, and o4-mini
pip install openai==0.28
# Install dependencies
!pip install -q openai faiss-cpu sentence-transformers
```

```
# Import Libraries
import openai
import pandas as pd, numpy as np, json, io, re, faiss, requests
from sklearn.model_selection import train_test_split
from sentence_transformers import SentenceTransformer
from google.colab import files
# Set your OpenAI API key
openai.api_key = 'API key has been removed for security reasons'
# Embedding model
model = SentenceTransformer('all-MiniLM-L6-v2')
# Upload and preprocess the dataset
def load_dataset(uploaded_file):
    data = []
    with io.StringIO(uploaded_file.decode('utf-8')) as f:
        for line in f:
            data.append(json.loads(line))
    return pd.DataFrame(data) # Returns a dataframe with original question and the ground truth
# Creates a FAISS Index to retrieve similar math problems for few-shot prompting
def create_faiss_index(texts):
    embeddings = model.encode(texts, convert_to_tensor=False)
    embeddings = np.array(embeddings, dtype=np.float32)
    index = faiss.IndexFlatL2(embeddings.shape[1])
    index.add(embeddings)
    return index, texts
# Retrieves contextual cues for prompting
def retrieve_relevant_docs(index, query, texts, k=5):
    query_embedding = model.encode([query])
```

```
    query_embedding = np.array(query_embedding, dtype=np.float32)
    _, indices = index.search(query_embedding, k)
    return [texts[i] for i in indices[0]]
# OpenAI API Call to Retrieve responses from the model
def get_openai_response(prompt, model="gpt-4", temperature=0.2, max_tokens=1000):
    try:
        response = openai.ChatCompletion.create(
            model=model,
            messages=[{"role": "user", "content": prompt}],
            max_tokens=max_tokens,
            temperature=temperature
        )
        return response.choices[0].message.content.strip()
    except Exception as e:
        print("OpenAI error:", e)
        return ""
# Extracts the final answer as a structured numerical output
def extract_final_answer(response):
    match = re.search(r"\[([\d.,\s\-+]+)\]", response)
    if match:
        numbers = re.findall(r"[-+]?\d*\.\d+|\d+", match.group(1))
        return [float(num) for num in numbers]
    return None
# Evaluates the accuracy with a tolerance of 1% from the ground truth
def evaluate_accuracy(predictions, ground_truths, tolerance=0.01):
    correct = 0
    for pred, gt in zip(predictions, ground_truths):
```

```
        pred_nums = extract_final_answer(pred)
        if pred_nums is None: continue
        gt_nums = re.findall(r"[-+]?\d*\.\d+|\d+", str(gt))
        gt_nums = [float(n) for n in gt_nums]
        if len(pred_nums) == len(gt_nums) and all(abs(p - g) <= max(tolerance, 0.01 * abs(g)) for p, g in
zip(pred_nums, gt_nums)):
            correct += 1
    return (correct / len(ground_truths)) * 100

uploaded = files.upload() # Upload the file from the computer
file = list(uploaded.values())[0]
df = load_dataset(file)
# Splits the dataset into training and testing records
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
print(f"Train: {len(train_df)}, Test: {len(test_df)}")
# Builds vector index
index, train_texts = create_faiss_index(train_df['text'].tolist())
# Prompt Template for Zero-shot Prompting
def zero_shot_prompt(question, context):
    return f"""You are a helpful assistant that answers math problems accurately. Please provide a
numerical answer.

Context:
{context}

Question:
{question}
```

```
final Answer in the format of: [value1, value2]:
"""
# Function/Agent to run zero-shot prompting for the dataset
def run_zero_shot(test_df):
    predictions, ground_truths = [], []
    for _, row in test_df.iterrows():
        q = row['original'].get('sQuestion', row['original'])
        context = "\n".join(retrieve_relevant_docs(index, q, train_texts))
        prompt = zero_shot_prompt(q, context)
        response = get_openai_response(prompt)
        predictions.append(response)
        ground_truths.append(row['answer'])
        print(f"[ZS] Q: {q}\nA: {response}\nGT: {row['answer']}\n")
    acc = evaluate_accuracy(predictions, ground_truths)
    print(f"Zero-Shot Accuracy: {acc:.2f}%")
run_zero_shot(test_df)
# Prompt Template for Few-shot Prompting
few_shot_examples = train_df.sample(3).to_dict(orient='records')


def few_shot_prompt(question, context, examples):
    shots = ""
    for ex in examples:
        shots += f"Question: {ex['original']['sQuestion']}\nAnswer: {ex['answer']}\n\n"
    return f"""You are a helpful assistant that answers math problems accurately.


Here are some examples:
```

```
{shots}

Context:

{context}


Question:

{question}


final Answer in the format of: [value1, value2]:

"""
# Function/Agent to run few-shot prompting for the dataset
def run_few_shot(test_df):
    predictions, ground_truths = [], []
    for _, row in test_df.iterrows():
        q = row['original'].get('sQuestion', row['original'])
        context = "\n".join(retrieve_relevant_docs(index, q, train_texts))
        prompt = few_shot_prompt(q, context, few_shot_examples)
        response = get_openai_response(prompt)
        predictions.append(response)
        ground_truths.append(row['answer'])
        print(f"[FS] Q: {q}\nA: {response}\nGT: {row['answer']}\n")
    acc = evaluate_accuracy(predictions, ground_truths)
    print(f"Few-Shot Accuracy: {acc:.2f}%")
run_few_shot(test_df)
# Prompt Template for CoT Prompting
def chain_of_thought_prompt(question, context):
```

```
    return f"""You are a helpful assistant that solves math problems by thinking step-by-step. Make sure to clearly express your final answer.

Context:
{context}

Question:
{question}

Solution:
final Answer in the format of: [value1, value2]:
"""
# Function/Agent to run CoT Prompting for the dataset
def run_chain_of_thought(test_df):
    predictions, ground_truths = [], []
    for _, row in test_df.iterrows():
        q = row['original'].get('sQuestion', row['original'])
        context = "\n".join(retrieve_relevant_docs(index, q, train_texts))
        prompt = chain_of_thought_prompt(q, context)
        response = get_openai_response(prompt)
        predictions.append(response)
        ground_truths.append(row['answer'])
        print(f"[CoT] Q: {q}\nA: {response}\nGT: {row['answer']}\n")
    acc = evaluate_accuracy(predictions, ground_truths)
    print(f"Chain-of-Thought Accuracy: {acc:.2f}%")
# Run these agents in separate cells to measure the runtime individually for each model.
run_zero_shot(test_df)
```

```
run_few_shot(test_df)

run_chain_of_thought(test_df)
```

Deepseek-V1

```
# Install Deepseek

pip install deepseek

# Install Dependencies

!pip install -q sentence-transformers faiss-cpu

# Import Libraries

import pandas as pd, numpy as np, re, json, requests, faiss

from sklearn.model_selection import train_test_split

from sentence_transformers import SentenceTransformer

# API Key and URL to deploy the model

API_KEY = "APi key has been removed for security reasons"

API_URL = "https://api.deepseek.com/v1/chat/completions"

# Embedding model

model = SentenceTransformer('all-MiniLM-L6-v2')

# Function/Agent to load dataset and create a dataframe

def load_dataset(file_obj):

    data = []

    for line in file_obj:

        data.append(json.loads(line))

    return pd.DataFrame(data)

# Upload the 5 datasets individually
```

```
from google.colab import files
uploaded = files.upload()


import io
df = load_dataset(io.StringIO(list(uploaded.values())[0].decode('utf-8')))
# Creates a FAISS Index to retrieve similar math problems for few-shot prompting
def create_faiss_index(texts):
    embeddings = model.encode(texts, convert_to_tensor=False)
    embeddings = np.array(embeddings, dtype=np.float32)
    index = faiss.IndexFlatL2(embeddings.shape[1])
    index.add(embeddings)
    return index, texts
# Retrieves contextual cues for prompting
def retrieve_relevant_docs(index, query, texts, k=5):
    query_emb = model.encode([query])
    query_emb = np.array(query_emb, dtype=np.float32)
    _, indices = index.search(query_emb, k)
    return [texts[i] for i in indices[0]]
# DeepSeek call to Deploy the V1 model
def get_deepseek_response(prompt, model="deepseek-chat", temperature=0.2, max_tokens=1000):
    headers = {
        "Authorization": f"Bearer {API_KEY}",
        "Content-Type": "application/json"
    }
    payload = {
        "model": model,
        "messages": [{"role": "user", "content": prompt}],
```

```
        "temperature": temperature,
        "max_tokens": max_tokens
    }
    response = requests.post(API_URL, headers=headers, data=json.dumps(payload))
    response_json = response.json()
    return response_json["choices"][0]["message"]["content"].strip()
# Extracts the final answer as a structured numerical output
def extract_final_answer(response):
    match = re.search(r"\[([\d.,\s\-+]+)\]", response)
    if match:
        numbers = re.findall(r"[-+]?\d*\.\d+|\d+", match.group(1))
        return [float(num) for num in numbers]
    return None
# Evaluates the accuracy with a tolerance of 1% from the ground truth
def evaluate_accuracy(predictions, ground_truths, tolerance=0.01):
    correct = 0
    for pred, gt in zip(predictions, ground_truths):
        pred_nums = extract_final_answer(pred)
        if pred_nums is None: continue
        gt_nums = re.findall(r"[-+]?\d*\.\d+|\d+", str(gt))
        gt_nums = [float(n) for n in gt_nums]
        if len(pred_nums) == len(gt_nums) and all(abs(p - g) <= max(tolerance, 0.01 * abs(g)) for p, g in
zip(pred_nums, gt_nums)):
            correct += 1
    return (correct / len(ground_truths)) * 100
# Prompt Template for Zero-shot Prompting
def zero_shot_prompt(question, context):
```

```
    return f"""You are a helpful assistant that answers math problems accurately.

Context:
{context}

Question:
{question}

final Answer in the format of: [value1, value2]:
"""
# Function/Agent to run zero-shot prompting for the dataset
def run_zero_shot(df):
    train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
    index, train_texts = create_faiss_index(train_df['text'].tolist())
    predictions, ground_truths = [], []

    for _, row in test_df.iterrows():
        question = row['original'].get('sQuestion', row['original'])
        context = "\n".join(retrieve_relevant_docs(index, question, train_texts))
        prompt = zero_shot_prompt(question, context)
        response = get_deepseek_response(prompt)
        predictions.append(response)
        ground_truths.append(row['answer'])
        print(f"[Zero-Shot] Q: {question}\nA: {response}\nGT: {row['answer']}\n")

    acc = evaluate_accuracy(predictions, ground_truths)
    print(f"Zero-Shot Accuracy: {acc:.2f}%")
```

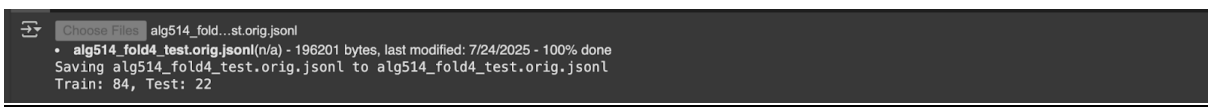
```
# Prompt Template for Few-shot Prompting
def few_shot_prompt(question, context, examples):
    shots = ""
    for ex in examples:
        shots += f"Question: {ex['original'].get('sQuestion', ex['original'])}\nAnswer: {ex['answer']}\n\n"
    return f"""You are a helpful assistant that answers math problems accurately.

Here are some examples:

{shots}
Context:
{context}

Question:
{question}

final Answer in the format of: [value1, value2]:
"""
# Function/Agent to run few-shot prompting for the dataset
def run_few_shot(df):
    train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
    few_shot_examples = train_df.sample(3).to_dict(orient='records')
    index, train_texts = create_faiss_index(train_df['text'].tolist())
    predictions, ground_truths = [], []
    for _, row in test_df.iterrows():
        question = row['original'].get('sQuestion', row['original'])
        context = "\n".join(retrieve_relevant_docs(index, question, train_texts))
```

```
        prompt = few_shot_prompt(question, context, few_shot_examples)
        response = get_deepseek_response(prompt)
        predictions.append(response)
        ground_truths.append(row['answer'])
        print(f"[Few-Shot] Q: {question}\nA: {response}\nGT: {row['answer']}\n")

    acc = evaluate_accuracy(predictions, ground_truths)
    print(f"Few-Shot Accuracy: {acc:.2f}%")
# Prompt Template for CoT Prompting
def chain_of_thought_prompt(question, context):
    return f"""You are a helpful assistant that solves math problems by thinking step-by-step. Make
sure to clearly express your final answer.

Context:
{context}

Question:
{question}

Solution:

final Answer in the format of: [value1, value2]:
"""
# Function/Agent to run CoT prompting for the dataset
def run_chain_of_thought(df):
    train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
    index, train_texts = create_faiss_index(train_df['text'].tolist())
```

```
    predictions, ground_truths = [], []

    for _, row in test_df.iterrows():
        question = row['original'].get('sQuestion', row['original'])
        context = "\n".join(retrieve_relevant_docs(index, question, train_texts))
        prompt = chain_of_thought_prompt(question, context)
        response = get_deepseek_response(prompt)
        predictions.append(response)
        ground_truths.append(row['answer'])
        print(f"[CoT] Q: {question}\nA: {response}\nGT: {row['answer']}\n")

    acc = evaluate_accuracy(predictions, ground_truths)
    print(f"Chain-of-Thought Accuracy: {acc:.2f}%")

# Run these agents in separate cells to measure the runtime individually for each model.
run_zero_shot(df)
run_few_shot(df)
run_chain_of_thought(df)
```

Sample vector embeddings:

OpenAI:

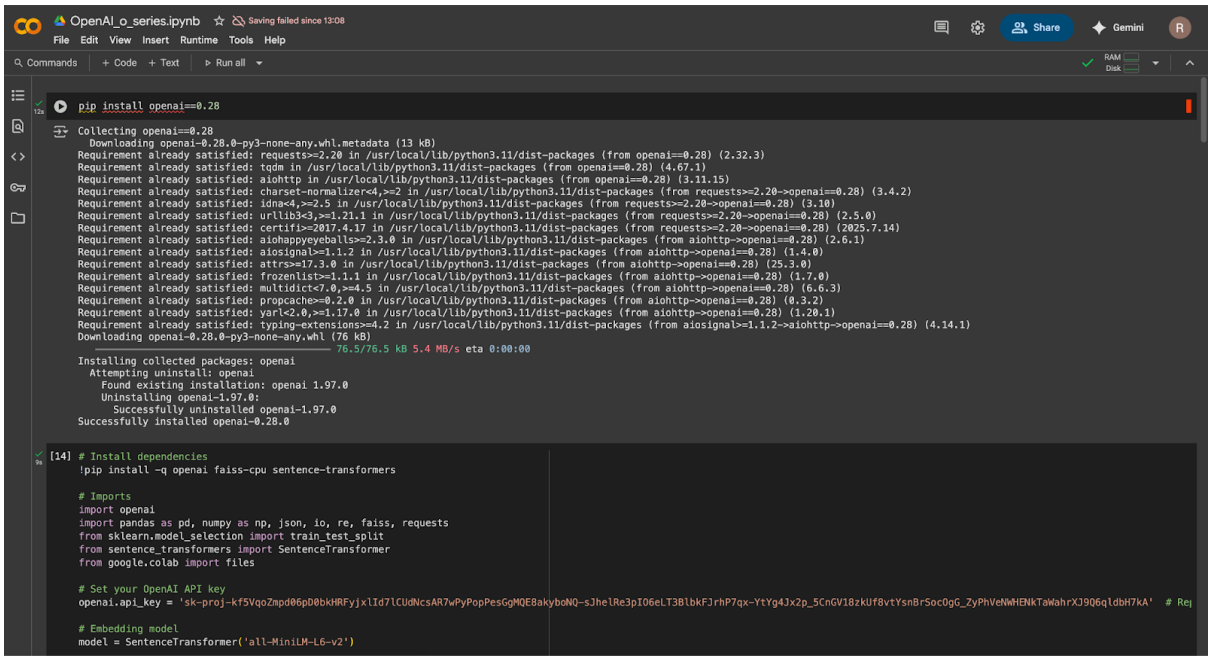


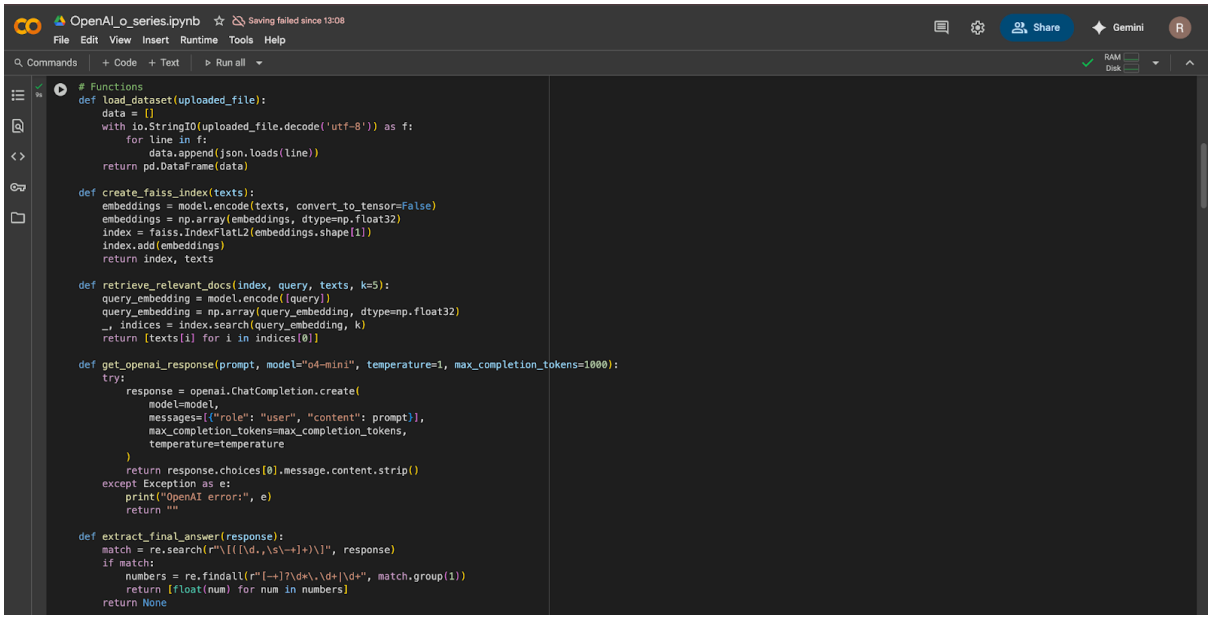
Deepseek:

```
• alg514_fold3_test.orig.jsonl(n/a) - 176931 bytes, last modified: 7/24/2025 - 100% done
Saving alg514_fold3_test.orig.jsonl to alg514_fold3_test.orig (1).jsonl
Train: 81, Test: 21
```

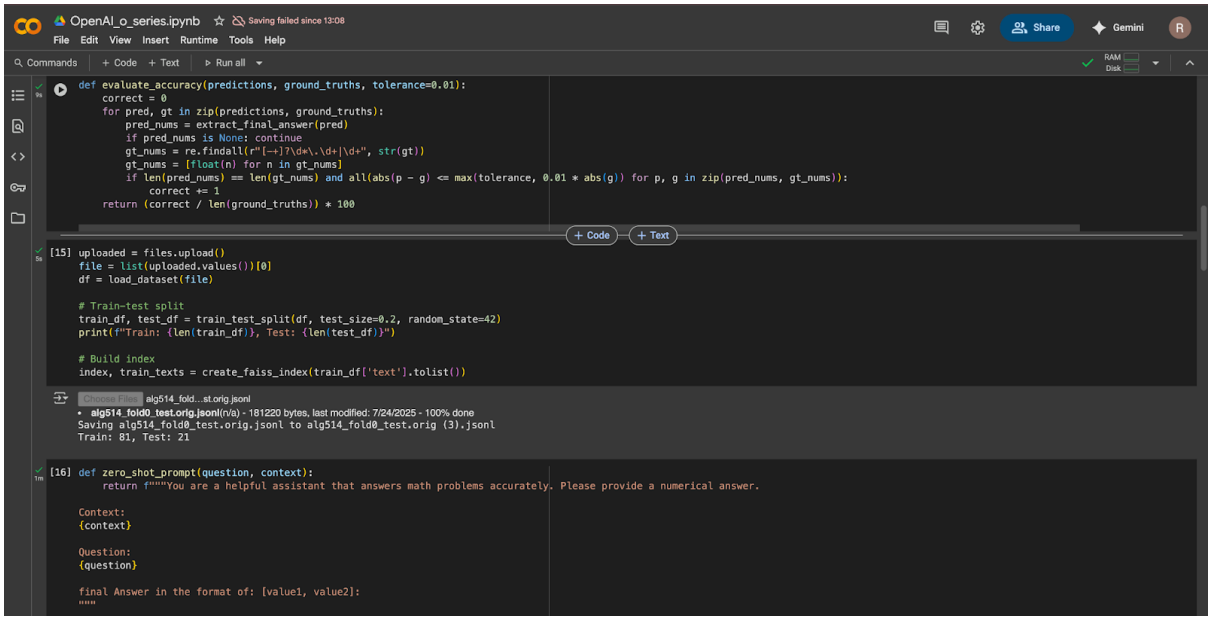
Sample Outputs:

OpenAI GPT 04-mini with Dataset 0:





```
# Functions
def load_dataset(uploaded_file):
    data = []
    with io.StringIO(uploaded_file.decode('utf-8')) as f:
        for line in f:
            data.append(json.loads(line))
    return pd.DataFrame(data)

def create_faiss_index(texts):
    embeddings = model.encode(texts, convert_to_tensor=False)
    embeddings = np.array(embeddings, dtype=np.float32)
    index = faiss.IndexFlatL2(embeddings.shape[1])
    index.add(embeddings)
    return index, texts

def retrieve_relevant_docs(index, query, texts, k=5):
    query_embedding = model.encode([query])
    query_embedding = np.array(query_embedding, dtype=np.float32)
    _, indices = index.search(query_embedding, k)
    return [texts[i] for i in indices[0]]

def get_openai_response(prompt, model="o4-mini", temperature=1, max_completion_tokens=1000):
    try:
        response = openai.ChatCompletion.create(
            model=model,
            messages=[{"role": "user", "content": prompt}],
            max_completion_tokens=max_completion_tokens,
            temperature=temperature
        )
        return response.choices[0].message.content.strip()
    except Exception as e:
        print("OpenAI error:", e)
        return ""

def extract_final_answer(response):
    match = re.search(r"\[([\d.,\s\-+]+)\]", response)
    if match:
        numbers = re.findall(r"[-+]?\d*\.\d+|\d+", match.group(1))
        return [float(num) for num in numbers]
    return None
```

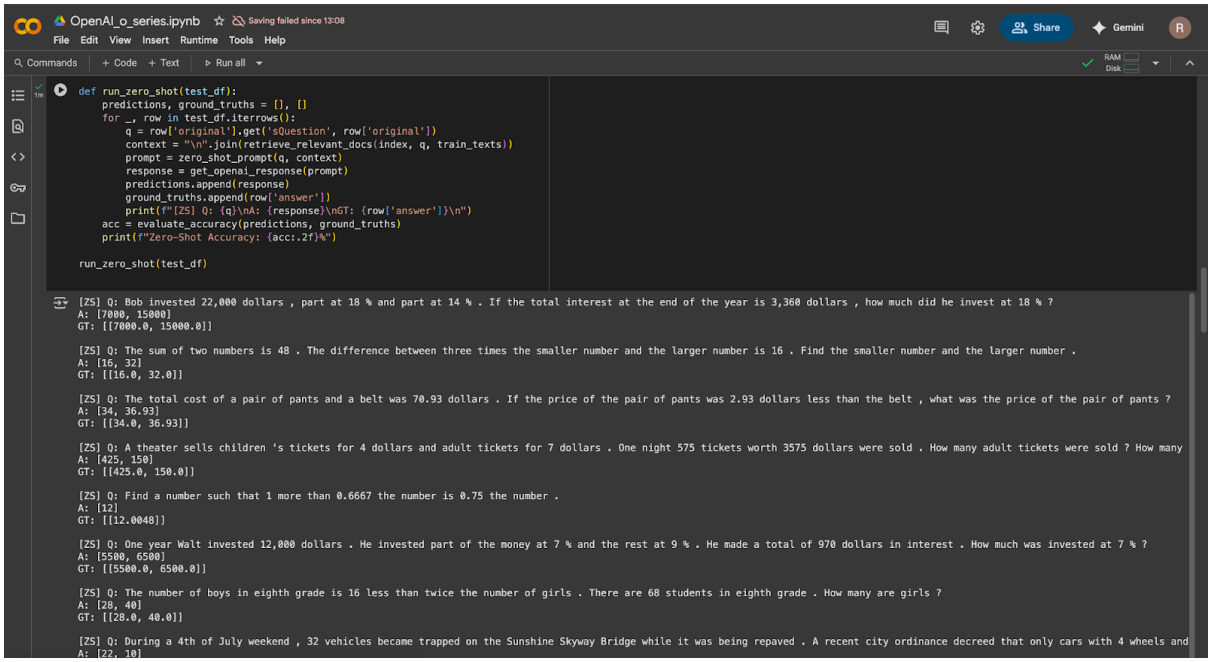


```
def evaluate_accuracy(predictions, ground_truths, tolerance=0.01):
    correct = 0
    for pred, gt in zip(predictions, ground_truths):
        pred_nums = extract_final_answer(pred)
        if pred_nums is None: continue
        gt_nums = re.findall(r"[-+]?\d*\.\d+|\d+", str(gt))
        gt_nums = [float(n) for n in gt_nums]
        if len(pred_nums) == len(gt_nums) and all(abs(p - g) <= max(tolerance, 0.01 * abs(g)) for p, g in zip(pred_nums, gt_nums)):
            correct += 1
    return (correct / len(ground_truths)) * 100
```

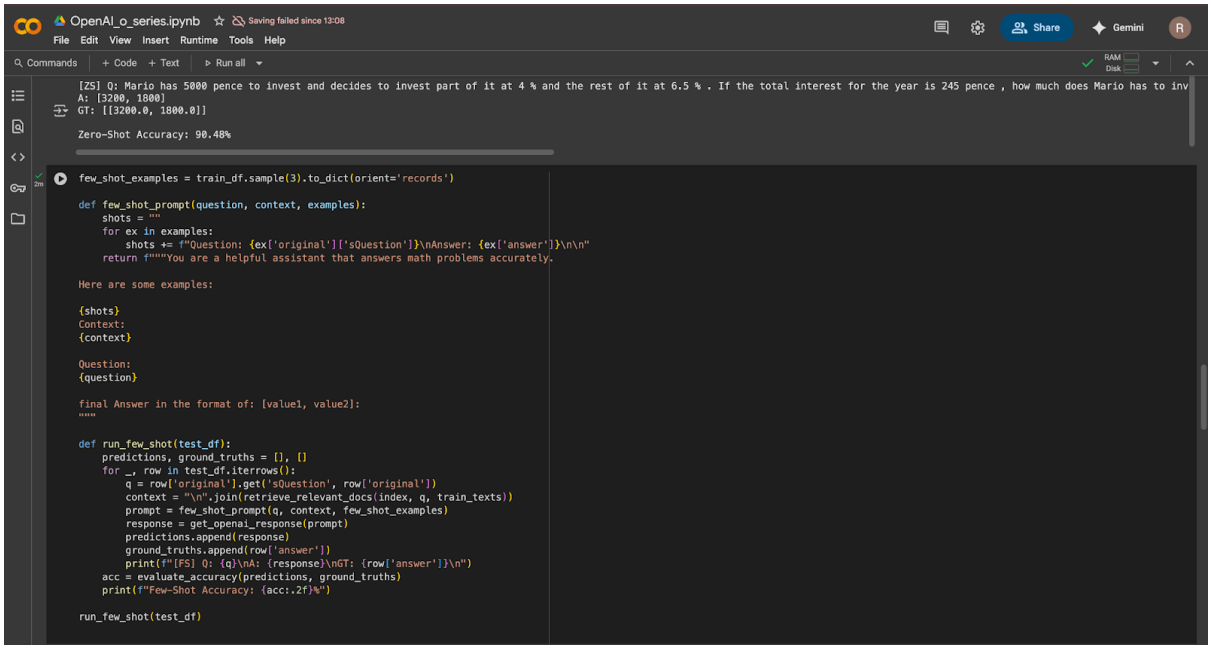
```
[15] uploaded = files.upload()
    file = list(uploaded.values())[0]
    df = load_dataset(file)

    # Train-test split
    train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
    print(f"Train: {len(train_df)}, Test: {len(test_df)}")

    # Build index
    index, train_texts = create_faiss_index(train_df['text'].tolist())
```

```
Choose Files  alg514_fold...st.orig.jsonl
• alg514_fold0_test.orig.jsonl(n/a) - 181220 bytes, last modified: 7/24/2025 - 100% done
Saving alg514_fold0_test.orig.jsonl to alg514_fold0_test.orig (3).jsonl
Train: 81, Test: 21
```

```
[16] def zero_shot_prompt(question, context):
        return f"""You are a helpful assistant that answers math problems accurately. Please provide a numerical answer.

    Context:
    {context}

    Question:
    {question}

    final Answer in the format of: [value1, value2]:
    """
```

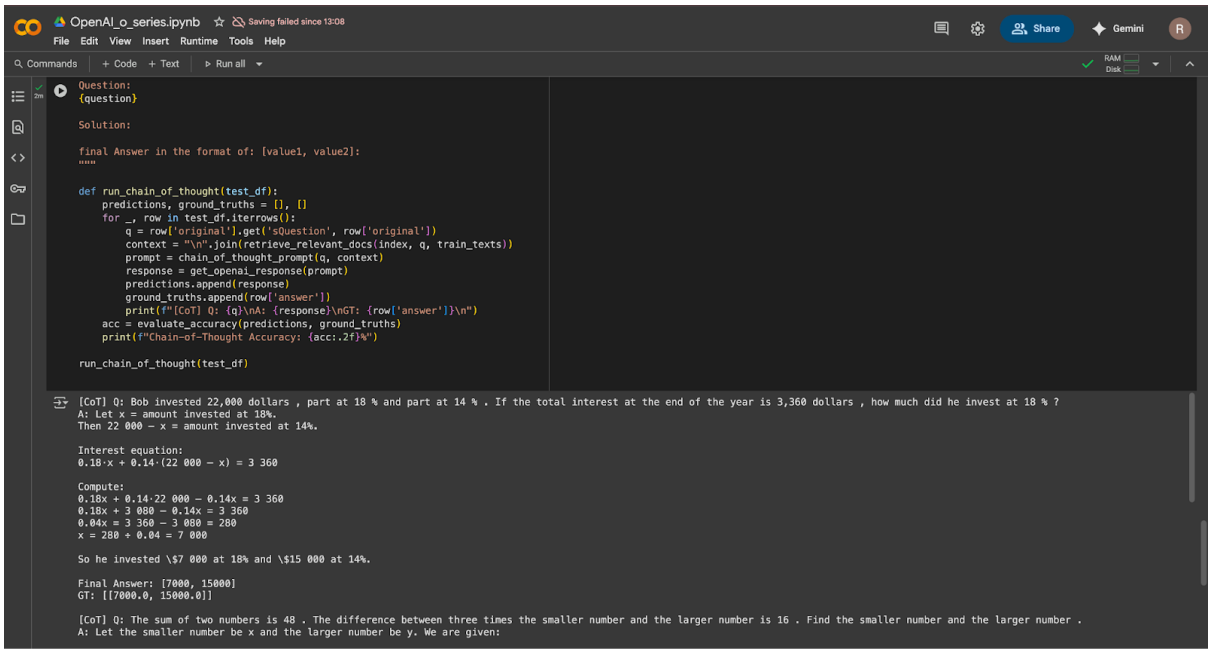

OpenAI_o_series.ipynb
Saving failed since 13:08
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
def run_zero_shot(test_df):
predictions, ground_truths = [], []
for _, row in test_df.iterrows():
q = row['original'].get('sQuestion', row['original'])
context = "\n".join(retrieve_relevant_docs(index, q, train_texts))
prompt = zero_shot_prompt(q, context)
response = get_openai_response(prompt)
predictions.append(response)
ground_truths.append(row['answer'])
print(f"[ZS] Q: {q}\nA: {response}\nGT: {row['answer']}\n")
acc = evaluate_accuracy(predictions, ground_truths)
print(f"Zero-Shot Accuracy: {acc:.2f}%")
run_zero_shot(test_df)
[ZS] Q: Bob invested 22,000 dollars , part at 18 % and part at 14 % . If the total interest at the end of the year is 3,360 dollars , how much did he invest at 18 % ?
A: [7000, 15000]
GT: [[7000.0, 15000.0]]
[ZS] Q: The sum of two numbers is 48 . The difference between three times the smaller number and the larger number is 16 . Find the smaller number and the larger number .
A: [16, 32]
GT: [[16.0, 32.0]]
[ZS] Q: The total cost of a pair of pants and a belt was 70.93 dollars . If the price of the pair of pants was 2.93 dollars less than the belt , what was the price of the pair of pants ?
A: [34, 36.93]
GT: [[34.0, 36.93]]
[ZS] Q: A theater sells children 's tickets for 4 dollars and adult tickets for 7 dollars . One night 575 tickets worth 3575 dollars were sold . How many adult tickets were sold ? How many
A: [425, 150]
GT: [[425.0, 150.0]]
[ZS] Q: Find a number such that 1 more than 0.6667 the number is 0.75 the number .
A: [12]
GT: [[12.0048]]
[ZS] Q: One year Walt invested 12,000 dollars . He invested part of the money at 7 % and the rest at 9 % . He made a total of 970 dollars in interest . How much was invested at 7 % ?
A: [5500, 6500]
GT: [[5500.0, 6500.0]]
[ZS] Q: The number of boys in eighth grade is 16 less than twice the number of girls . There are 68 students in eighth grade . How many are girls ?
A: [28, 40]
GT: [[28.0, 40.0]]
[ZS] Q: During a 4th of July weekend , 32 vehicles became trapped on the Sunshine Skyway Bridge while it was being repaved . A recent city ordinance decreed that only cars with 4 wheels and
A: [22, 10]

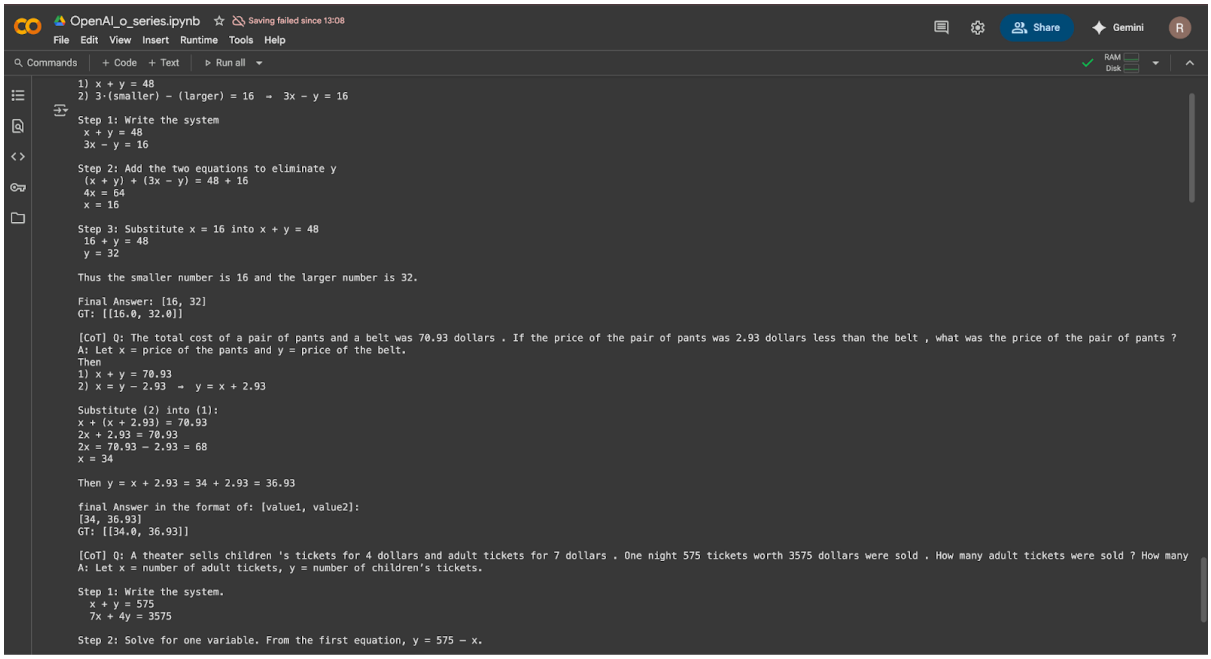

OpenAI_o_series.ipynb
Saving failed since 13:08
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
[ZS] Q: Lisa invested 8000 dollars , part at 4 % and the rest at 5 % per year . How much did she invest at 4 % and 5 % if her total return per year on the investments was 380 dollars ?
A: [2000, 6000]
GT: [[2000.0, 6000.0]]
[ZS] Q: Suppose you invested 10,000 dollars , part at 6 % annual interest and the rest at 9 % annual interest . If you received 684 dollars in interest after one year , how much did you inv
A: [7200, 2800]
GT: [[7200.0, 2800.0]]
[ZS] Q: A merchant wants to combine peanuts selling 2.40 dollars per pound and cashews selling for 6.00 dollars per pound to form 60 pounds which will sell at 3.00 dollars per pound . What
A: [50, 10]
GT: [[50.0, 10.0]]
[ZS] Q: Ilida went to Minewaska Sate Park one day this summer . All of the people at the park were either hiking or bike riding . There were 178 more hikers than bike riders . If there were
A: [427, 249]
GT: [[427.0, 249.0]]
[ZS] Q: 3 Pairs of jeans and 6 shirts costs 104.25 dollars . The cost of 4 jeans and 5 shirts is 112.15 dollars . Find the cost of each pair of jeans and each shirt .
A: [16.85, 8.95]
GT: [[16.85, 8.95]]
[ZS] Q: One unit of whole wheat flour has 13.6 grams of protein and 2.5 grams of fat . One unit of whole milk has 3.4 grams of protein and 3.7 grams of fat . To the nearest tenth , find the
A:
GT: [[5.416068866571019, 0.39454806312769]]
[ZS] Q: 2 high speed trains are 380 miles apart and traveling toward each other . They meet in 4 hours . If one train is 15 miles per hour faster than the other , find the speed of the slow
A: [40, 55]
GT: [[40.0, 55.0]]
[ZS] Q: 24 is divided in two parts such that 7 time the first part added to 5 times the second part makes 146 . What is the first part ?
A: [13, 11]
GT: [[13.0, 11.0]]
[ZS] Q: A department store held a sale to sell all of the 214 winter jackets that remained after the season ended . Until noon , each jacket in the store was priced at 31.95 dollars . At no
A: [81, 133]
GT: [[81.0, 133.0]]
[ZS] Q: An art dealer sold 16 etchings for 630 dollars . He sold some of them at 35 dollars each and the rest at 45 dollars each . How many etchings did he sell at 35 dollars ? How many etc
A: [9, 7]
GT: [[9.0, 7.0]]
[ZS] Q: In a family there are 2 cars . The sum of the average miles per gallon obtained by the two cars in a particular week is 45 . The first car has consumed 40 gallons during that week ,
A: [25, 20]
GT: [[25.0, 20.0]]
[ZS] Q: 2 trains leave simultaneously traveling on the same track in opposite directions at speeds of 50 and 64 miles per hour . How many hours will it take before they are 285 miles apart
A: [2, 30]
GT: [[2.5]]


OpenAI_o_series.ipynb
Saving failed since 13:08
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
[ZS] Q: Mario has 5000 pence to invest and decides to invest part of it at 4 % and the rest of it at 6.5 % . If the total interest for the year is 245 pence , how much does Mario has to inv
A: [3200, 1800]
GT: [[3200.0, 1800.0]]
Zero-Shot Accuracy: 90.48%
few_shot_examples = train_df.sample(3).to_dict(orient='records')
def few_shot_prompt(question, context, examples):
shots = ""
for ex in examples:
shots += f"Question: {ex['original']['sQuestion']}\nAnswer: {ex['answer']}\n\n"
return f"""You are a helpful assistant that answers math problems accurately.
Here are some examples:
{shots}
Context:
{context}
Question:
{question}
final Answer in the format of: [value1, value2]:
"""
def run_few_shot(test_df):
predictions, ground_truths = [], []
for _, row in test_df.iterrows():
q = row['original'].get('sQuestion', row['original'])
context = "\n".join(retrieve_relevant_docs(index, q, train_texts))
prompt = few_shot_prompt(q, context, few_shot_examples)
response = get_openai_response(prompt)
predictions.append(response)
ground_truths.append(row['answer'])
print(f"[FS] Q: {q}\nA: {response}\nGT: {row['answer']}\n")
acc = evaluate_accuracy(predictions, ground_truths)
print(f"Few-Shot Accuracy: {acc:.2f}%")
run_few_shot(test_df)


OpenAI_o_series.ipynb
Saving failed since 13:08
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
[FS] Q: Bob invested 22,000 dollars , part at 18 % and part at 14 % . If the total interest at the end of the year is 3,360 dollars , how much did he invest at 18 % ?
A: [7000.0, 15000.0]
GT: [[7000.0, 15000.0]]
[FS] Q: The sum of two numbers is 48 . The difference between three times the smaller number and the larger number is 16 . Find the smaller number and the larger number .
A: [16.0, 32.0]
GT: [[16.0, 32.0]]
[FS] Q: The total cost of a pair of pants and a belt was 70.93 dollars . If the price of the pair of pants was 2.93 dollars less than the belt , what was the price of the pair of pants ?
A: [34.0, 36.93]
GT: [[34.0, 36.93]]
[FS] Q: A theater sells children 's tickets for 4 dollars and adult tickets for 7 dollars . One night 575 tickets worth 3575 dollars were sold . How many adult tickets were sold ? How many
A: [425, 150]
GT: [[425.0, 150.0]]
[FS] Q: Find a number such that 1 more than 0.6667 the number is 0.75 the number .
A: [12.0]
GT: [[12.0048]]
[FS] Q: One year Walt invested 12,000 dollars . He invested part of the money at 7 % and the rest at 9 % . He made a total of 970 dollars in interest . How much was invested at 7 % ?
A: [5500.0, 6500.0]
GT: [[5500.0, 6500.0]]
[FS] Q: The number of boys in eighth grade is 16 less than twice the number of girls . There are 68 students in eighth grade . How many are girls ?
A: [28.0, 40.0]
GT: [[28.0, 40.0]]
[FS] Q: During a 4th of July weekend , 32 vehicles became trapped on the Sunshine Skyway Bridge while it was being repaved . A recent city ordinance decreed that only cars with 4 wheels and
A: [22.0, 10.0]
GT: [[22.0, 10.0]]
[FS] Q: Lisa invested 8000 dollars , part at 4 % and the rest at 5 % per year . How much did she invest at 4 % and 5 % if her total return per year on the investments was 380 dollars ?
A: [2000.0, 6000.0]
GT: [[2000.0, 6000.0]]
[FS] Q: Suppose you invested 10,000 dollars , part at 6 % annual interest and the rest at 9 % annual interest . If you received 684 dollars in interest after one year , how much did you inv
A: [7200.0, 2800.0]
GT: [[7200.0, 2800.0]]
[FS] Q: A merchant wants to combine peanuts selling 2.40 dollars per pound and cashews selling for 6.00 dollars per pound to form 60 pounds which will sell at 3.00 dollars per pound . What
A: [50.0, 10.0]
GT: [[50.0, 10.0]]
[FS] Q: Ilida went to Minewaska Sate Park one day this summer . All of the people at the park were either hiking or bike riding . There were 178 more hikers than bike riders . If there were
A: [427.0, 249.0]
GT: [[427.0, 249.0]]


OpenAI_o_series.ipynb
Saving failed since 13:08
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
[FS] Q: 3 Pairs of jeans and 6 shirts costs 104.25 dollars . The cost of 4 jeans and 5 shirts is 112.15 dollars . Find the cost of each pair of jeans and each shirt .
A: [16.85, 8.95]
GT: [[16.85, 8.95]]
[FS] Q: One unit of whole wheat flour has 13.6 grams of protein and 2.5 grams of fat . One unit of whole milk has 3.4 grams of protein and 3.7 grams of fat . To the nearest tenth , find the
A:
GT: [[5.416068866571019, 0.39454806312769]]
[FS] Q: 2 high speed trains are 380 miles apart and traveling toward each other . They meet in 4 hours . If one train is 15 miles per hour faster than the other , find the speed of the slow
A: [40.0, 55.0]
GT: [[40.0, 55.0]]
[FS] Q: 24 is divided in two parts such that 7 time the first part added to 5 times the second part makes 146 . What is the first part ?
A: [13.0, 11.0]
GT: [[13.0, 11.0]]
[FS] Q: A department store held a sale to sell all of the 214 winter jackets that remained after the season ended . Until noon , each jacket in the store was priced at 31.95 dollars . At no
A: [81.0, 133.0]
GT: [[81.0, 133.0]]
[FS] Q: An art dealer sold 16 etchings for 630 dollars . He sold some of them at 35 dollars each and the rest at 45 dollars each . How many etchings did he sell at 35 dollars ? How many etc
A: [9, 7]
GT: [[9.0, 7.0]]
[FS] Q: In a family there are 2 cars . The sum of the average miles per gallon obtained by the two cars in a particular week is 45 . The first car has consumed 40 gallons during that week ,
A: [25.0, 20.0]
GT: [[25.0, 20.0]]
[FS] Q: 2 trains leave simultaneously traveling on the same track in opposite directions at speeds of 50 and 64 miles per hour . How many hours will it take before they are 285 miles apart
A: [2.5]
GT: [[2.5]]
[FS] Q: Mario has 5000 pence to invest and decides to invest part of it at 4 % and the rest of it at 6.5 % . If the total interest for the year is 245 pence , how much does Mario has to inv
A: [3200.0, 1800.0]
GT: [[3200.0, 1800.0]]
Few-Shot Accuracy: 95.24%
def chain_of_thought_prompt(question, context):
return f"""You are a helpful assistant that solves math problems by thinking step-by-step. Make sure to clearly express your final answer.
Context:
{context}


OpenAI_o_series.ipynb
Saving failed since 13:08
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
Question:
{question}
Solution:
final Answer in the format of: [value1, value2]:
"""
def run_chain_of_thought(test_df):
predictions, ground_truths = [], []
for _, row in test_df.iterrows():
q = row['original'].get('sQuestion', row['original'])
context = "\n".join(retrieve_relevant_docs(index, q, train_texts))
prompt = chain_of_thought_prompt(q, context)
response = get_openai_response(prompt)
predictions.append(response)
ground_truths.append(row['answer'])
print(f"[CoT] Q: {q}\nA: {response}\nGT: {row['answer']}\n")
acc = evaluate_accuracy(predictions, ground_truths)
print(f"Chain-of-Thought Accuracy: {acc:.2f}%")
run_chain_of_thought(test_df)
[CoT] Q: Bob invested 22,000 dollars , part at 18 % and part at 14 % . If the total interest at the end of the year is 3,360 dollars , how much did he invest at 18 % ?
A: Let x = amount invested at 18%.
Then 22 000 − x = amount invested at 14%.
Interest equation:
0.18·x + 0.14·(22 000 − x) = 3 360
Compute:
0.18x + 0.14·22 000 − 0.14x = 3 360
0.18x + 3 080 − 0.14x = 3 360
0.04x = 3 360 − 3 080 = 280
x = 280 ÷ 0.04 = 7 000
So he invested \$7 000 at 18% and \$15 000 at 14%.
Final Answer: [7000, 15000]
GT: [[7000.0, 15000.0]]
[CoT] Q: The sum of two numbers is 48 . The difference between three times the smaller number and the larger number is 16 . Find the smaller number and the larger number .
A: Let the smaller number be x and the larger number be y. We are given:



OpenAI_o_series.ipynb — Saving failed since 13:08

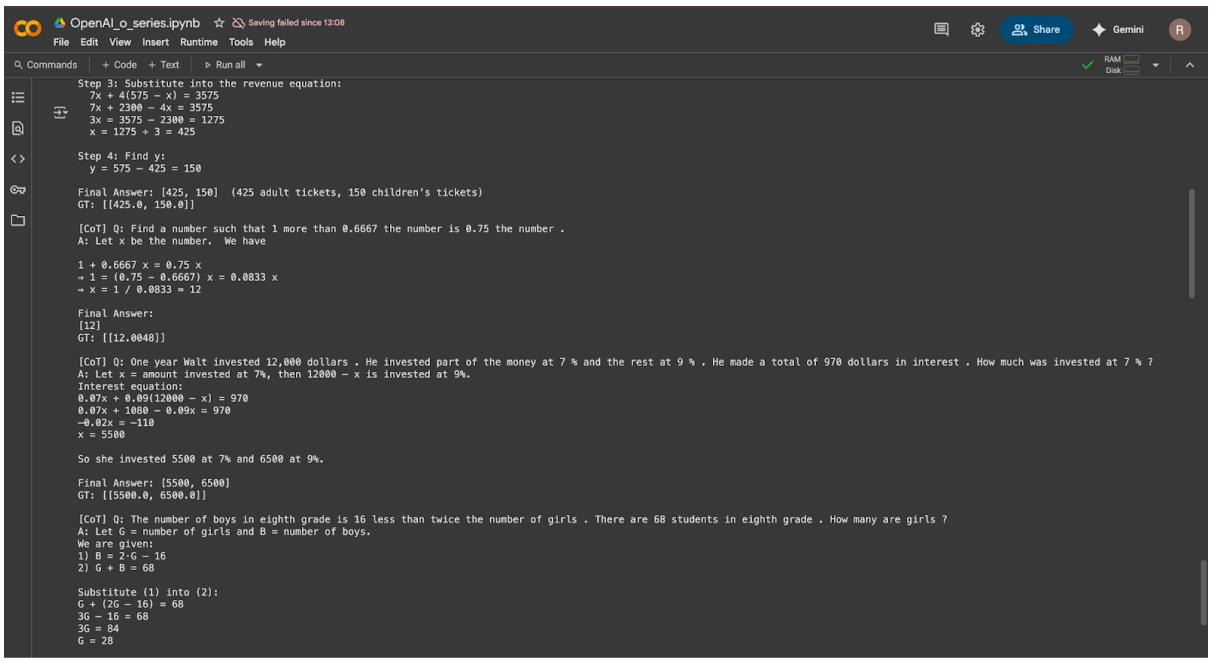
File Edit View Insert Runtime Tools Help

Share Gemini

Commands + Code + Text Run all

RAM Disk

```
1) x + y = 48
2) 3·(smaller) − (larger) = 16  →  3x − y = 16

Step 1: Write the system
 x + y = 48
 3x − y = 16

Step 2: Add the two equations to eliminate y
 (x + y) + (3x − y) = 48 + 16
 4x = 64
 x = 16

Step 3: Substitute x = 16 into x + y = 48
 16 + y = 48
 y = 32

Thus the smaller number is 16 and the larger number is 32.

Final Answer: [16, 32]
GT: [[16.0, 32.0]]

[CoT] Q: The total cost of a pair of pants and a belt was 70.93 dollars . If the price of the pair of pants was 2.93 dollars less than the belt , what was the price of the pair of pants ?
A: Let x = price of the pants and y = price of the belt.
Then
1) x + y = 70.93
2) x = y − 2.93  →  y = x + 2.93

Substitute (2) into (1):
x + (x + 2.93) = 70.93
2x + 2.93 = 70.93
2x = 70.93 − 2.93 = 68
x = 34

Then y = x + 2.93 = 34 + 2.93 = 36.93

final Answer in the format of: [value1, value2]:
[34, 36.93]
GT: [[34.0, 36.93]]

[CoT] Q: A theater sells children 's tickets for 4 dollars and adult tickets for 7 dollars . One night 575 tickets worth 3575 dollars were sold . How many adult tickets were sold ? How many
A: Let x = number of adult tickets, y = number of children's tickets.

Step 1: Write the system.
  x + y = 575
  7x + 4y = 3575

Step 2: Solve for one variable. From the first equation, y = 575 − x.
```



OpenAI_o_series.ipynb — Saving failed since 13:08

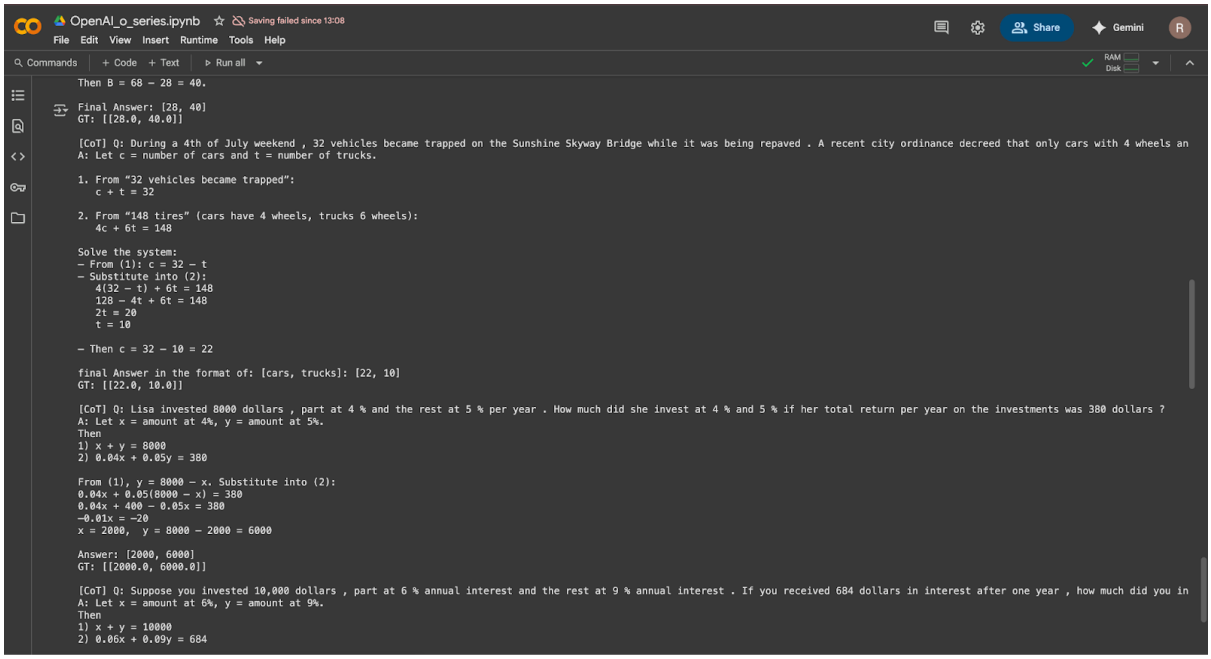
File Edit View Insert Runtime Tools Help

Share Gemini

Commands + Code + Text Run all

RAM Disk

```
Step 3: Substitute into the revenue equation:
  7x + 4(575 − x) = 3575
  7x + 2300 − 4x = 3575
  3x = 3575 − 2300 = 1275
  x = 1275 ÷ 3 = 425

Step 4: Find y:
  y = 575 − 425 = 150

Final Answer: [425, 150]  (425 adult tickets, 150 children's tickets)
GT: [[425.0, 150.0]]

[CoT] Q: Find a number such that 1 more than 0.6667 the number is 0.75 the number .
A: Let x be the number.  We have

1 + 0.6667 x = 0.75 x
→ 1 = (0.75 − 0.6667) x = 0.0833 x
→ x = 1 / 0.0833 ≈ 12

Final Answer:
[12]
GT: [[12.0048]]

[CoT] Q: One year Walt invested 12,000 dollars . He invested part of the money at 7 % and the rest at 9 % . He made a total of 970 dollars in interest . How much was invested at 7 % ?
A: Let x = amount invested at 7%, then 12000 − x is invested at 9%.
Interest equation:
0.07x + 0.09(12000 − x) = 970
0.07x + 1080 − 0.09x = 970
−0.02x = −110
x = 5500

So she invested 5500 at 7% and 6500 at 9%.

Final Answer: [5500, 6500]
GT: [[5500.0, 6500.0]]

[CoT] Q: The number of boys in eighth grade is 16 less than twice the number of girls . There are 68 students in eighth grade . How many are girls ?
A: Let G = number of girls and B = number of boys.
We are given:
1) B = 2·G − 16
2) G + B = 68

Substitute (1) into (2):
G + (2G − 16) = 68
3G − 16 = 68
3G = 84
G = 28
```

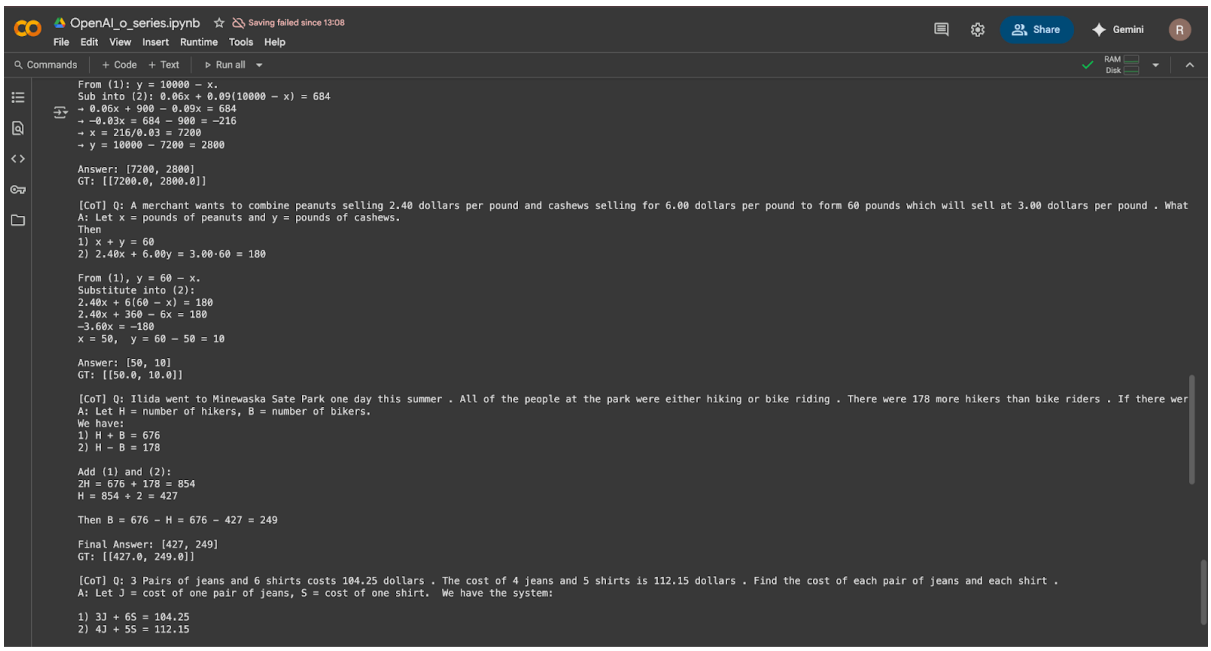


OpenAI_o_series.ipynb — Saving failed since 13:08

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all

```
Then B = 68 − 28 = 40.

Final Answer: [28, 40]
GT: [[28.0, 40.0]]

[CoT] Q: During a 4th of July weekend , 32 vehicles became trapped on the Sunshine Skyway Bridge while it was being repaved . A recent city ordinance decreed that only cars with 4 wheels an
A: Let c = number of cars and t = number of trucks.

1. From “32 vehicles became trapped”:
   c + t = 32

2. From “148 tires” (cars have 4 wheels, trucks 6 wheels):
   4c + 6t = 148

Solve the system:
− From (1): c = 32 − t
− Substitute into (2):
   4(32 − t) + 6t = 148
   128 − 4t + 6t = 148
   2t = 20
   t = 10

− Then c = 32 − 10 = 22

final Answer in the format of: [cars, trucks]: [22, 10]
GT: [[22.0, 10.0]]

[CoT] Q: Lisa invested 8000 dollars , part at 4 % and the rest at 5 % per year . How much did she invest at 4 % and 5 % if her total return per year on the investments was 380 dollars ?
A: Let x = amount at 4%, y = amount at 5%.
Then
1) x + y = 8000
2) 0.04x + 0.05y = 380

From (1), y = 8000 − x. Substitute into (2):
0.04x + 0.05(8000 − x) = 380
0.04x + 400 − 0.05x = 380
−0.01x = −20
x = 2000,  y = 8000 − 2000 = 6000

Answer: [2000, 6000]
GT: [[2000.0, 6000.0]]

[CoT] Q: Suppose you invested 10,000 dollars , part at 6 % annual interest and the rest at 9 % annual interest . If you received 684 dollars in interest after one year , how much did you in
A: Let x = amount at 6%, y = amount at 9%.
Then
1) x + y = 10000
2) 0.06x + 0.09y = 684
```

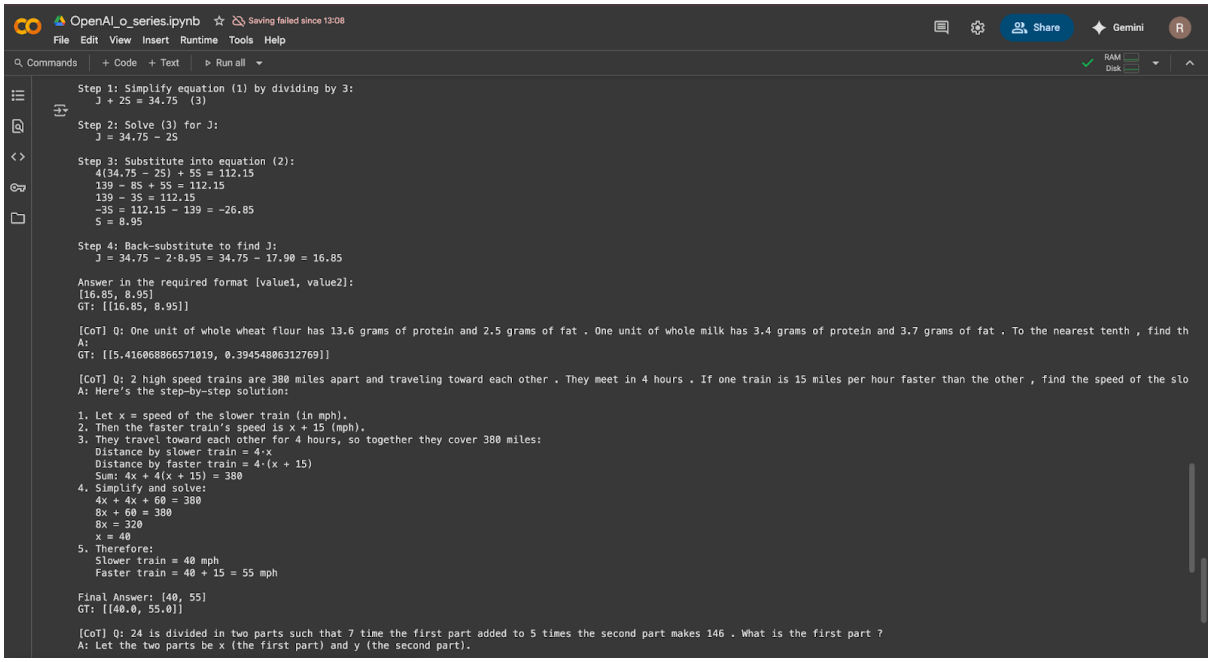


OpenAI_o_series.ipynb — Saving failed since 13:08

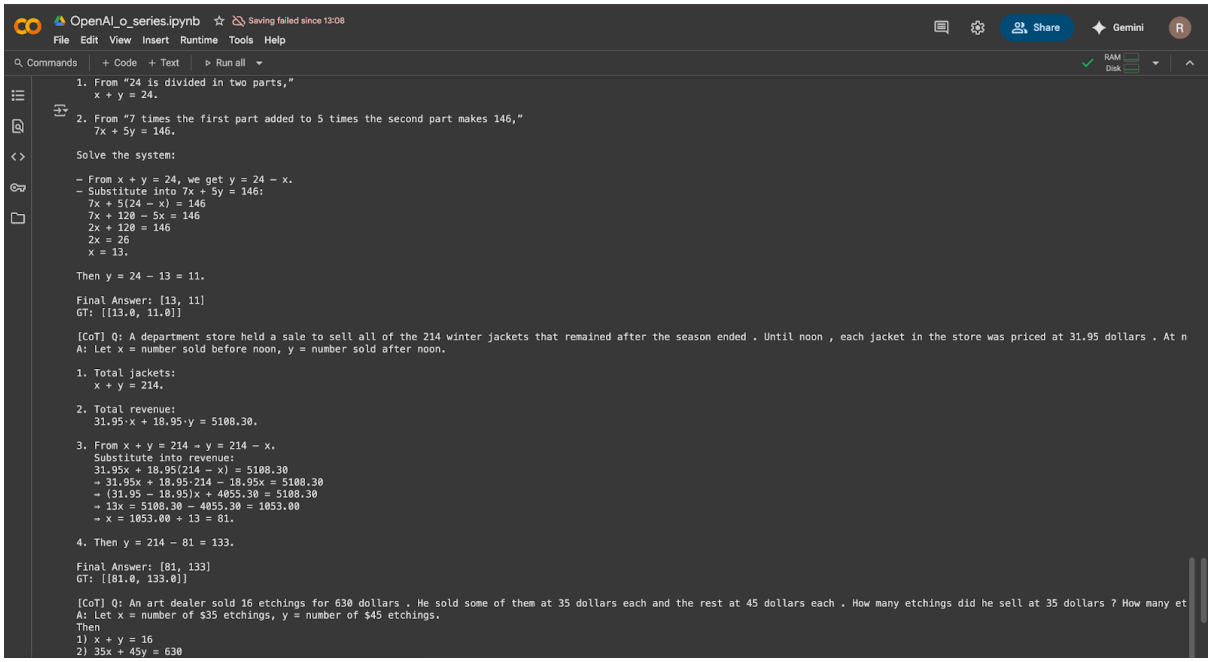
File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all

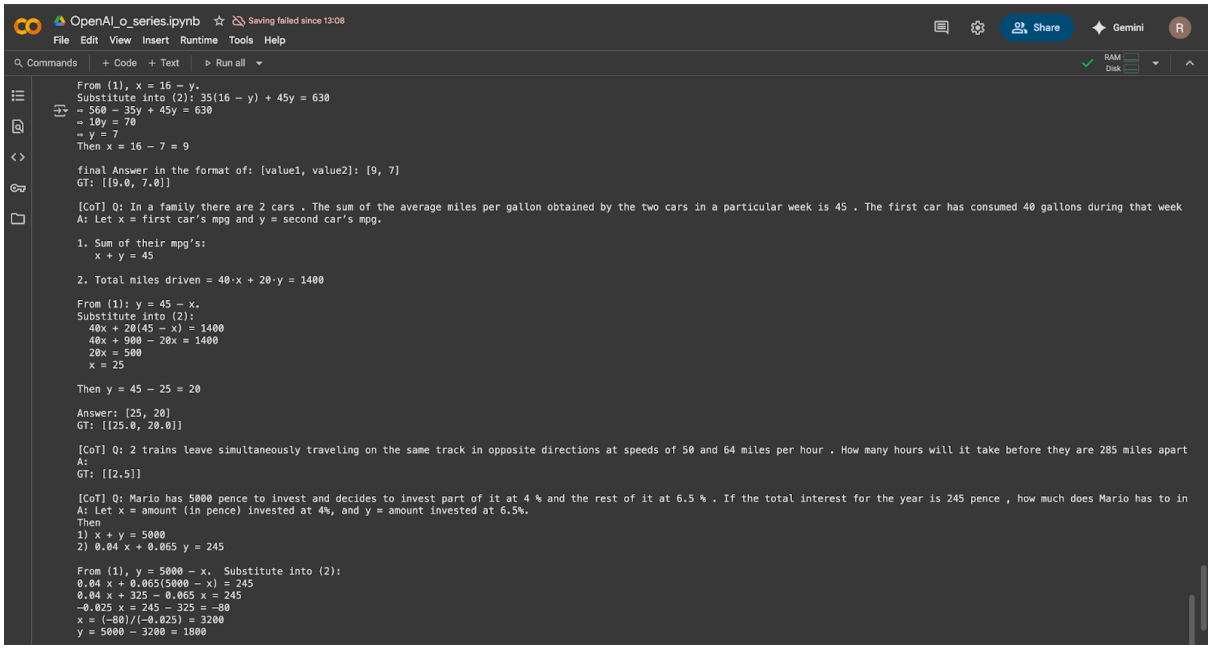
```
From (1): y = 10000 − x.
Sub into (2): 0.06x + 0.09(10000 − x) = 684
→ 0.06x + 900 − 0.09x = 684
→ −0.03x = 684 − 900 = −216
→ x = 216/0.03 = 7200
→ y = 10000 − 7200 = 2800

Answer: [7200, 2800]
GT: [[7200.0, 2800.0]]

[CoT] Q: A merchant wants to combine peanuts selling 2.40 dollars per pound and cashews selling for 6.00 dollars per pound to form 60 pounds which will sell at 3.00 dollars per pound . What
A: Let x = pounds of peanuts and y = pounds of cashews.
Then
1) x + y = 60
2) 2.40x + 6.00y = 3.00·60 = 180

From (1), y = 60 − x.
Substitute into (2):
2.40x + 6(60 − x) = 180
2.40x + 360 − 6x = 180
−3.60x = −180
x = 50,  y = 60 − 50 = 10

Answer: [50, 10]
GT: [[50.0, 10.0]]

[CoT] Q: Ilida went to Minewaska Sate Park one day this summer . All of the people at the park were either hiking or bike riding . There were 178 more hikers than bike riders . If there wer
A: Let H = number of hikers, B = number of bikers.
We have:
1) H + B = 676
2) H − B = 178

Add (1) and (2):
2H = 676 + 178 = 854
H = 854 ÷ 2 = 427

Then B = 676 − H = 676 − 427 = 249

Final Answer: [427, 249]
GT: [[427.0, 249.0]]

[CoT] Q: 3 Pairs of jeans and 6 shirts costs 104.25 dollars . The cost of 4 jeans and 5 shirts is 112.15 dollars . Find the cost of each pair of jeans and each shirt .
A: Let J = cost of one pair of jeans, S = cost of one shirt.  We have the system:

1) 3J + 6S = 104.25
2) 4J + 5S = 112.15
```

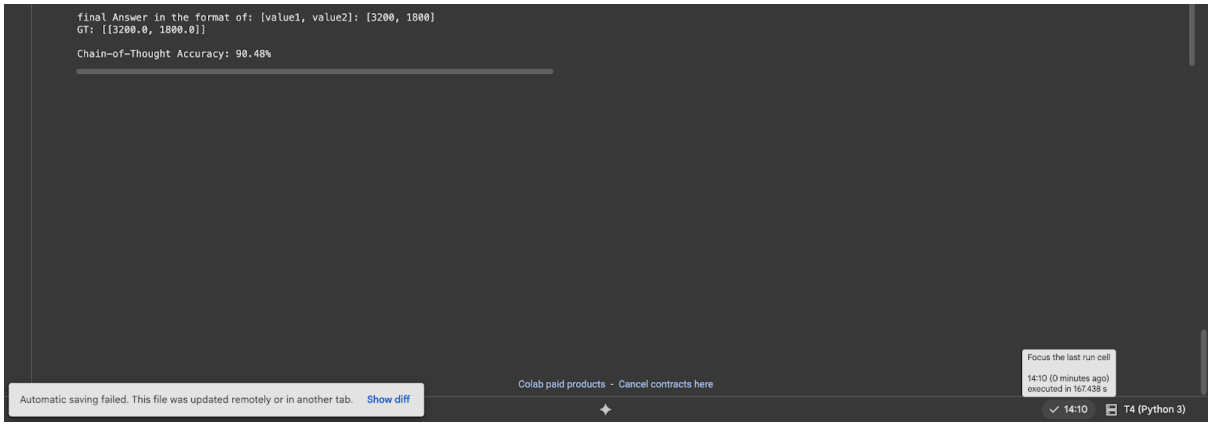



```
Step 1: Simplify equation (1) by dividing by 3:
   J + 2S = 34.75  (3)

Step 2: Solve (3) for J:
   J = 34.75 - 2S

Step 3: Substitute into equation (2):
   4(34.75 - 2S) + 5S = 112.15
   139 - 8S + 5S = 112.15
   139 - 3S = 112.15
   -3S = 112.15 - 139 = -26.85
   S = 8.95

Step 4: Back-substitute to find J:
   J = 34.75 - 2·8.95 = 34.75 - 17.90 = 16.85

Answer in the required format [value1, value2]:
[16.85, 8.95]
GT: [[16.85, 8.95]]

[CoT] Q: One unit of whole wheat flour has 13.6 grams of protein and 2.5 grams of fat . One unit of whole milk has 3.4 grams of protein and 3.7 grams of fat . To the nearest tenth , find th
A:
GT: [[5.416068866571019, 0.39454806312769]]

[CoT] Q: 2 high speed trains are 380 miles apart and traveling toward each other . They meet in 4 hours . If one train is 15 miles per hour faster than the other , find the speed of the slo
A: Here's the step-by-step solution:

1. Let x = speed of the slower train (in mph).
2. Then the faster train's speed is x + 15 (mph).
3. They travel toward each other for 4 hours, so together they cover 380 miles:
   Distance by slower train = 4·x
   Distance by faster train = 4·(x + 15)
   Sum: 4x + 4(x + 15) = 380
4. Simplify and solve:
   4x + 4x + 60 = 380
   8x + 60 = 380
   8x = 320
   x = 40
5. Therefore:
   Slower train = 40 mph
   Faster train = 40 + 15 = 55 mph

Final Answer: [40, 55]
GT: [[40.0, 55.0]]

[CoT] Q: 24 is divided in two parts such that 7 time the first part added to 5 times the second part makes 146 . What is the first part ?
A: Let the two parts be x (the first part) and y (the second part).
```

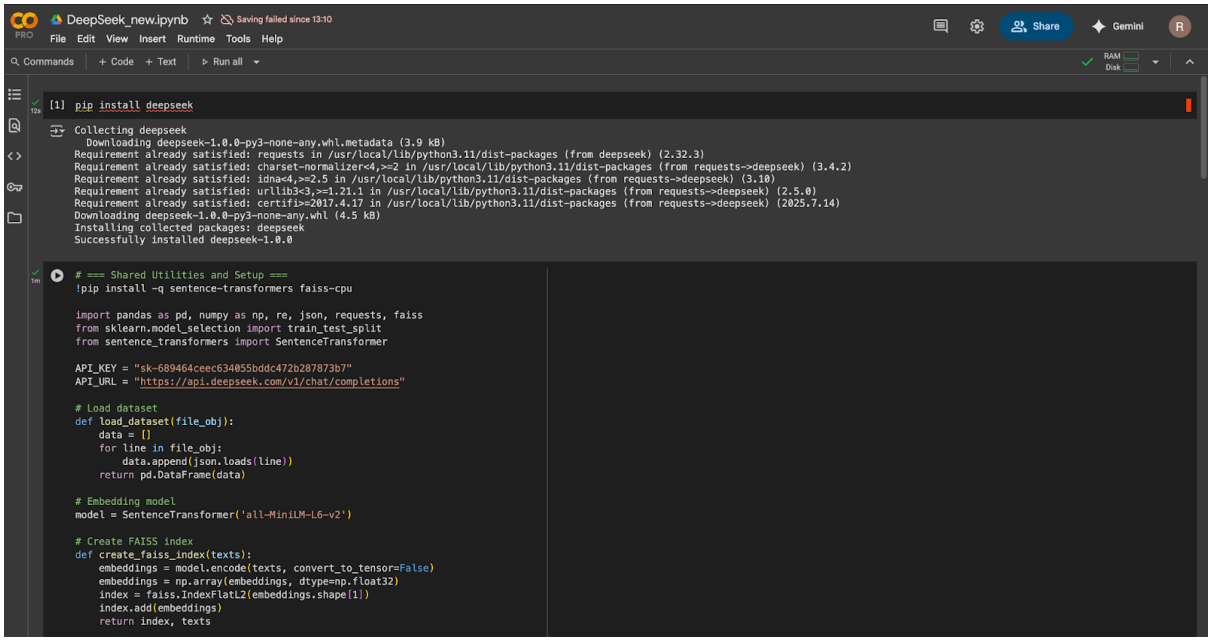


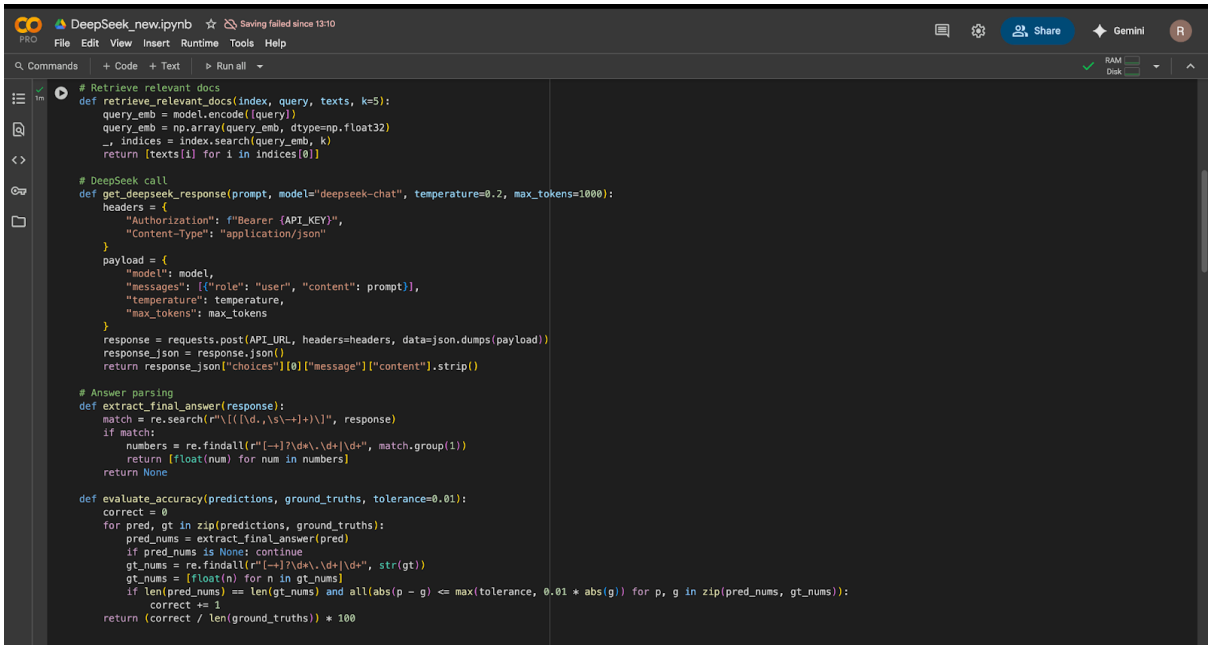

```
1. From "24 is divided in two parts,"
   x + y = 24.

2. From "7 times the first part added to 5 times the second part makes 146,"
   7x + 5y = 146.

Solve the system:

- From x + y = 24, we get y = 24 - x.
- Substitute into 7x + 5y = 146:
  7x + 5(24 - x) = 146
  7x + 120 - 5x = 146
  2x + 120 = 146
  2x = 26
  x = 13.

Then y = 24 - 13 = 11.

Final Answer: [13, 11]
GT: [[13.0, 11.0]]

[CoT] Q: A department store held a sale to sell all of the 214 winter jackets that remained after the season ended . Until noon , each jacket in the store was priced at 31.95 dollars . At n
A: Let x = number sold before noon, y = number sold after noon.

1. Total jackets:
   x + y = 214.

2. Total revenue:
   31.95·x + 18.95·y = 5108.30.

3. From x + y = 214 → y = 214 - x.
   Substitute into revenue:
   31.95x + 18.95(214 - x) = 5108.30
   → 31.95x + 18.95·214 - 18.95x = 5108.30
   → (31.95 - 18.95)x + 4055.30 = 5108.30
   → 13x = 5108.30 - 4055.30 = 1053.00
   → x = 1053.00 ÷ 13 = 81.

4. Then y = 214 - 81 = 133.

Final Answer: [81, 133]
GT: [[81.0, 133.0]]

[CoT] Q: An art dealer sold 16 etchings for 630 dollars . He sold some of them at 35 dollars each and the rest at 45 dollars each . How many etchings did he sell at 35 dollars ? How many et
A: Let x = number of $35 etchings, y = number of $45 etchings.
Then
1) x + y = 16
2) 35x + 45y = 630
```

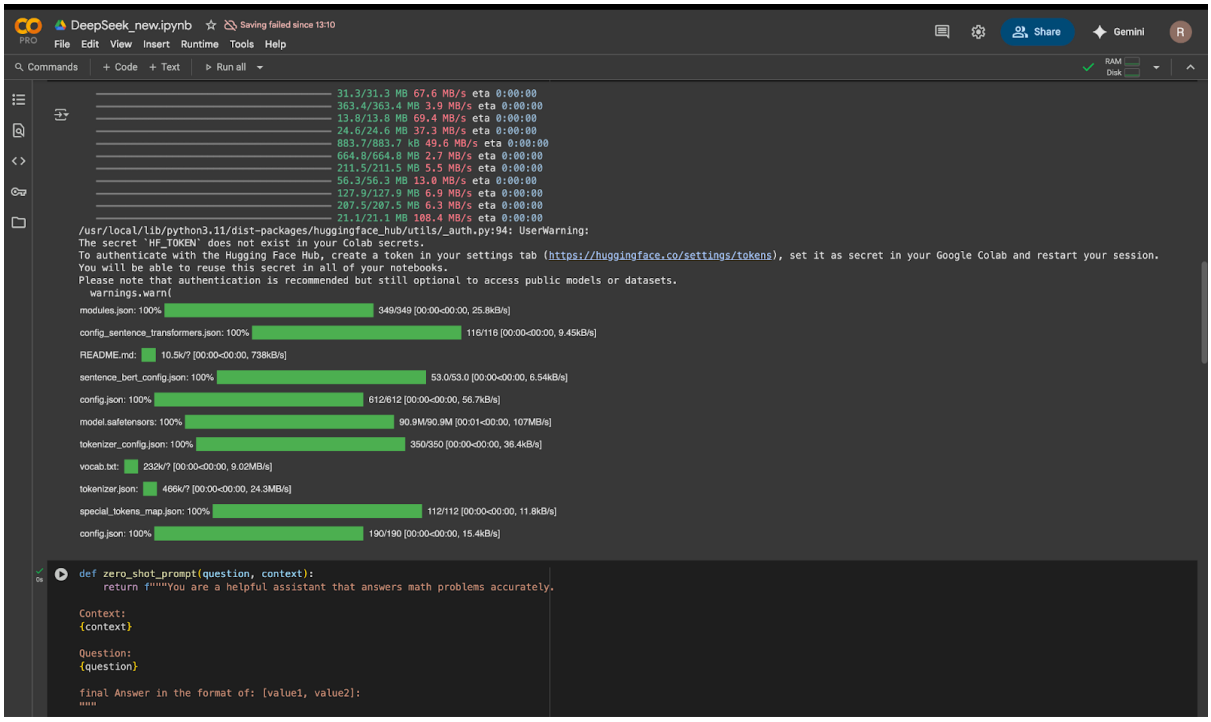


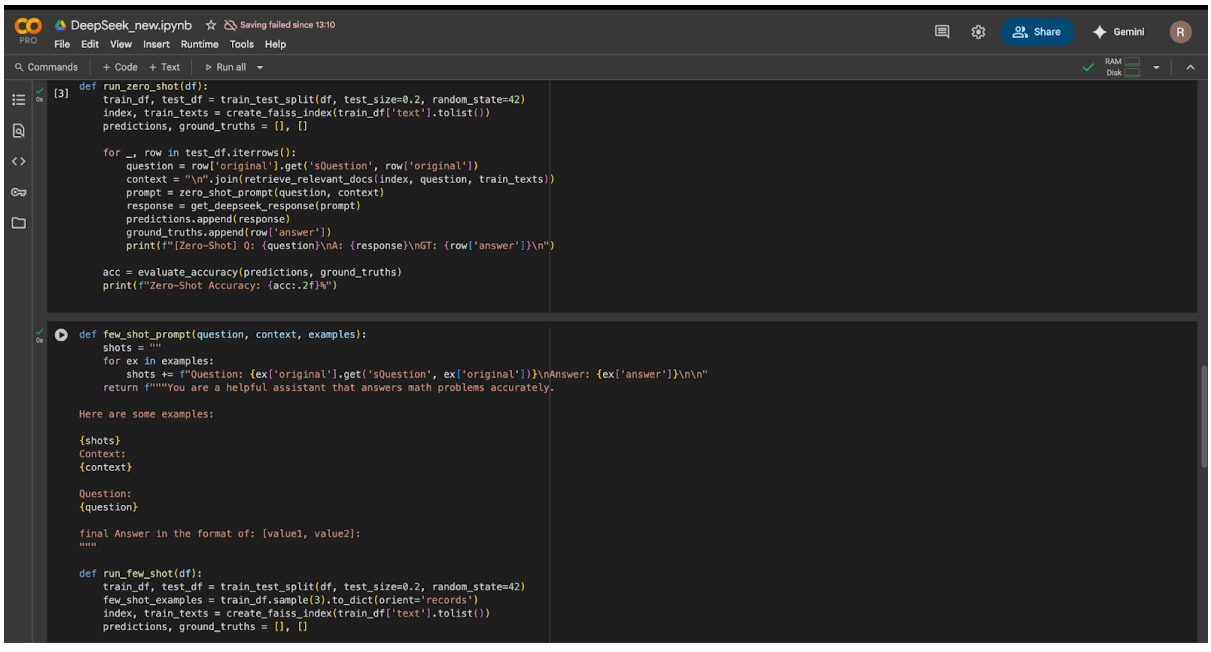


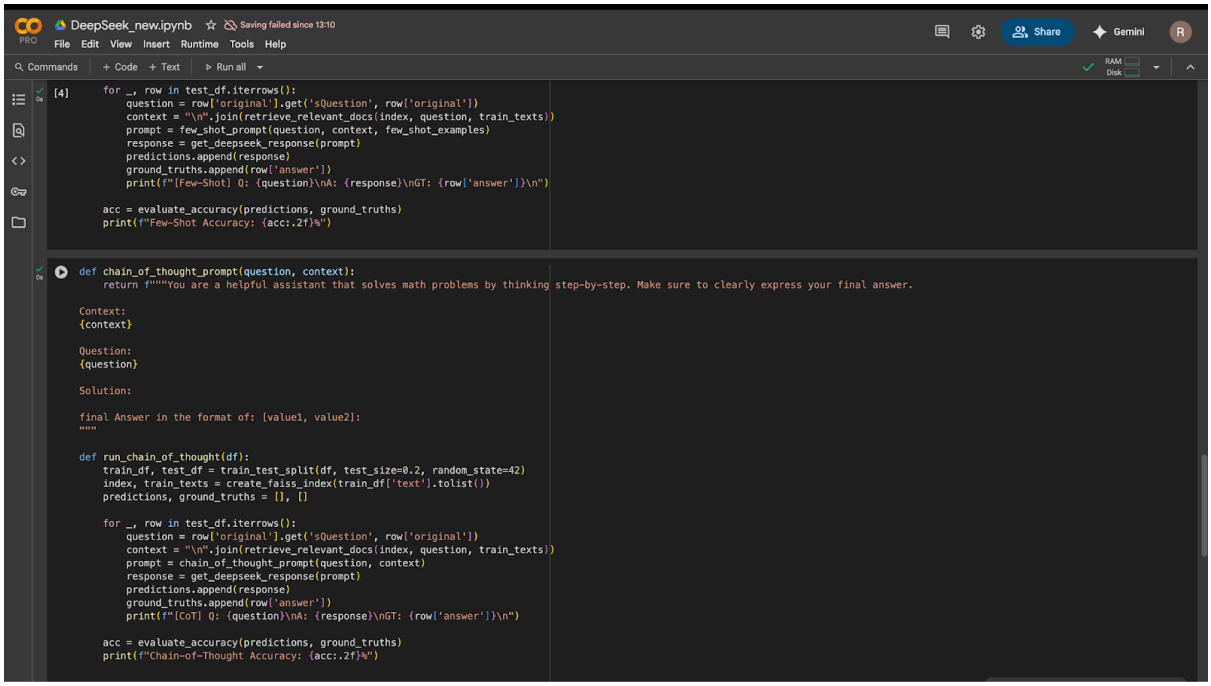
DeepSeek-V1 Model with Dataset 4:


DeepSeek_new.ipynb
Saving failed since 13:10
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
[1] pip install deepseek
Collecting deepseek
Downloading deepseek-1.0.0-py3-none-any.whl.metadata (3.9 kB)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from deepseek) (2.32.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests->deepseek) (3.4.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests->deepseek) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests->deepseek) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests->deepseek) (2025.7.14)
Downloading deepseek-1.0.0-py3-none-any.whl (4.5 kB)
Installing collected packages: deepseek
Successfully installed deepseek-1.0.0
=== Shared Utilities and Setup ===
!pip install -q sentence-transformers faiss-cpu
import pandas as pd, numpy as np, re, json, requests, faiss
from sklearn.model_selection import train_test_split
from sentence_transformers import SentenceTransformer
API_KEY = "sk-689464ceec634055bddc472b287873b7"
API_URL = "https://api.deepseek.com/v1/chat/completions"
# Load dataset
def load_dataset(file_obj):
data = []
for line in file_obj:
data.append(json.loads(line))
return pd.DataFrame(data)
# Embedding model
model = SentenceTransformer('all-MiniLM-L6-v2')
# Create FAISS index
def create_faiss_index(texts):
embeddings = model.encode(texts, convert_to_tensor=False)
embeddings = np.array(embeddings, dtype=np.float32)
index = faiss.IndexFlatL2(embeddings.shape[1])
index.add(embeddings)
return index, texts


DeepSeek_new.ipynb
Saving failed since 13:10
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
# Retrieve relevant docs
def retrieve_relevant_docs(index, query, texts, k=5):
query_emb = model.encode([query])
query_emb = np.array(query_emb, dtype=np.float32)
_, indices = index.search(query_emb, k)
return [texts[i] for i in indices[0]]
# DeepSeek call
def get_deepseek_response(prompt, model="deepseek-chat", temperature=0.2, max_tokens=1000):
headers = {
"Authorization": f"Bearer {API_KEY}",
"Content-Type": "application/json"
}
payload = {
"model": model,
"messages": [{"role": "user", "content": prompt}],
"temperature": temperature,
"max_tokens": max_tokens
}
response = requests.post(API_URL, headers=headers, data=json.dumps(payload))
response_json = response.json()
return response_json["choices"][0]["message"]["content"].strip()
# Answer parsing
def extract_final_answer(response):
match = re.search(r"\[([\d.,\s\-+]+)\]", response)
if match:
numbers = re.findall(r"[-+]?\d*\.\d+|\d+", match.group(1))
return [float(num) for num in numbers]
return None
def evaluate_accuracy(predictions, ground_truths, tolerance=0.01):
correct = 0
for pred, gt in zip(predictions, ground_truths):
pred_nums = extract_final_answer(pred)
if pred_nums is None: continue
gt_nums = re.findall(r"[-+]?\d*\.\d+|\d+", str(gt))
gt_nums = [float(n) for n in gt_nums]
if len(pred_nums) == len(gt_nums) and all(abs(p - g) <= max(tolerance, 0.01 * abs(g)) for p, g in zip(pred_nums, gt_nums)):
correct += 1
return (correct / len(ground_truths)) * 100

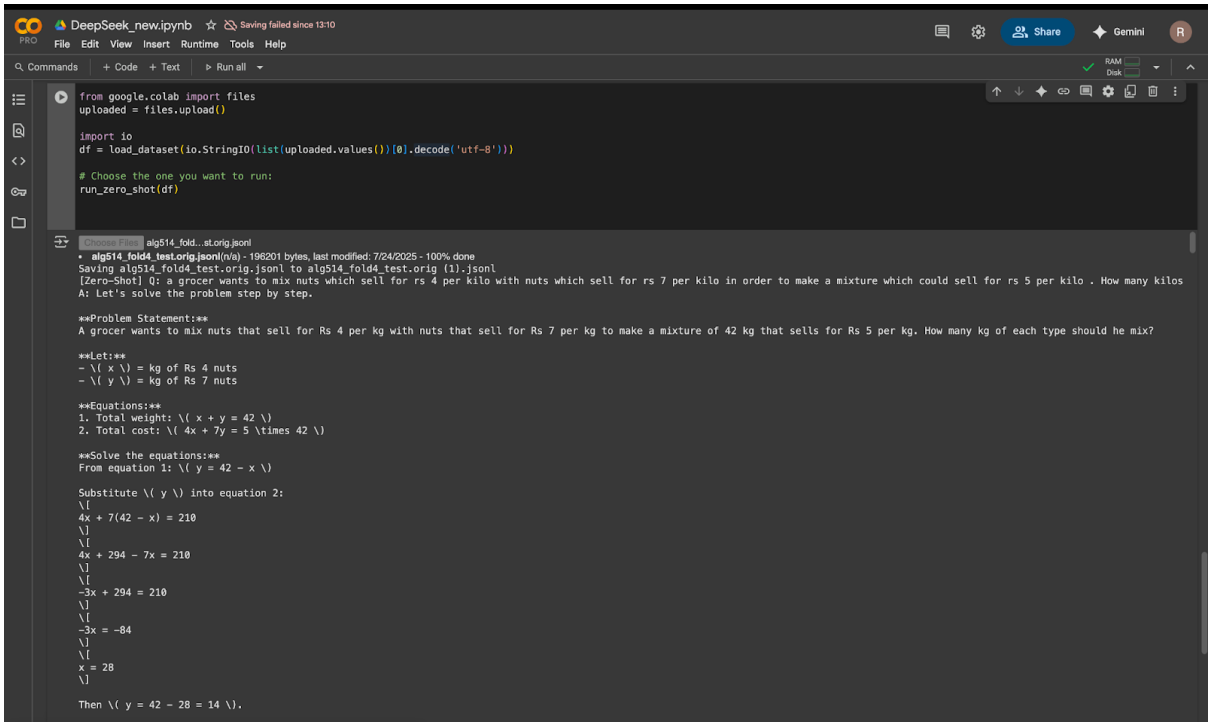

DeepSeek_new.ipynb
Saving failed since 13:10
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
31.3/31.3 MB 67.6 MB/s eta 0:00:00
363.4/363.4 MB 3.9 MB/s eta 0:00:00
13.8/13.8 MB 69.4 MB/s eta 0:00:00
24.6/24.6 MB 37.3 MB/s eta 0:00:00
883.7/883.7 kB 49.6 MB/s eta 0:00:00
664.8/664.8 MB 2.7 MB/s eta 0:00:00
211.5/211.5 MB 5.5 MB/s eta 0:00:00
56.3/56.3 MB 13.0 MB/s eta 0:00:00
127.9/127.9 MB 6.9 MB/s eta 0:00:00
207.5/207.5 MB 6.3 MB/s eta 0:00:00
21.1/21.1 MB 108.4 MB/s eta 0:00:00
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
modules.json: 100% 349/349 [00:00<00:00, 25.8kB/s]
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 9.45kB/s]
README.md: 10.5k/? [00:00<00:00, 738kB/s]
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 6.54kB/s]
config.json: 100% 612/612 [00:00<00:00, 56.7kB/s]
model.safetensors: 100% 90.9M/90.9M [00:01<00:00, 107MB/s]
tokenizer_config.json: 100% 350/350 [00:00<00:00, 36.4kB/s]
vocab.txt: 232k/? [00:00<00:00, 9.02MB/s]
tokenizer.json: 466k/? [00:00<00:00, 24.3MB/s]
special_tokens_map.json: 100% 112/112 [00:00<00:00, 11.8kB/s]
config.json: 100% 190/190 [00:00<00:00, 15.4kB/s]
def zero_shot_prompt(question, context):
return f"""You are a helpful assistant that answers math problems accurately.
Context:
{context}
Question:
{question}
final Answer in the format of: [value1, value2]:
"""


DeepSeek_new.ipynb
Saving failed since 13:10
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
[3]
def run_zero_shot(df):
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
index, train_texts = create_faiss_index(train_df['text'].tolist())
predictions, ground_truths = [], []
for _, row in test_df.iterrows():
question = row['original'].get('sQuestion', row['original'])
context = "\n".join(retrieve_relevant_docs(index, question, train_texts))
prompt = zero_shot_prompt(question, context)
response = get_deepseek_response(prompt)
predictions.append(response)
ground_truths.append(row['answer'])
print(f"[Zero-Shot] Q: {question}\nA: {response}\nGT: {row['answer']}\n")
acc = evaluate_accuracy(predictions, ground_truths)
print(f"Zero-Shot Accuracy: {acc:.2f}%")
def few_shot_prompt(question, context, examples):
shots = ""
for ex in examples:
shots += f"Question: {ex['original'].get('sQuestion', ex['original'])}\nAnswer: {ex['answer']}\n\n"
return f"""You are a helpful assistant that answers math problems accurately.
Here are some examples:
{shots}
Context:
{context}
Question:
{question}
final Answer in the format of: [value1, value2]:
"""
def run_few_shot(df):
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
few_shot_examples = train_df.sample(3).to_dict(orient='records')
index, train_texts = create_faiss_index(train_df['text'].tolist())
predictions, ground_truths = [], []



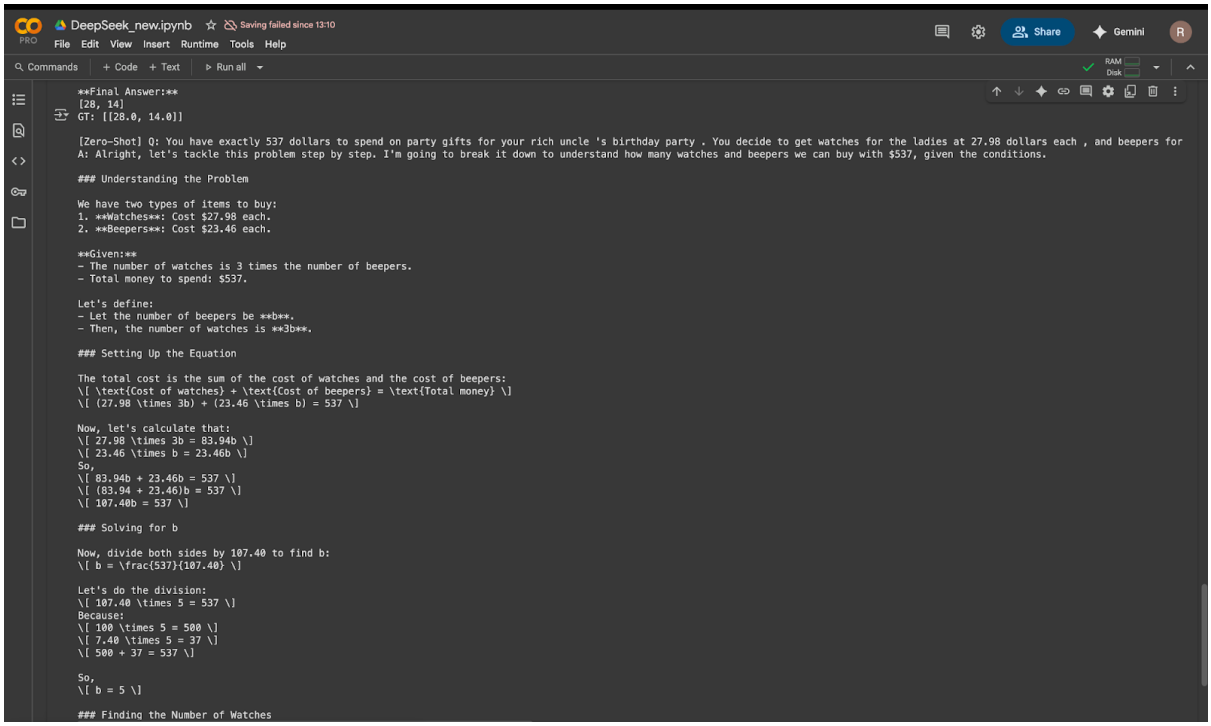
```
[4]     for _, row in test_df.iterrows():
            question = row['original'].get('sQuestion', row['original'])
            context = "\n".join(retrieve_relevant_docs(index, question, train_texts))
            prompt = few_shot_prompt(question, context, few_shot_examples)
            response = get_deepseek_response(prompt)
            predictions.append(response)
            ground_truths.append(row['answer'])
            print(f"[Few-Shot] Q: {question}\nA: {response}\nGT: {row['answer']}\n")

        acc = evaluate_accuracy(predictions, ground_truths)
        print(f"Few-Shot Accuracy: {acc:.2f}%")
```

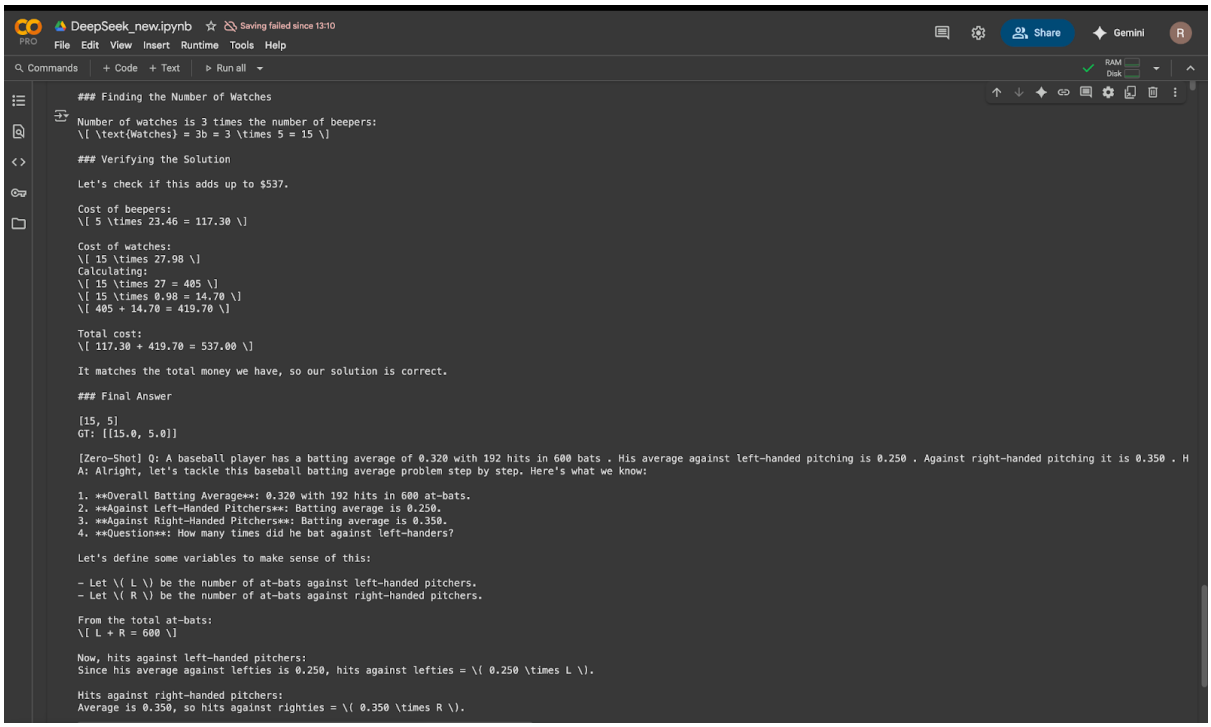
```
def chain_of_thought_prompt(question, context):
    return f"""You are a helpful assistant that solves math problems by thinking step-by-step. Make sure to clearly express your final answer.

Context:
{context}

Question:
{question}

Solution:

final Answer in the format of: [value1, value2]:
"""

def run_chain_of_thought(df):
    train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
    index, train_texts = create_faiss_index(train_df['text'].tolist())
    predictions, ground_truths = [], []

    for _, row in test_df.iterrows():
        question = row['original'].get('sQuestion', row['original'])
        context = "\n".join(retrieve_relevant_docs(index, question, train_texts))
        prompt = chain_of_thought_prompt(question, context)
        response = get_deepseek_response(prompt)
        predictions.append(response)
        ground_truths.append(row['answer'])
        print(f"[CoT] Q: {question}\nA: {response}\nGT: {row['answer']}\n")

    acc = evaluate_accuracy(predictions, ground_truths)
    print(f"Chain-of-Thought Accuracy: {acc:.2f}%")
```

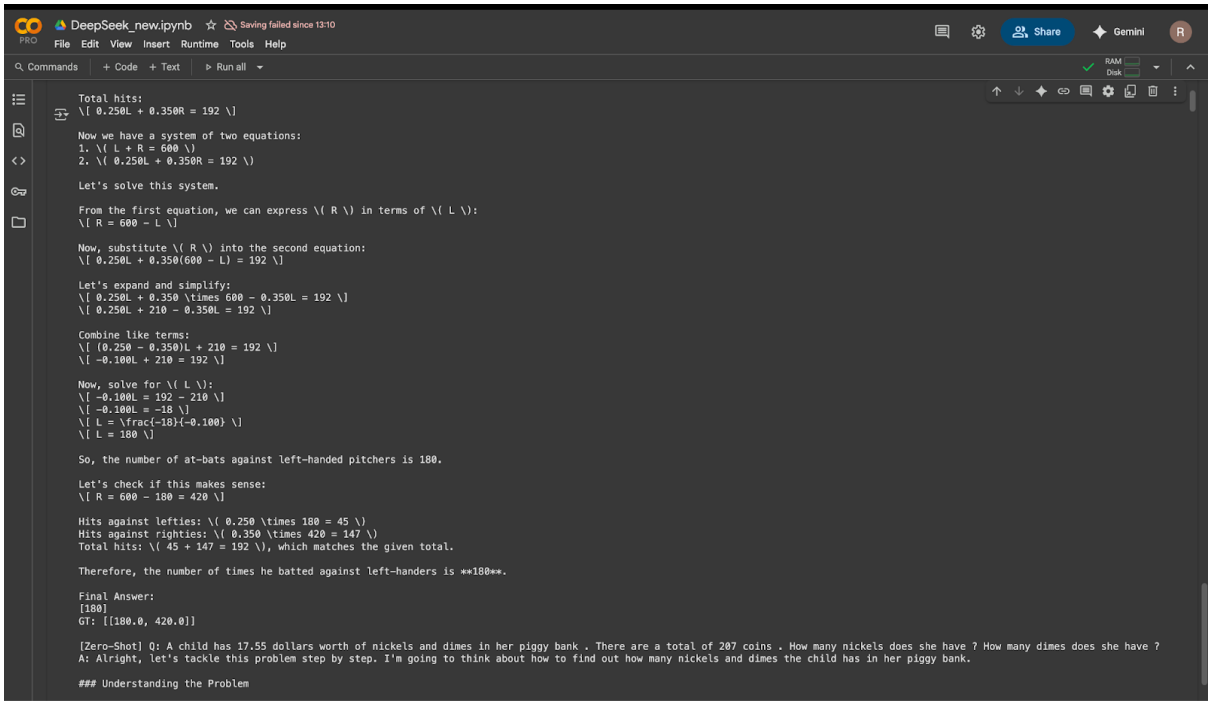


```
from google.colab import files
uploaded = files.upload()

import io
df = load_dataset(io.StringIO(list(uploaded.values())[0].decode('utf-8')))

# Choose the one you want to run:
run_zero_shot(df)
```

```
Choose Files  alg514_fold...st.orig.jsonl
• alg514_fold4_test.orig.jsonl(n/a) - 196201 bytes, last modified: 7/24/2025 - 100% done
Saving alg514_fold4_test.orig.jsonl to alg514_fold4_test.orig (1).jsonl
[Zero-Shot] Q: a grocer wants to mix nuts which sell for rs 4 per kilo with nuts which sell for rs 7 per kilo in order to make a mixture which could sell for rs 5 per kilo . How many kilos
A: Let's solve the problem step by step.

**Problem Statement:**
A grocer wants to mix nuts that sell for Rs 4 per kg with nuts that sell for Rs 7 per kg to make a mixture of 42 kg that sells for Rs 5 per kg. How many kg of each type should he mix?

**Let:**
- \( x \) = kg of Rs 4 nuts
- \( y \) = kg of Rs 7 nuts

**Equations:**
1. Total weight: \( x + y = 42 \)
2. Total cost: \( 4x + 7y = 5 \times 42 \)

**Solve the equations:**
From equation 1: \( y = 42 - x \)

Substitute \( y \) into equation 2:
\[
4x + 7(42 - x) = 210
\]
\[
4x + 294 - 7x = 210
\]
\[
-3x + 294 = 210
\]
\[
-3x = -84
\]
\[
x = 28
\]

Then \( y = 42 - 28 = 14 \).
```


DeepSeek_new.ipynb
Saving failed since 13:10
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
Final Answer:
[28, 14]
GT: [[28.0, 14.0]]
[Zero-Shot] Q: You have exactly 537 dollars to spend on party gifts for your rich uncle 's birthday party . You decide to get watches for the ladies at 27.98 dollars each , and beepers for
A: Alright, let's tackle this problem step by step. I'm going to break it down to understand how many watches and beepers we can buy with $537, given the conditions.
Understanding the Problem
We have two types of items to buy:
1. **Watches**: Cost $27.98 each.
2. **Beepers**: Cost $23.46 each.
Given:
- The number of watches is 3 times the number of beepers.
- Total money to spend: $537.
Let's define:
- Let the number of beepers be **b**.
- Then, the number of watches is **3b**.
Setting Up the Equation
The total cost is the sum of the cost of watches and the cost of beepers:
\[\text{Cost of watches} + \text{Cost of beepers} = \text{Total money} \]
\[(27.98 \times 3b) + (23.46 \times b) = 537 \]
Now, let's calculate that:
\[27.98 \times 3b = 83.94b \]
\[23.46 \times b = 23.46b \]
So,
\[83.94b + 23.46b = 537 \]
\[(83.94 + 23.46)b = 537 \]
\[107.40b = 537 \]
Solving for b
Now, divide both sides by 107.40 to find b:
\[b = \frac{537}{107.40} \]
Let's do the division:
\[107.40 \times 5 = 537 \]
Because:
\[100 \times 5 = 500 \]
\[7.40 \times 5 = 37 \]
\[500 + 37 = 537 \]
So,
\[b = 5 \]
Finding the Number of Watches


DeepSeek_new.ipynb
Saving failed since 13:10
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
Finding the Number of Watches
Number of watches is 3 times the number of beepers:
\[\text{Watches} = 3b = 3 \times 5 = 15 \]
Verifying the Solution
Let's check if this adds up to $537.
Cost of beepers:
\[5 \times 23.46 = 117.30 \]
Cost of watches:
\[15 \times 27.98 \]
Calculating:
\[15 \times 27 = 405 \]
\[15 \times 0.98 = 14.70 \]
\[405 + 14.70 = 419.70 \]
Total cost:
\[117.30 + 419.70 = 537.00 \]
It matches the total money we have, so our solution is correct.
Final Answer
[15, 5]
GT: [[15.0, 5.0]]
[Zero-Shot] Q: A baseball player has a batting average of 0.320 with 192 hits in 600 bats . His average against left-handed pitching is 0.250 . Against right-handed pitching it is 0.350 . H
A: Alright, let's tackle this baseball batting average problem step by step. Here's what we know:
1. **Overall Batting Average**: 0.320 with 192 hits in 600 at-bats.
2. **Against Left-Handed Pitchers**: Batting average is 0.250.
3. **Against Right-Handed Pitchers**: Batting average is 0.350.
4. **Question**: How many times did he bat against left-handers?
Let's define some variables to make sense of this:
- Let \(L \) be the number of at-bats against left-handed pitchers.
- Let \(R \) be the number of at-bats against right-handed pitchers.
From the total at-bats:
\[L + R = 600 \]
Now, hits against left-handed pitchers:
Since his average against lefties is 0.250, hits against lefties = \(0.250 \times L \).
Hits against right-handed pitchers:
Average is 0.350, so hits against righties = \(0.350 \times R \).



DeepSeek_new.ipynb — Saving failed since 13:10

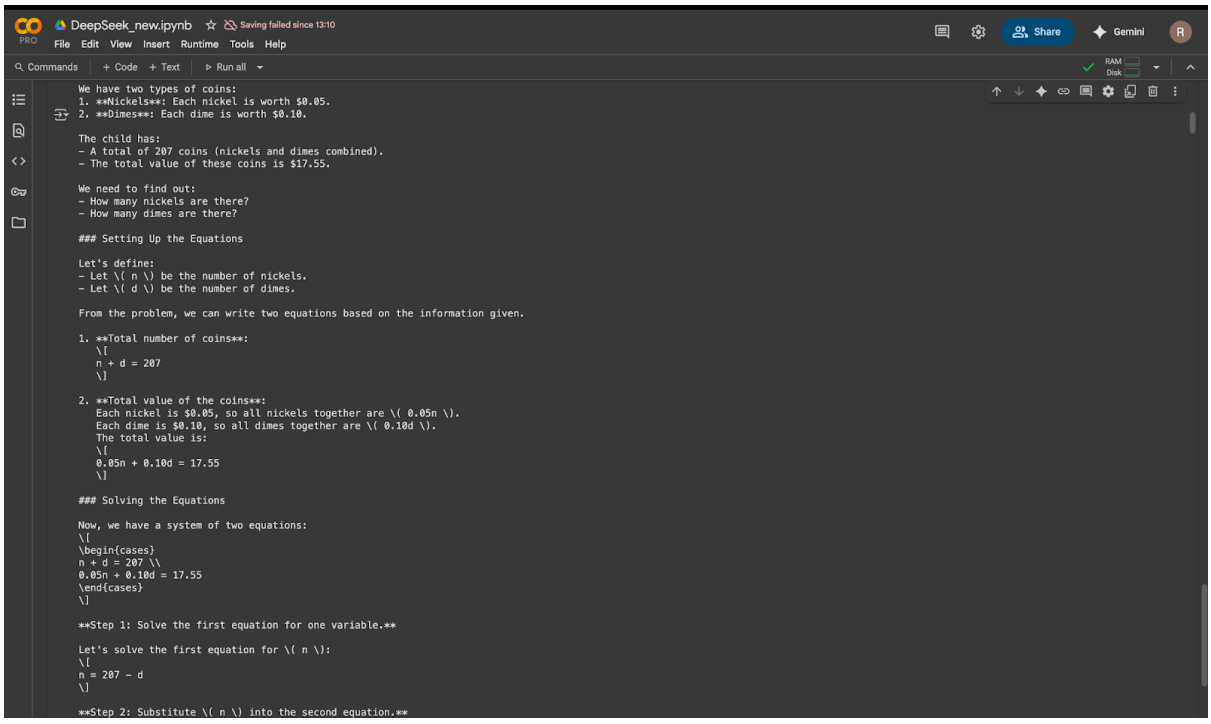
File Edit View Insert Runtime Tools Help

Share Gemini

Commands + Code + Text Run all

RAM Disk

```
Total hits:
\[ 0.250L + 0.350R = 192 \]

Now we have a system of two equations:
1. \( L + R = 600 \)
2. \( 0.250L + 0.350R = 192 \)

Let's solve this system.

From the first equation, we can express \( R \) in terms of \( L \):
\[ R = 600 - L \]

Now, substitute \( R \) into the second equation:
\[ 0.250L + 0.350(600 - L) = 192 \]

Let's expand and simplify:
\[ 0.250L + 0.350 \times 600 - 0.350L = 192 \]
\[ 0.250L + 210 - 0.350L = 192 \]

Combine like terms:
\[ (0.250 - 0.350)L + 210 = 192 \]
\[ -0.100L + 210 = 192 \]

Now, solve for \( L \):
\[ -0.100L = 192 - 210 \]
\[ -0.100L = -18 \]
\[ L = \frac{-18}{-0.100} \]
\[ L = 180 \]

So, the number of at-bats against left-handed pitchers is 180.

Let's check if this makes sense:
\[ R = 600 - 180 = 420 \]

Hits against lefties: \( 0.250 \times 180 = 45 \)
Hits against righties: \( 0.350 \times 420 = 147 \)
Total hits: \( 45 + 147 = 192 \), which matches the given total.

Therefore, the number of times he batted against left-handers is **180**.

Final Answer:
[180]
GT: [[180.0, 420.0]]

[Zero-Shot] Q: A child has 17.55 dollars worth of nickels and dimes in her piggy bank . There are a total of 207 coins . How many nickels does she have ? How many dimes does she have ?
A: Alright, let's tackle this problem step by step. I'm going to think about how to find out how many nickels and dimes the child has in her piggy bank.

### Understanding the Problem
```



DeepSeek_new.ipynb — Saving failed since 13:10

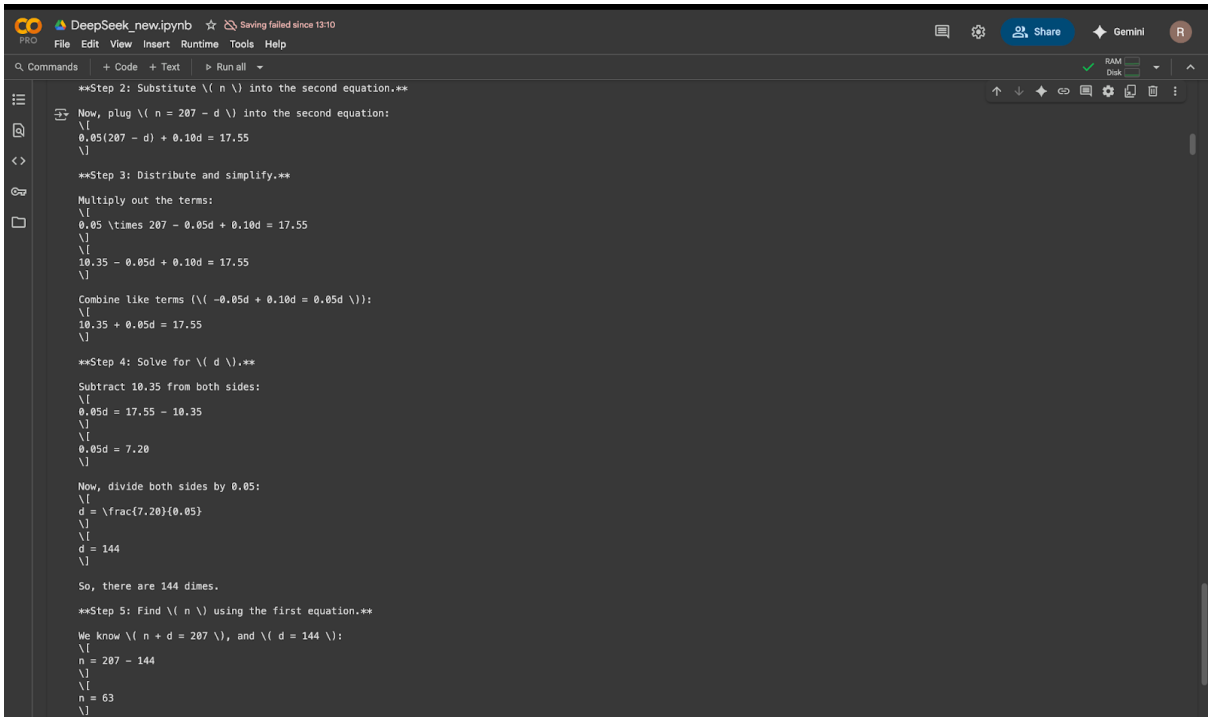
File Edit View Insert Runtime Tools Help

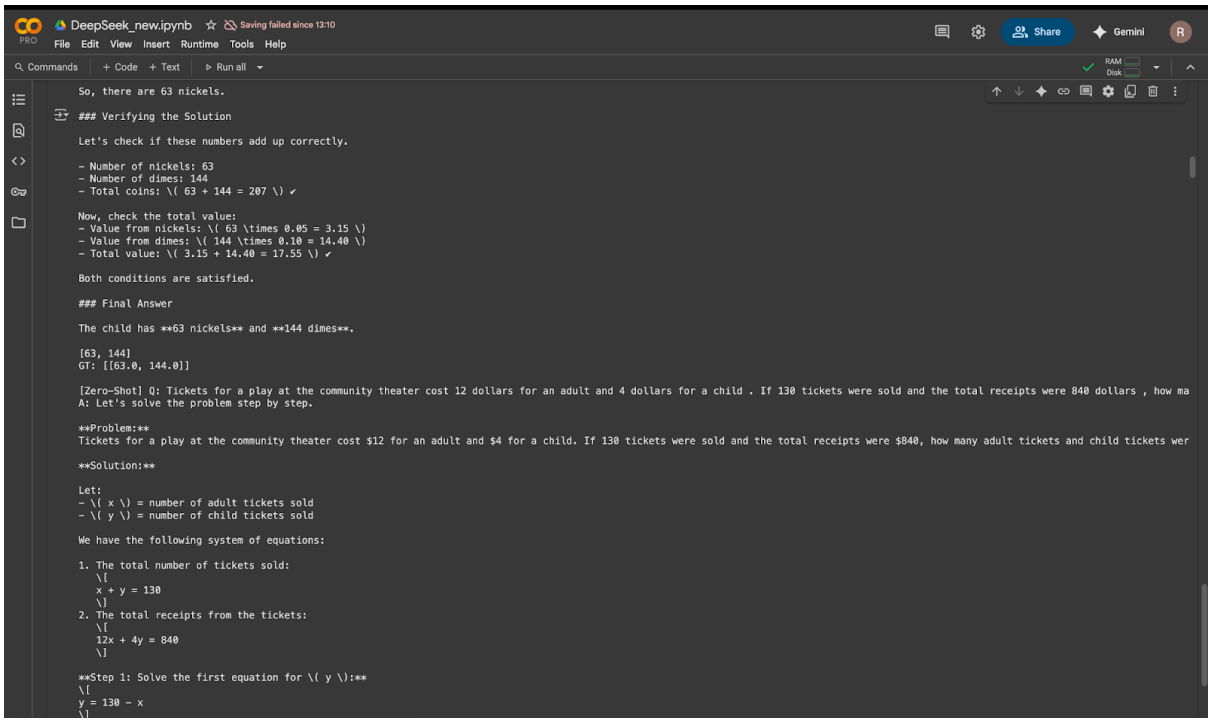
Share Gemini

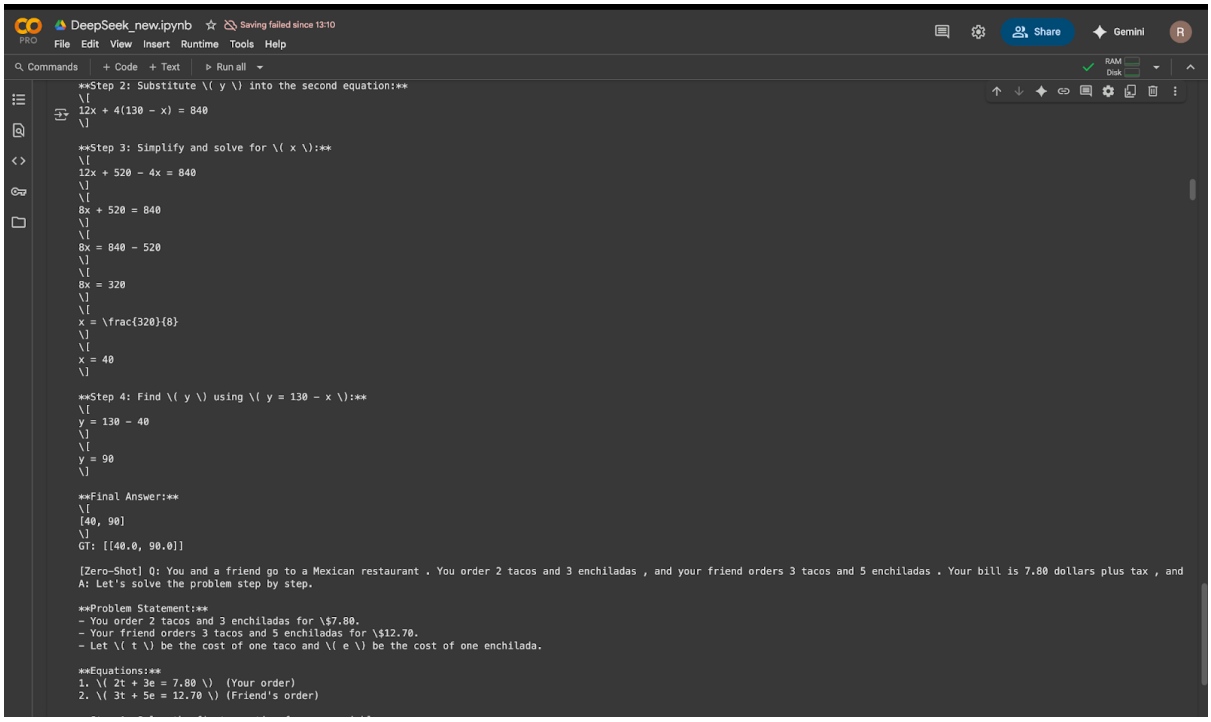
Commands + Code + Text Run all

RAM Disk

```
We have two types of coins:
1. **Nickels**: Each nickel is worth $0.05.
2. **Dimes**: Each dime is worth $0.10.

The child has:
- A total of 207 coins (nickels and dimes combined).
- The total value of these coins is $17.55.

We need to find out:
- How many nickels are there?
- How many dimes are there?

### Setting Up the Equations

Let's define:
- Let \( n \) be the number of nickels.
- Let \( d \) be the number of dimes.

From the problem, we can write two equations based on the information given.

1. **Total number of coins**:
   \[
   n + d = 207
   \]

2. **Total value of the coins**:
   Each nickel is $0.05, so all nickels together are \( 0.05n \).
   Each dime is $0.10, so all dimes together are \( 0.10d \).
   The total value is:
   \[
   0.05n + 0.10d = 17.55
   \]

### Solving the Equations

Now, we have a system of two equations:
\[
\begin{cases}
n + d = 207 \\
0.05n + 0.10d = 17.55
\end{cases}
\]

**Step 1: Solve the first equation for one variable.**

Let's solve the first equation for \( n \):
\[
n = 207 - d
\]

**Step 2: Substitute \( n \) into the second equation.**
```


DeepSeek_new.ipynb
Saving failed since 13:10
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
Step 2: Substitute \(n \) into the second equation.
Now, plug \(n = 207 - d \) into the second equation:
\[
0.05(207 - d) + 0.10d = 17.55
\]
Step 3: Distribute and simplify.
Multiply out the terms:
\[
0.05 \times 207 - 0.05d + 0.10d = 17.55
\]
\[
10.35 - 0.05d + 0.10d = 17.55
\]
Combine like terms (\(-0.05d + 0.10d = 0.05d \)):
\[
10.35 + 0.05d = 17.55
\]
Step 4: Solve for \(d \).
Subtract 10.35 from both sides:
\[
0.05d = 17.55 - 10.35
\]
\[
0.05d = 7.20
\]
Now, divide both sides by 0.05:
\[
d = \frac{7.20}{0.05}
\]
\[
d = 144
\]
So, there are 144 dimes.
Step 5: Find \(n \) using the first equation.
We know \(n + d = 207 \), and \(d = 144 \):
\[
n = 207 - 144
\]
\[
n = 63
\]


DeepSeek_new.ipynb
Saving failed since 13:10
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
So, there are 63 nickels.
Verifying the Solution
Let's check if these numbers add up correctly.
- Number of nickels: 63
- Number of dimes: 144
- Total coins: \(63 + 144 = 207 \) ✔
Now, check the total value:
- Value from nickels: \(63 \times 0.05 = 3.15 \)
- Value from dimes: \(144 \times 0.10 = 14.40 \)
- Total value: \(3.15 + 14.40 = 17.55 \) ✔
Both conditions are satisfied.
Final Answer
The child has **63 nickels** and **144 dimes**.
[63, 144]
GT: [[63.0, 144.0]]
[Zero-Shot] Q: Tickets for a play at the community theater cost 12 dollars for an adult and 4 dollars for a child . If 130 tickets were sold and the total receipts were 840 dollars , how ma
A: Let's solve the problem step by step.
Problem:
Tickets for a play at the community theater cost $12 for an adult and $4 for a child. If 130 tickets were sold and the total receipts were $840, how many adult tickets and child tickets wer
Solution:
Let:
- \(x \) = number of adult tickets sold
- \(y \) = number of child tickets sold
We have the following system of equations:
1. The total number of tickets sold:
\[
x + y = 130
\]
2. The total receipts from the tickets:
\[
12x + 4y = 840
\]
Step 1: Solve the first equation for \(y \):
\[
y = 130 - x
\]



DeepSeek_new.ipynb — Saving failed since 13:10

File Edit View Insert Runtime Tools Help

Share Gemini

Commands + Code + Text Run all

RAM Disk

Step 2: Substitute \(y \) into the second equation:
\[
12x + 4(130 - x) = 840
\]

Step 3: Simplify and solve for \(x \):
\[
12x + 520 - 4x = 840
\]
\[
8x + 520 = 840
\]
\[
8x = 840 - 520
\]
\[
8x = 320
\]
\[
x = \frac{320}{8}
\]
\[
x = 40
\]

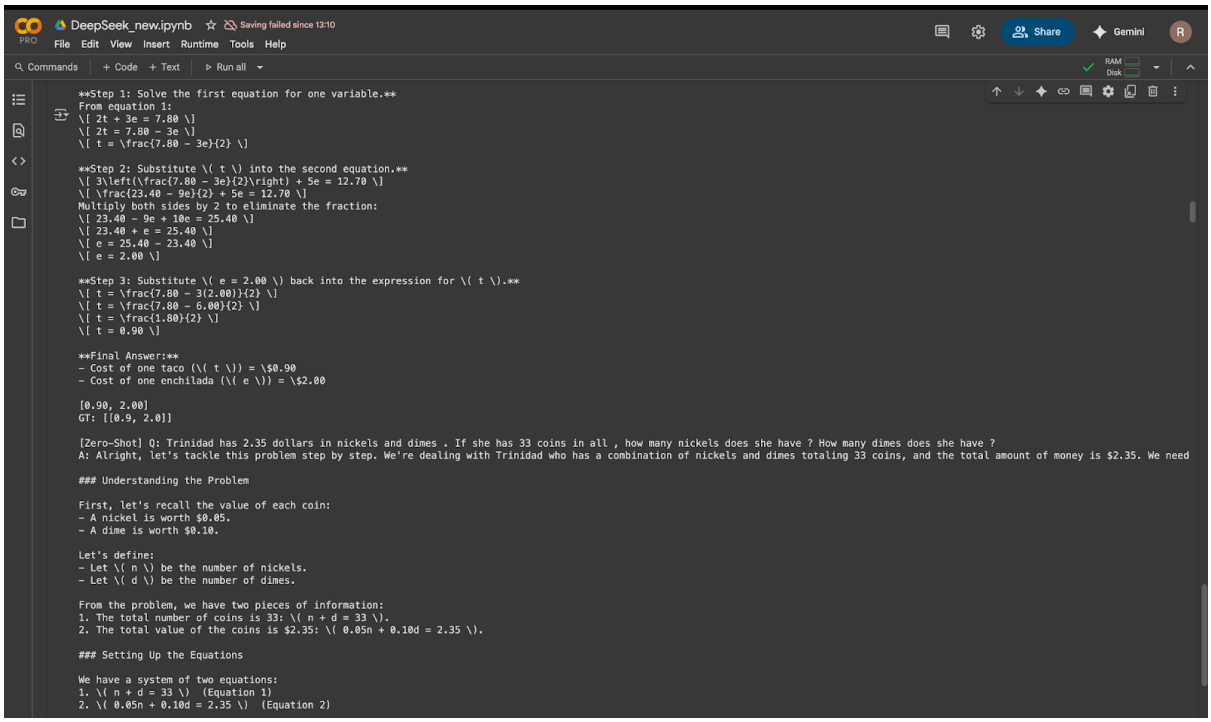
Step 4: Find \(y \) using \(y = 130 - x \):
\[
y = 130 - 40
\]
\[
y = 90
\]

Final Answer:
\[
[40, 90]
\]
GT: [[40.0, 90.0]]

[Zero-Shot] Q: You and a friend go to a Mexican restaurant . You order 2 tacos and 3 enchiladas , and your friend orders 3 tacos and 5 enchiladas . Your bill is 7.80 dollars plus tax , and
A: Let's solve the problem step by step.

Problem Statement:
- You order 2 tacos and 3 enchiladas for \$7.80.
- Your friend orders 3 tacos and 5 enchiladas for \$12.70.
- Let \(t \) be the cost of one taco and \(e \) be the cost of one enchilada.

Equations:
1. \(2t + 3e = 7.80 \) (Your order)
2. \(3t + 5e = 12.70 \) (Friend's order)



DeepSeek_new.ipynb — Saving failed since 13:10

File Edit View Insert Runtime Tools Help

Share Gemini

Commands + Code + Text Run all

RAM Disk

Step 1: Solve the first equation for one variable.
From equation 1:
\[2t + 3e = 7.80 \]
\[2t = 7.80 - 3e \]
\[t = \frac{7.80 - 3e}{2} \]

Step 2: Substitute \(t \) into the second equation.
\[3\left(\frac{7.80 - 3e}{2}\right) + 5e = 12.70 \]
\[\frac{23.40 - 9e}{2} + 5e = 12.70 \]
Multiply both sides by 2 to eliminate the fraction:
\[23.40 - 9e + 10e = 25.40 \]
\[23.40 + e = 25.40 \]
\[e = 25.40 - 23.40 \]
\[e = 2.00 \]

Step 3: Substitute \(e = 2.00 \) back into the expression for \(t \).
\[t = \frac{7.80 - 3(2.00)}{2} \]
\[t = \frac{7.80 - 6.00}{2} \]
\[t = \frac{1.80}{2} \]
\[t = 0.90 \]

Final Answer:
- Cost of one taco (\(t \)) = \$0.90
- Cost of one enchilada (\(e \)) = \$2.00

[0.90, 2.00]
GT: [[0.9, 2.0]]

[Zero-Shot] Q: Trinidad has 2.35 dollars in nickels and dimes . If she has 33 coins in all , how many nickels does she have ? How many dimes does she have ?
A: Alright, let's tackle this problem step by step. We're dealing with Trinidad who has a combination of nickels and dimes totaling 33 coins, and the total amount of money is $2.35. We need

Understanding the Problem

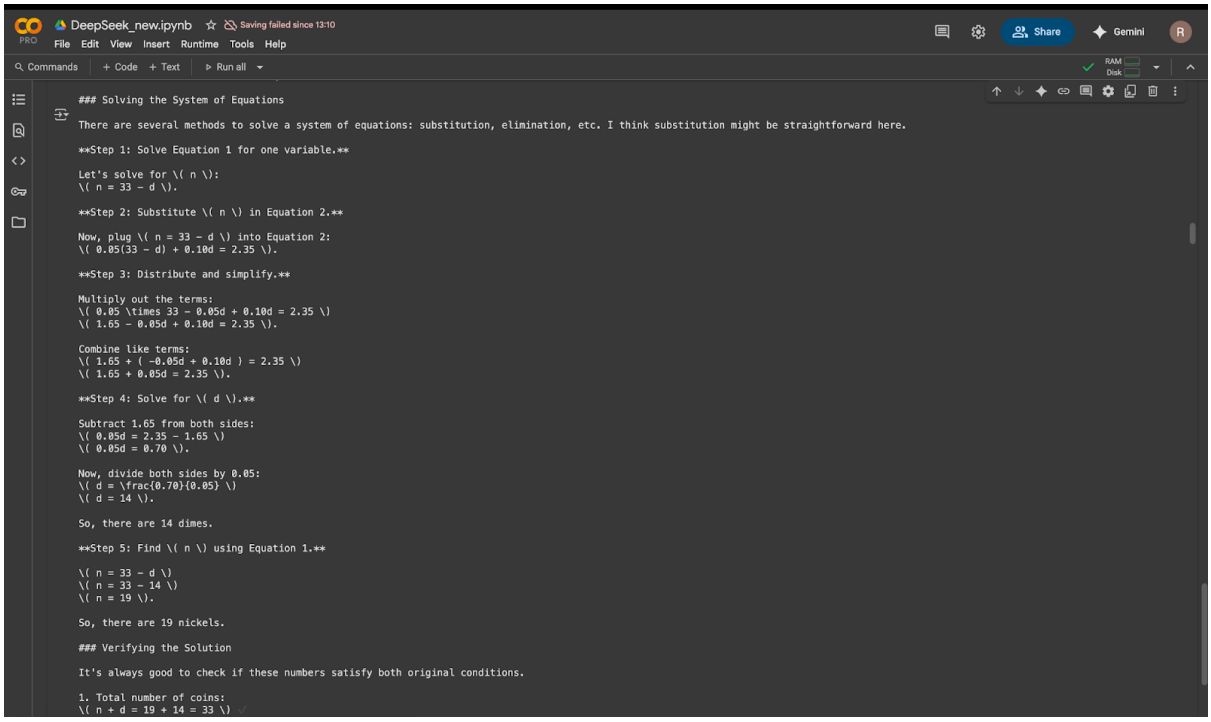
First, let's recall the value of each coin:
- A nickel is worth $0.05.
- A dime is worth $0.10.

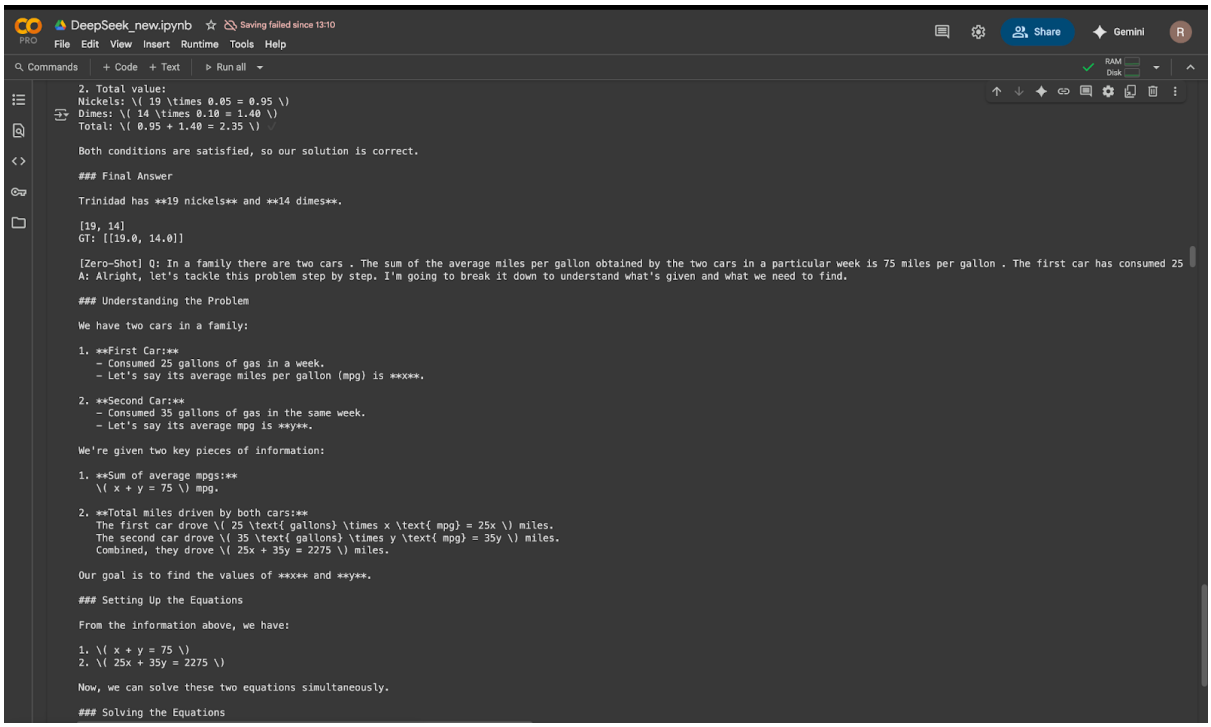
Let's define:
- Let \(n \) be the number of nickels.
- Let \(d \) be the number of dimes.

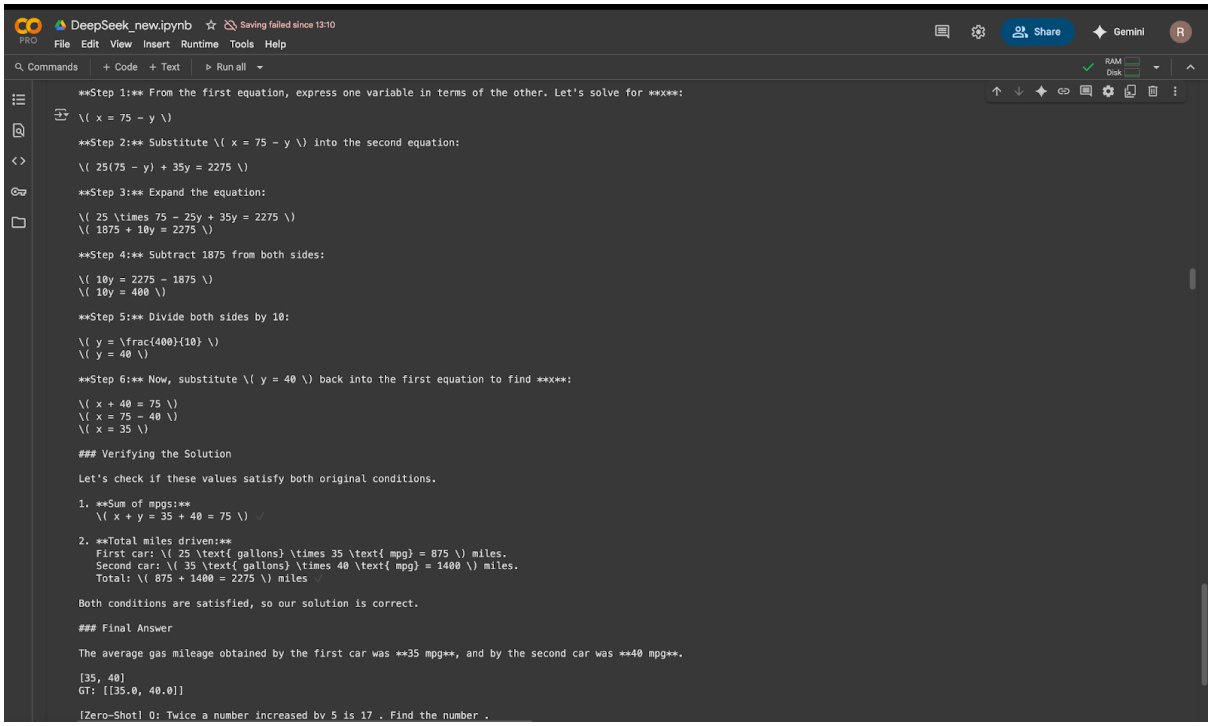
From the problem, we have two pieces of information:
1. The total number of coins is 33: \(n + d = 33 \).
2. The total value of the coins is $2.35: \(0.05n + 0.10d = 2.35 \).

Setting Up the Equations

We have a system of two equations:
1. \(n + d = 33 \) (Equation 1)
2. \(0.05n + 0.10d = 2.35 \) (Equation 2)


DeepSeek_new.ipynb
Saving failed since 13:10
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
Solving the System of Equations
There are several methods to solve a system of equations: substitution, elimination, etc. I think substitution might be straightforward here.
Step 1: Solve Equation 1 for one variable.
Let's solve for \(n \):
\(n = 33 - d \).
Step 2: Substitute \(n \) in Equation 2.
Now, plug \(n = 33 - d \) into Equation 2:
\(0.05(33 - d) + 0.10d = 2.35 \).
Step 3: Distribute and simplify.
Multiply out the terms:
\(0.05 \times 33 - 0.05d + 0.10d = 2.35 \)
\(1.65 - 0.05d + 0.10d = 2.35 \).
Combine like terms:
\(1.65 + (-0.05d + 0.10d) = 2.35 \)
\(1.65 + 0.05d = 2.35 \).
Step 4: Solve for \(d \).
Subtract 1.65 from both sides:
\(0.05d = 2.35 - 1.65 \)
\(0.05d = 0.70 \).
Now, divide both sides by 0.05:
\(d = \frac{0.70}{0.05} \)
\(d = 14 \).
So, there are 14 dimes.
Step 5: Find \(n \) using Equation 1.
\(n = 33 - d \)
\(n = 33 - 14 \)
\(n = 19 \).
So, there are 19 nickels.
Verifying the Solution
It's always good to check if these numbers satisfy both original conditions.
1. Total number of coins:
\(n + d = 19 + 14 = 33 \)


DeepSeek_new.ipynb
Saving failed since 13:10
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
2. Total value:
Nickels: \(19 \times 0.05 = 0.95 \)
Dimes: \(14 \times 0.10 = 1.40 \)
Total: \(0.95 + 1.40 = 2.35 \)
Both conditions are satisfied, so our solution is correct.
Final Answer
Trinidad has **19 nickels** and **14 dimes**.
[19, 14]
GT: [[19.0, 14.0]]
[Zero-Shot] Q: In a family there are two cars . The sum of the average miles per gallon obtained by the two cars in a particular week is 75 miles per gallon . The first car has consumed 25
A: Alright, let's tackle this problem step by step. I'm going to break it down to understand what's given and what we need to find.
Understanding the Problem
We have two cars in a family:
1. **First Car:**
- Consumed 25 gallons of gas in a week.
- Let's say its average miles per gallon (mpg) is **x**.
2. **Second Car:**
- Consumed 35 gallons of gas in the same week.
- Let's say its average mpg is **y**.
We're given two key pieces of information:
1. **Sum of average mpgs:**
\(x + y = 75 \) mpg.
2. **Total miles driven by both cars:**
The first car drove \(25 \text{ gallons} \times x \text{ mpg} = 25x \) miles.
The second car drove \(35 \text{ gallons} \times y \text{ mpg} = 35y \) miles.
Combined, they drove \(25x + 35y = 2275 \) miles.
Our goal is to find the values of **x** and **y**.
Setting Up the Equations
From the information above, we have:
1. \(x + y = 75 \)
2. \(25x + 35y = 2275 \)
Now, we can solve these two equations simultaneously.
Solving the Equations



DeepSeek_new.ipynb — Saving failed since 13:10

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all

Share Gemini

Step 1: From the first equation, express one variable in terms of the other. Let's solve for **x**:

$$x = 75 - y$$

Step 2: Substitute $x = 75 - y$ into the second equation:

$$25(75 - y) + 35y = 2275$$

Step 3: Expand the equation:

$$25 \times 75 - 25y + 35y = 2275$$
$$1875 + 10y = 2275$$

Step 4: Subtract 1875 from both sides:

$$10y = 2275 - 1875$$
$$10y = 400$$

Step 5: Divide both sides by 10:

$$y = \frac{400}{10}$$
$$y = 40$$

Step 6: Now, substitute $y = 40$ back into the first equation to find **x**:

$$x + 40 = 75$$
$$x = 75 - 40$$
$$x = 35$$

Verifying the Solution

Let's check if these values satisfy both original conditions.

1. **Sum of mpgs:**
   $x + y = 35 + 40 = 75$ ✓

2. **Total miles driven:**
   First car: $25 \text{ gallons} \times 35 \text{ mpg} = 875$ miles.
   Second car: $35 \text{ gallons} \times 40 \text{ mpg} = 1400$ miles.
   Total: $875 + 1400 = 2275$ miles ✓

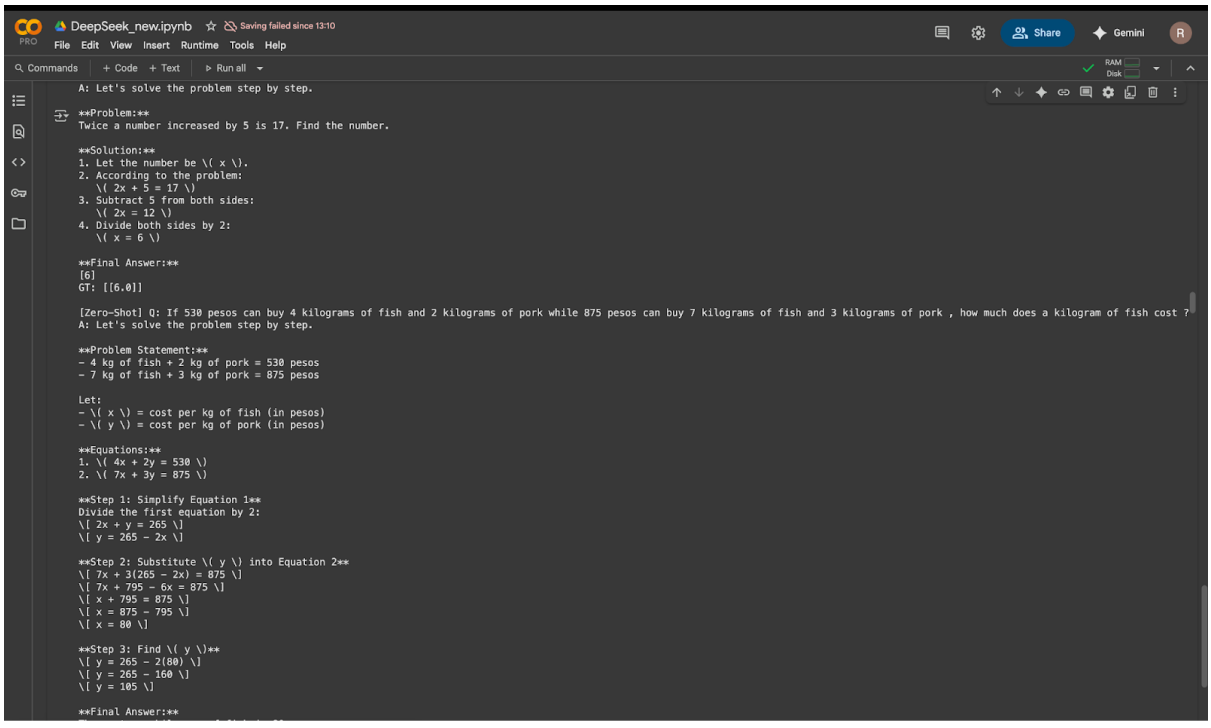
Both conditions are satisfied, so our solution is correct.

Final Answer

The average gas mileage obtained by the first car was **35 mpg**, and by the second car was **40 mpg**.

[35, 40]
GT: [[35.0, 40.0]]

[Zero-Shot] Q: Twice a number increased by 5 is 17 . Find the number .



DeepSeek_new.ipynb — Saving failed since 13:10

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all

Share Gemini

A: Let's solve the problem step by step.

Problem:
Twice a number increased by 5 is 17. Find the number.

Solution:
1. Let the number be x.
2. According to the problem:
   $2x + 5 = 17$
3. Subtract 5 from both sides:
   $2x = 12$
4. Divide both sides by 2:
   $x = 6$

Final Answer:
[6]
GT: [[6.0]]

[Zero-Shot] Q: If 530 pesos can buy 4 kilograms of fish and 2 kilograms of pork while 875 pesos can buy 7 kilograms of fish and 3 kilograms of pork , how much does a kilogram of fish cost ?
A: Let's solve the problem step by step.

Problem Statement:
- 4 kg of fish + 2 kg of pork = 530 pesos
- 7 kg of fish + 3 kg of pork = 875 pesos

Let:
- x = cost per kg of fish (in pesos)
- y = cost per kg of pork (in pesos)

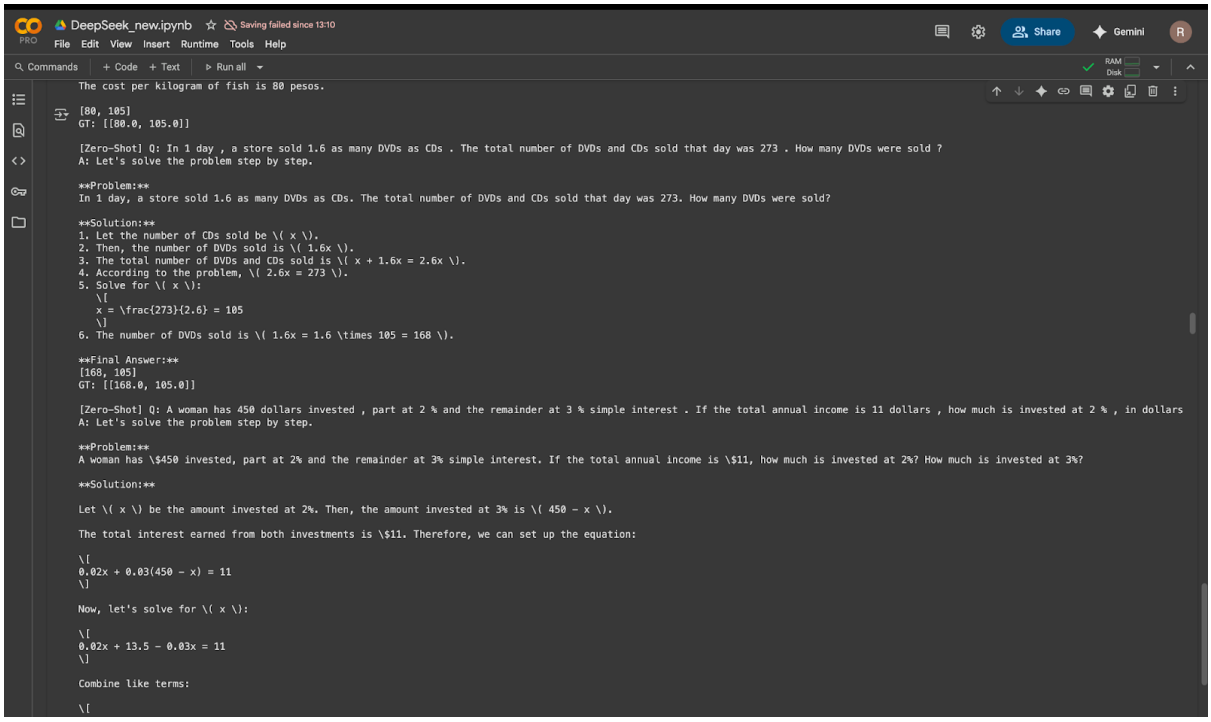
Equations:
1. $4x + 2y = 530$
2. $7x + 3y = 875$

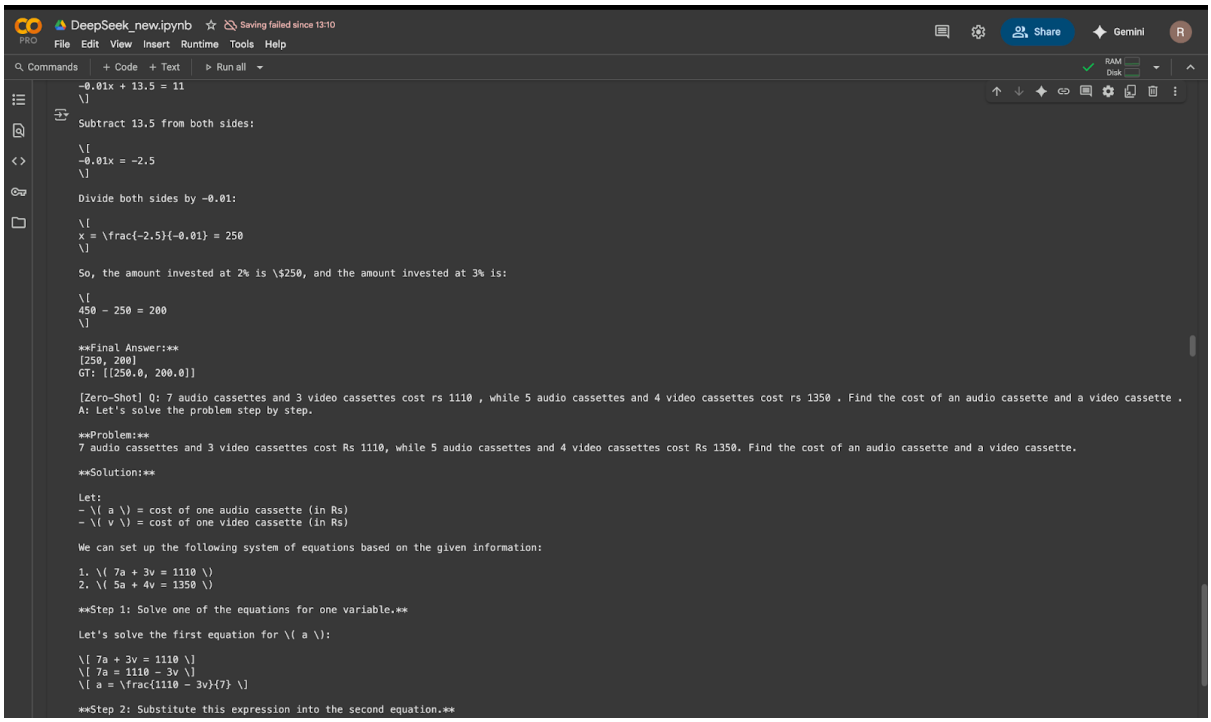
Step 1: Simplify Equation 1
Divide the first equation by 2:
$$2x + y = 265$$
$$y = 265 - 2x$$

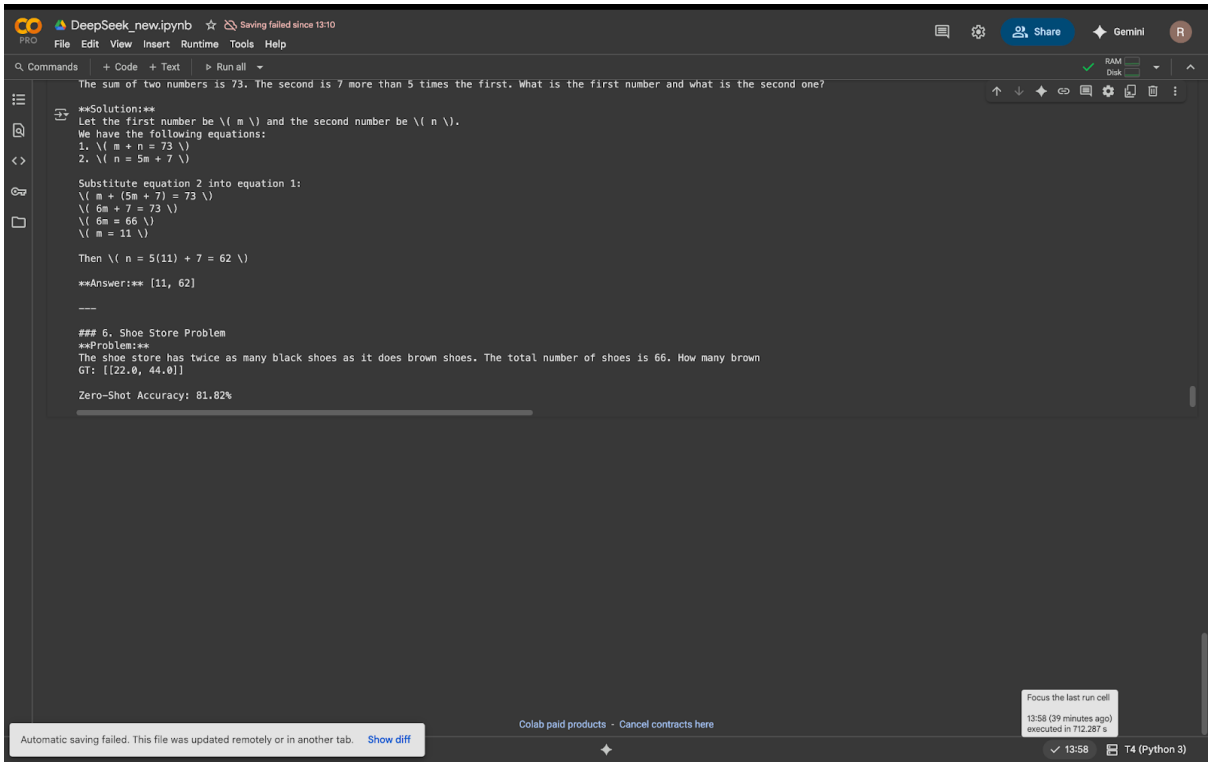
Step 2: Substitute y into Equation 2
$$7x + 3(265 - 2x) = 875$$
$$7x + 795 - 6x = 875$$
$$x + 795 = 875$$
$$x = 875 - 795$$
$$x = 80$$

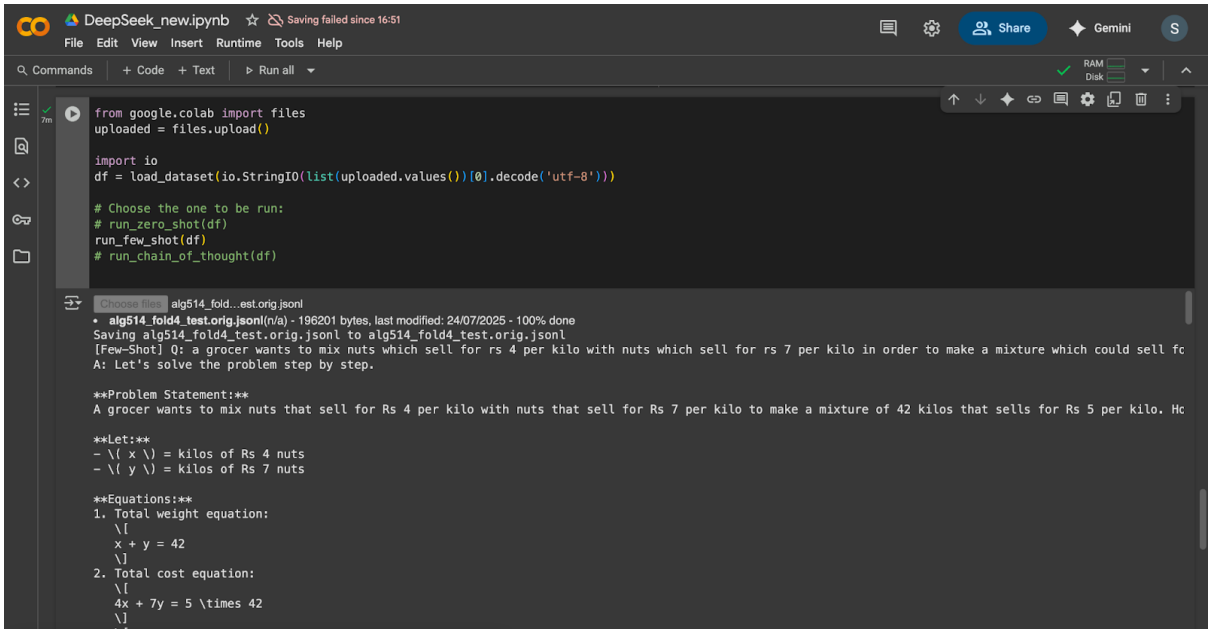
Step 3: Find y
$$y = 265 - 2(80)$$
$$y = 265 - 160$$
$$y = 105$$

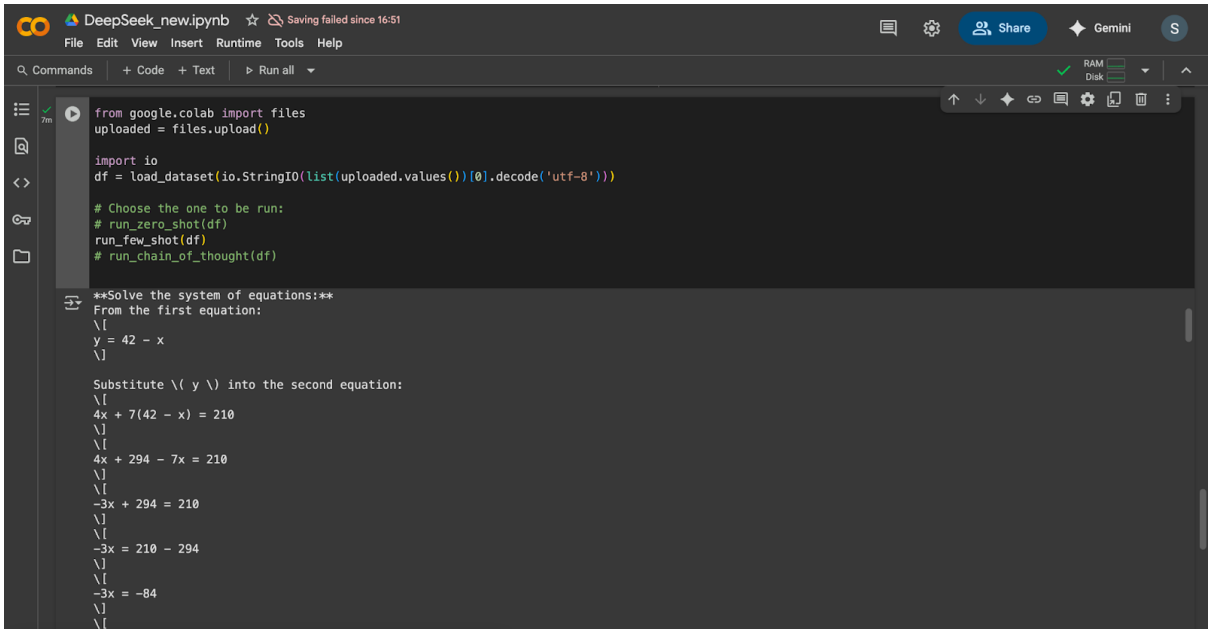
Final Answer:

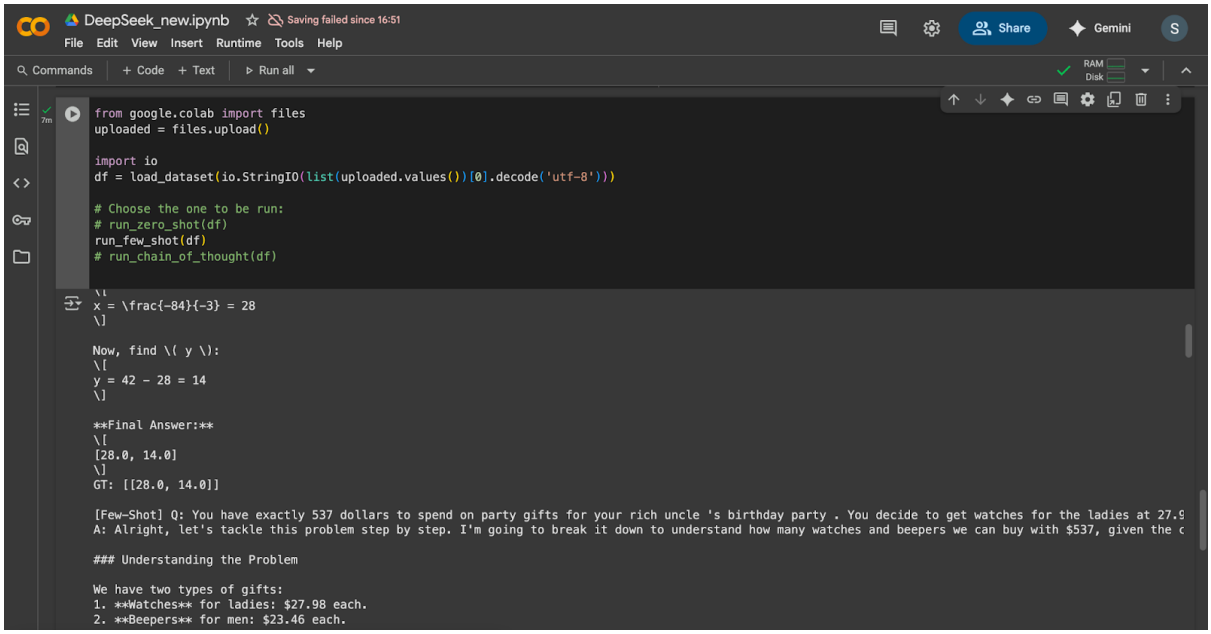

DeepSeek_new.ipynb
Saving failed since 13:10
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
The cost per kilogram of fish is 80 pesos.
[80, 105]
GT: [[80.0, 105.0]]
[Zero-Shot] Q: In 1 day , a store sold 1.6 as many DVDs as CDs . The total number of DVDs and CDs sold that day was 273 . How many DVDs were sold ?
A: Let's solve the problem step by step.
Problem:
In 1 day, a store sold 1.6 as many DVDs as CDs. The total number of DVDs and CDs sold that day was 273. How many DVDs were sold?
Solution:
1. Let the number of CDs sold be \(x \).
2. Then, the number of DVDs sold is \(1.6x \).
3. The total number of DVDs and CDs sold is \(x + 1.6x = 2.6x \).
4. According to the problem, \(2.6x = 273 \).
5. Solve for \(x \):
\[
x = \frac{273}{2.6} = 105
\]
6. The number of DVDs sold is \(1.6x = 1.6 \times 105 = 168 \).
Final Answer:
[168, 105]
GT: [[168.0, 105.0]]
[Zero-Shot] Q: A woman has 450 dollars invested , part at 2 % and the remainder at 3 % simple interest . If the total annual income is 11 dollars , how much is invested at 2 % , in dollars
A: Let's solve the problem step by step.
Problem:
A woman has \$450 invested, part at 2% and the remainder at 3% simple interest. If the total annual income is \$11, how much is invested at 2%? How much is invested at 3%?
Solution:
Let \(x \) be the amount invested at 2%. Then, the amount invested at 3% is \(450 - x \).
The total interest earned from both investments is \$11. Therefore, we can set up the equation:
\[
0.02x + 0.03(450 - x) = 11
\]
Now, let's solve for \(x \):
\[
0.02x + 13.5 - 0.03x = 11
\]
Combine like terms:
\[

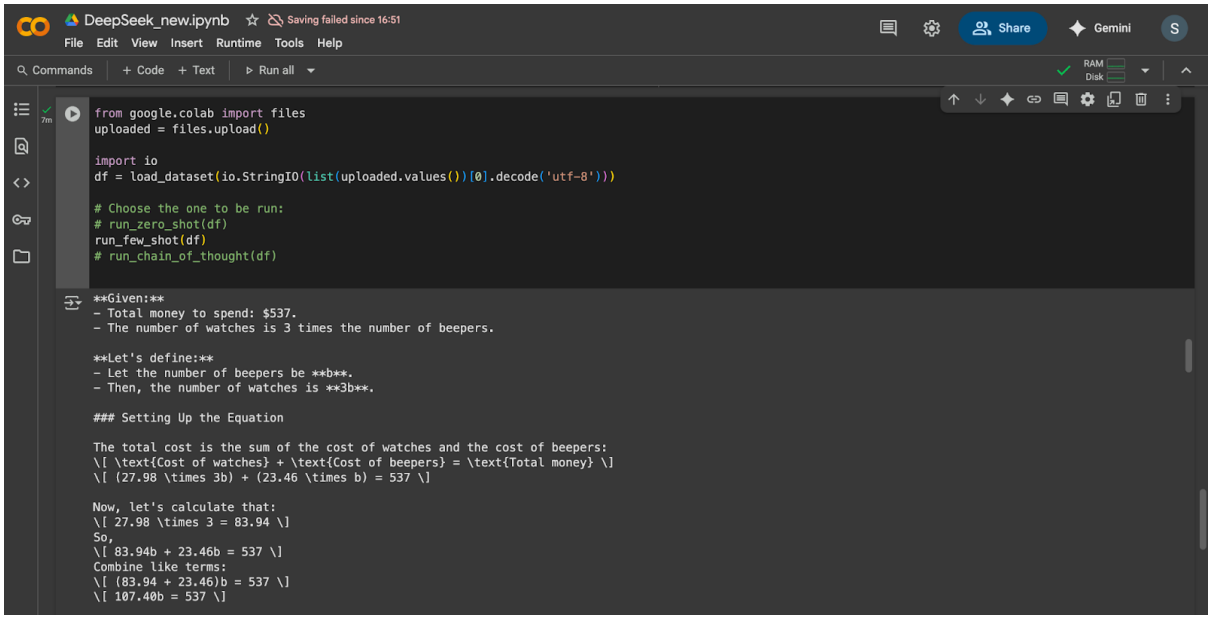

DeepSeek_new.ipynb
Saving failed since 13:10
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
-0.01x + 13.5 = 11
\]
Subtract 13.5 from both sides:
\[
-0.01x = -2.5
\]
Divide both sides by -0.01:
\[
x = \frac{-2.5}{-0.01} = 250
\]
So, the amount invested at 2% is \$250, and the amount invested at 3% is:
\[
450 - 250 = 200
\]
Final Answer:
[250, 200]
GT: [[250.0, 200.0]]
[Zero-Shot] Q: 7 audio cassettes and 3 video cassettes cost rs 1110 , while 5 audio cassettes and 4 video cassettes cost rs 1350 . Find the cost of an audio cassette and a video cassette .
A: Let's solve the problem step by step.
Problem:
7 audio cassettes and 3 video cassettes cost Rs 1110, while 5 audio cassettes and 4 video cassettes cost Rs 1350. Find the cost of an audio cassette and a video cassette.
Solution:
Let:
- \(a \) = cost of one audio cassette (in Rs)
- \(v \) = cost of one video cassette (in Rs)
We can set up the following system of equations based on the given information:
1. \(7a + 3v = 1110 \)
2. \(5a + 4v = 1350 \)
Step 1: Solve one of the equations for one variable.
Let's solve the first equation for \(a \):
\[7a + 3v = 1110 \]
\[7a = 1110 - 3v \]
\[a = \frac{1110 - 3v}{7} \]
Step 2: Substitute this expression into the second equation.

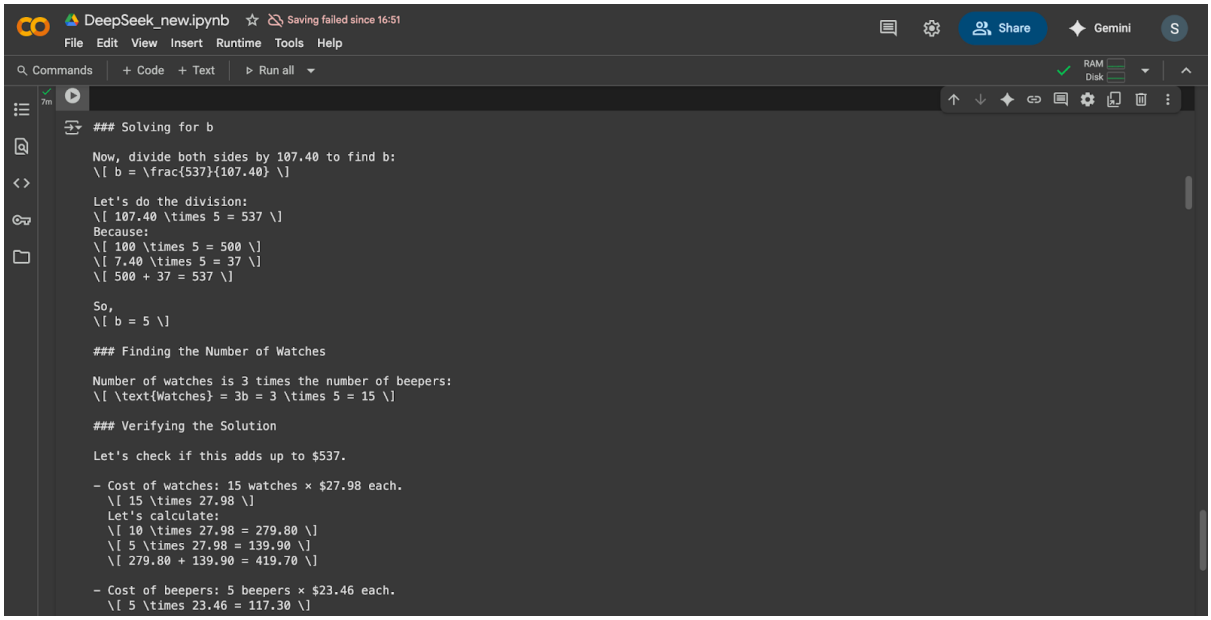

DeepSeek_new.ipynb
Saving failed since 13:10
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
The sum of two numbers is 73. The second is 7 more than 5 times the first. What is the first number and what is the second one?
Solution:
Let the first number be \(m \) and the second number be \(n \).
We have the following equations:
1. \(m + n = 73 \)
2. \(n = 5m + 7 \)
Substitute equation 2 into equation 1:
\(m + (5m + 7) = 73 \)
\(6m + 7 = 73 \)
\(6m = 66 \)
\(m = 11 \)
Then \(n = 5(11) + 7 = 62 \)
Answer: [11, 62]
---
6. Shoe Store Problem
Problem:
The shoe store has twice as many black shoes as it does brown shoes. The total number of shoes is 66. How many brown
GT: [[22.0, 44.0]]
Zero-Shot Accuracy: 81.82%
Automatic saving failed. This file was updated remotely or in another tab.
Show diff
Colab paid products - Cancel contracts here
Focus the last run cell
13:58 (39 minutes ago) executed in 712.287 s
13:58
T4 (Python 3)

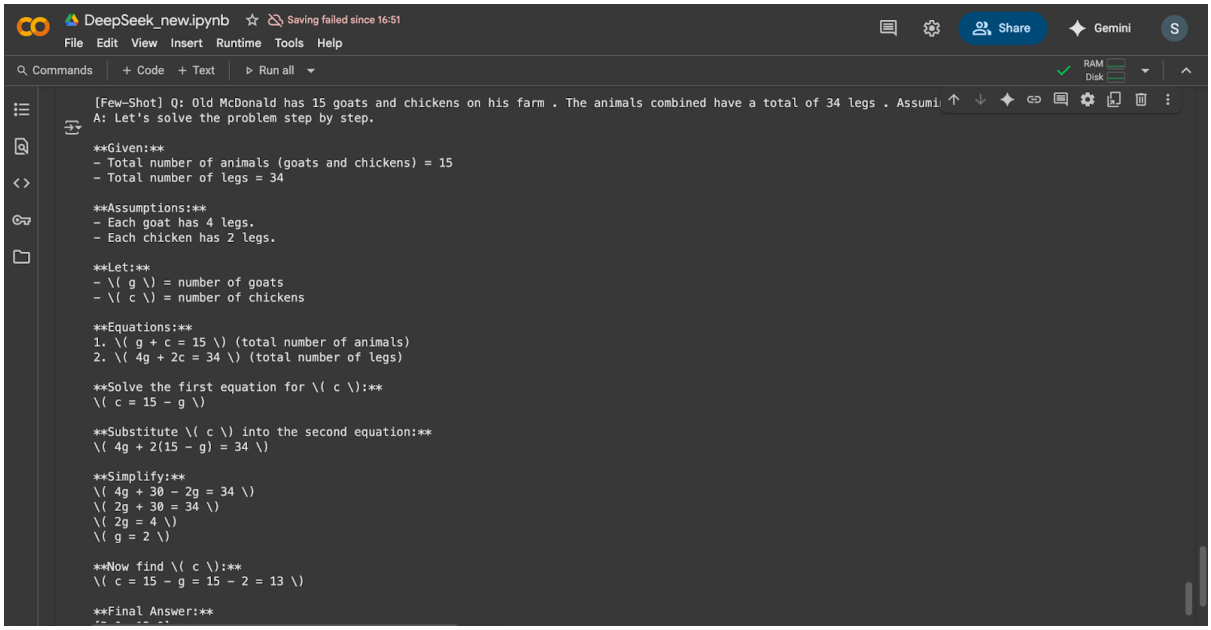

DeepSeek_new.ipynb
Saving failed since 16:51
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
from google.colab import files
uploaded = files.upload()
import io
df = load_dataset(io.StringIO(list(uploaded.values())[0].decode('utf-8')))
# Choose the one to be run:
# run_zero_shot(df)
run_few_shot(df)
# run_chain_of_thought(df)
Choose files
alg514_fold...est.orig.jsonl
alg514_fold4_test.orig.jsonl(n/a) - 196201 bytes, last modified: 24/07/2025 - 100% done
Saving alg514_fold4_test.orig.jsonl to alg514_fold4_test.orig.jsonl
[Few-Shot] Q: a grocer wants to mix nuts which sell for rs 4 per kilo with nuts which sell for rs 7 per kilo in order to make a mixture which could sell fo
A: Let's solve the problem step by step.
Problem Statement:
A grocer wants to mix nuts that sell for Rs 4 per kilo with nuts that sell for Rs 7 per kilo to make a mixture of 42 kilos that sells for Rs 5 per kilo. Ho
Let:
- \(x \) = kilos of Rs 4 nuts
- \(y \) = kilos of Rs 7 nuts
Equations:
1. Total weight equation:
\[
x + y = 42
\]
2. Total cost equation:
\[
4x + 7y = 5 \times 42
\]

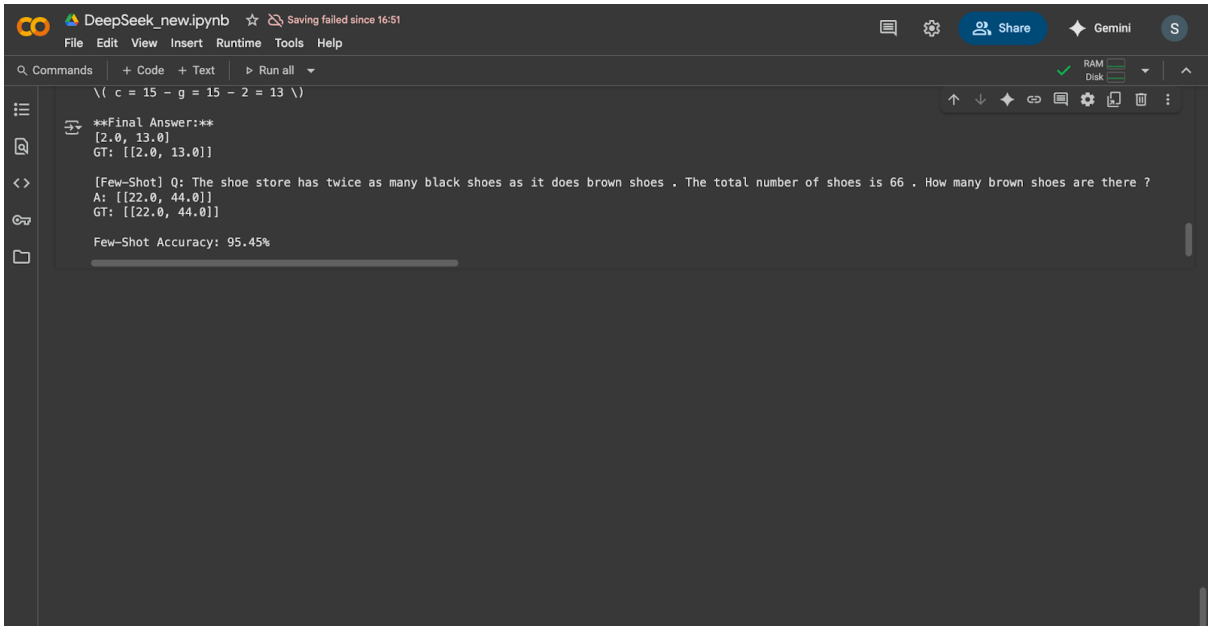

DeepSeek_new.ipynb
Saving failed since 16:51
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
from google.colab import files
uploaded = files.upload()
import io
df = load_dataset(io.StringIO(list(uploaded.values())[0].decode('utf-8')))
# Choose the one to be run:
# run_zero_shot(df)
run_few_shot(df)
# run_chain_of_thought(df)
Solve the system of equations:
From the first equation:
\[
y = 42 - x
\]
Substitute \(y \) into the second equation:
\[
4x + 7(42 - x) = 210
\]
\[
4x + 294 - 7x = 210
\]
\[
-3x + 294 = 210
\]
\[
-3x = 210 - 294
\]
\[
-3x = -84
\]
\[

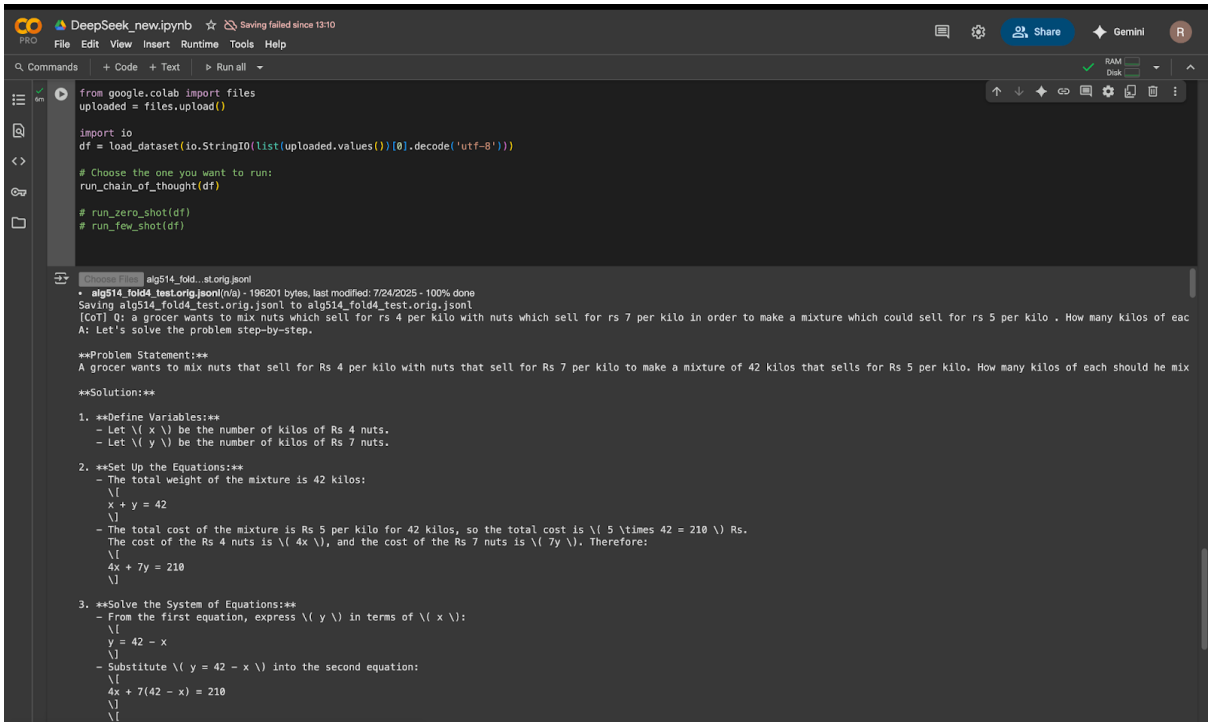

DeepSeek_new.ipynb
Saving failed since 16:51
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
from google.colab import files
uploaded = files.upload()
import io
df = load_dataset(io.StringIO(list(uploaded.values())[0].decode('utf-8')))
# Choose the one to be run:
# run_zero_shot(df)
run_few_shot(df)
# run_chain_of_thought(df)
x = \frac{-84}{-3} = 28
\]
Now, find \(y \):
\[
y = 42 - 28 = 14
\]
Final Answer:
\[
[28.0, 14.0]
\]
GT: [[28.0, 14.0]]
[Few-Shot] Q: You have exactly 537 dollars to spend on party gifts for your rich uncle 's birthday party . You decide to get watches for the ladies at 27.9
A: Alright, let's tackle this problem step by step. I'm going to break it down to understand how many watches and beepers we can buy with $537, given the c
Understanding the Problem
We have two types of gifts:
1. **Watches** for ladies: $27.98 each.
2. **Beepers** for men: $23.46 each.


DeepSeek_new.ipynb
Saving failed since 16:51
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
from google.colab import files
uploaded = files.upload()
import io
df = load_dataset(io.StringIO(list(uploaded.values())[0].decode('utf-8')))
# Choose the one to be run:
# run_zero_shot(df)
run_few_shot(df)
# run_chain_of_thought(df)
Given:
- Total money to spend: $537.
- The number of watches is 3 times the number of beepers.
Let's define:
- Let the number of beepers be **b**.
- Then, the number of watches is **3b**.
Setting Up the Equation
The total cost is the sum of the cost of watches and the cost of beepers:
\[\text{Cost of watches} + \text{Cost of beepers} = \text{Total money} \]
\[(27.98 \times 3b) + (23.46 \times b) = 537 \]
Now, let's calculate that:
\[27.98 \times 3 = 83.94 \]
So,
\[83.94b + 23.46b = 537 \]
Combine like terms:
\[(83.94 + 23.46)b = 537 \]
\[107.40b = 537 \]


DeepSeek_new.ipynb
Saving failed since 16:51
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
Solving for b
Now, divide both sides by 107.40 to find b:
\[b = \frac{537}{107.40} \]
Let's do the division:
\[107.40 \times 5 = 537 \]
Because:
\[100 \times 5 = 500 \]
\[7.40 \times 5 = 37 \]
\[500 + 37 = 537 \]
So,
\[b = 5 \]
Finding the Number of Watches
Number of watches is 3 times the number of beepers:
\[\text{Watches} = 3b = 3 \times 5 = 15 \]
Verifying the Solution
Let's check if this adds up to $537.
- Cost of watches: 15 watches × $27.98 each.
\[15 \times 27.98 \]
Let's calculate:
\[10 \times 27.98 = 279.80 \]
\[5 \times 27.98 = 139.90 \]
\[279.80 + 139.90 = 419.70 \]
- Cost of beepers: 5 beepers × $23.46 each.
\[5 \times 23.46 = 117.30 \]


DeepSeek_new.ipynb
Saving failed since 16:51
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
[Few-Shot] Q: Old McDonald has 15 goats and chickens on his farm . The animals combined have a total of 34 legs . Assumi
A: Let's solve the problem step by step.
Given:
- Total number of animals (goats and chickens) = 15
- Total number of legs = 34
Assumptions:
- Each goat has 4 legs.
- Each chicken has 2 legs.
Let:
- \(g \) = number of goats
- \(c \) = number of chickens
Equations:
1. \(g + c = 15 \) (total number of animals)
2. \(4g + 2c = 34 \) (total number of legs)
Solve the first equation for \(c \):
\(c = 15 - g \)
Substitute \(c \) into the second equation:
\(4g + 2(15 - g) = 34 \)
Simplify:
\(4g + 30 - 2g = 34 \)
\(2g + 30 = 34 \)
\(2g = 4 \)
\(g = 2 \)
Now find \(c \):
\(c = 15 - g = 15 - 2 = 13 \)
Final Answer:


DeepSeek_new.ipynb
Saving failed since 16:51
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
\(c = 15 - g = 15 - 2 = 13 \)
Final Answer:
[2.0, 13.0]
GT: [[2.0, 13.0]]
[Few-Shot] Q: The shoe store has twice as many black shoes as it does brown shoes . The total number of shoes is 66 . How many brown shoes are there ?
A: [[22.0, 44.0]]
GT: [[22.0, 44.0]]
Few-Shot Accuracy: 95.45%



DeepSeek_new.ipynb — Saving failed since 13:10

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all

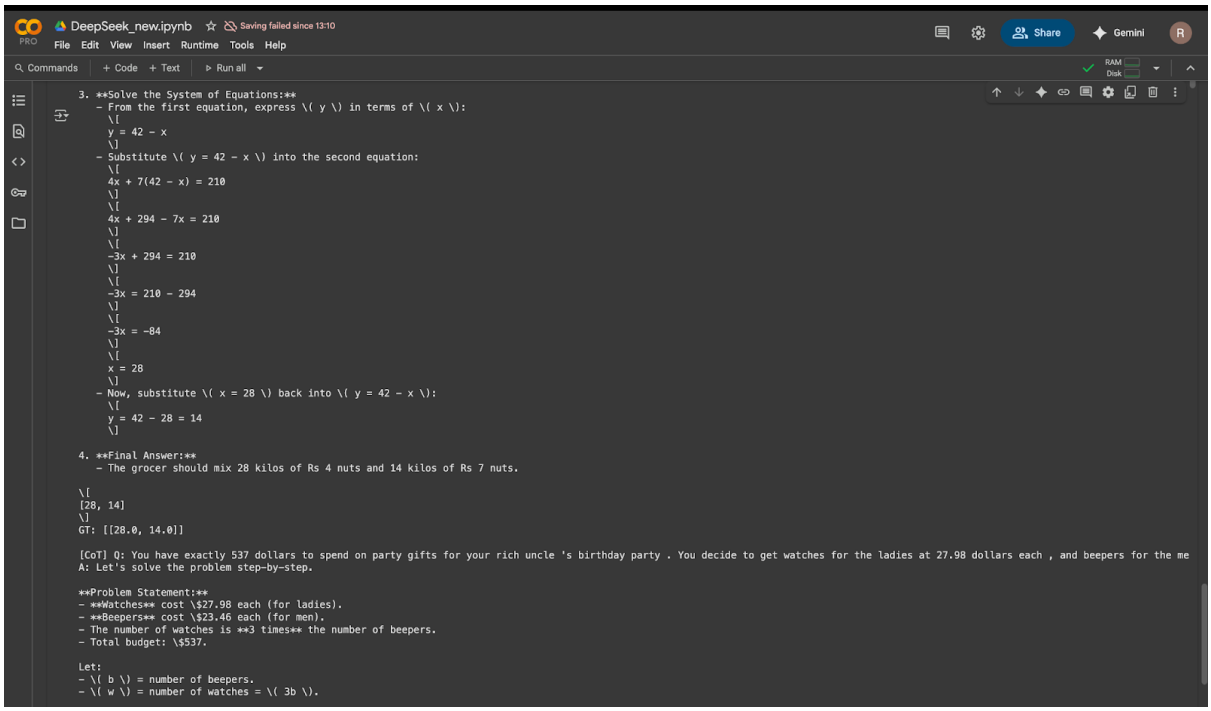
Share Gemini

```
from google.colab import files
uploaded = files.upload()

import io
df = load_dataset(io.StringIO(list(uploaded.values())[0].decode('utf-8')))

# Choose the one you want to run:
run_chain_of_thought(df)

# run_zero_shot(df)
# run_few_shot(df)
```

Choose Files alg514_fold...st.orig.jsonl

- **alg514_fold4_test.orig.jsonl**(n/a) - 196201 bytes, last modified: 7/24/2025 - 100% done

```
Saving alg514_fold4_test.orig.jsonl to alg514_fold4_test.orig.jsonl
[CoT] Q: a grocer wants to mix nuts which sell for rs 4 per kilo with nuts which sell for rs 7 per kilo in order to make a mixture which could sell for rs 5 per kilo . How many kilos of eac
A: Let's solve the problem step-by-step.

**Problem Statement:**
A grocer wants to mix nuts that sell for Rs 4 per kilo with nuts that sell for Rs 7 per kilo to make a mixture of 42 kilos that sells for Rs 5 per kilo. How many kilos of each should he mix

**Solution:**

1. **Define Variables:**
   - Let \( x \) be the number of kilos of Rs 4 nuts.
   - Let \( y \) be the number of kilos of Rs 7 nuts.

2. **Set Up the Equations:**
   - The total weight of the mixture is 42 kilos:
     \[
     x + y = 42
     \]
   - The total cost of the mixture is Rs 5 per kilo for 42 kilos, so the total cost is \( 5 \times 42 = 210 \) Rs.
     The cost of the Rs 4 nuts is \( 4x \), and the cost of the Rs 7 nuts is \( 7y \). Therefore:
     \[
     4x + 7y = 210
     \]

3. **Solve the System of Equations:**
   - From the first equation, express \( y \) in terms of \( x \):
     \[
     y = 42 - x
     \]
   - Substitute \( y = 42 - x \) into the second equation:
     \[
     4x + 7(42 - x) = 210
     \]
     \[
```



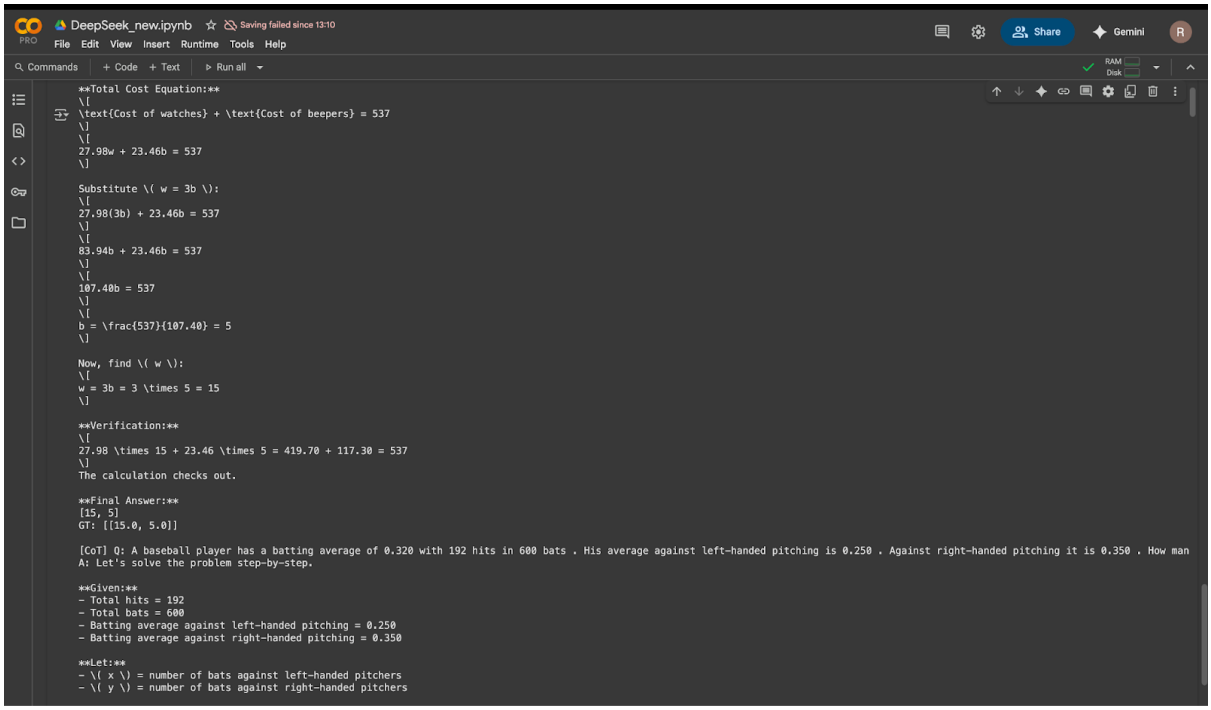
DeepSeek_new.ipynb — Saving failed since 13:10

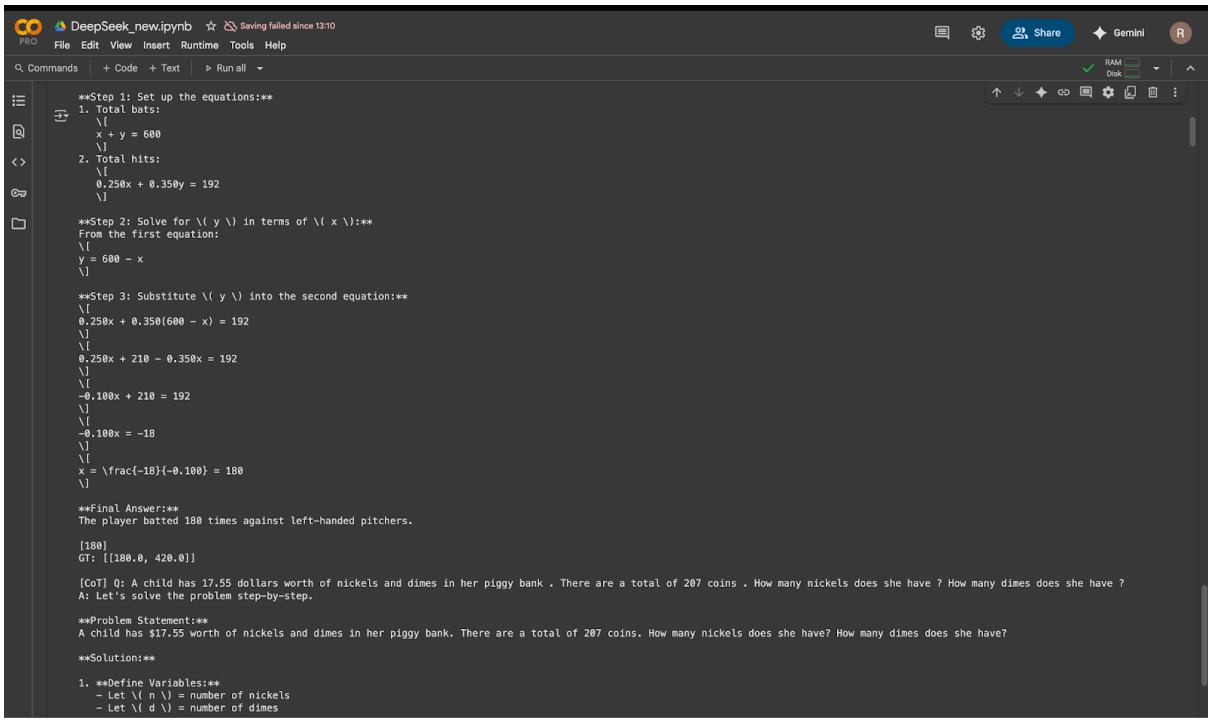
File Edit View Insert Runtime Tools Help

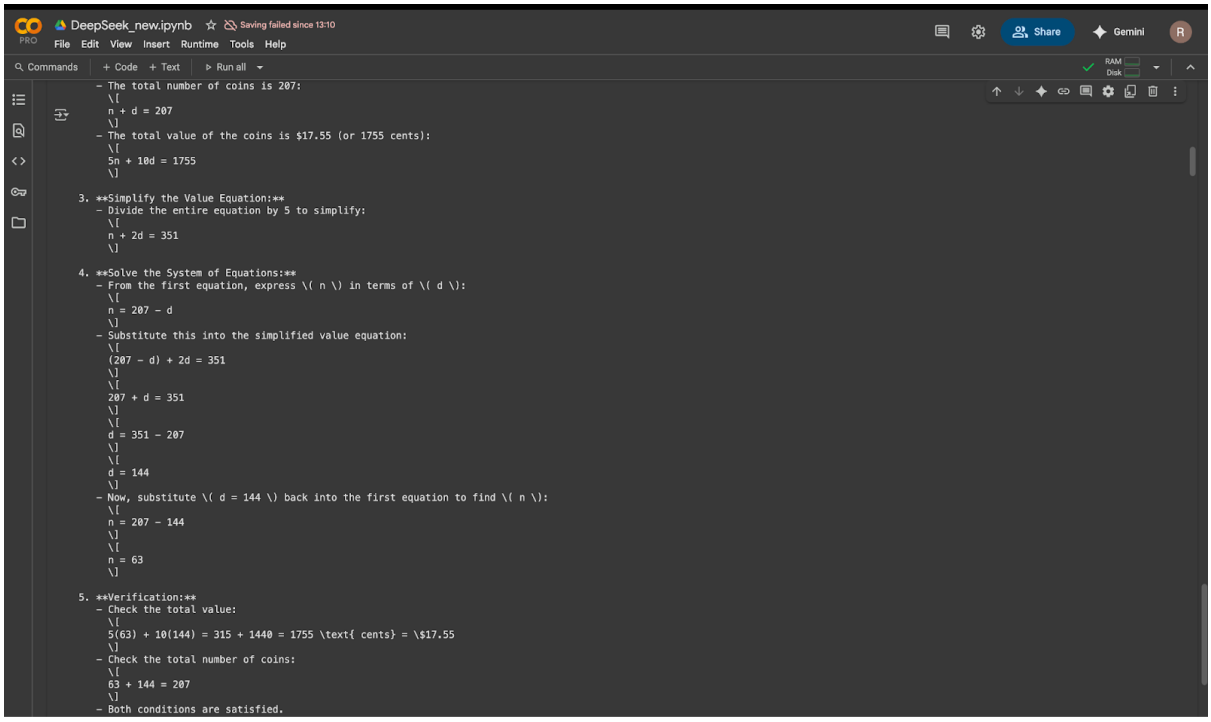
Commands + Code + Text ▷ Run all

Share Gemini

```
3. **Solve the System of Equations:**
   - From the first equation, express \( y \) in terms of \( x \):
     \[
     y = 42 - x
     \]
   - Substitute \( y = 42 - x \) into the second equation:
     \[
     4x + 7(42 - x) = 210
     \]
     \[
     4x + 294 - 7x = 210
     \]
     \[
     -3x + 294 = 210
     \]
     \[
     -3x = 210 - 294
     \]
     \[
     -3x = -84
     \]
     \[
     x = 28
     \]
   - Now, substitute \( x = 28 \) back into \( y = 42 - x \):
     \[
     y = 42 - 28 = 14
     \]

4. **Final Answer:**
   - The grocer should mix 28 kilos of Rs 4 nuts and 14 kilos of Rs 7 nuts.

\[
[28, 14]
\]
GT: [[28.0, 14.0]]

[CoT] Q: You have exactly 537 dollars to spend on party gifts for your rich uncle 's birthday party . You decide to get watches for the ladies at 27.98 dollars each , and beepers for the me
A: Let's solve the problem step-by-step.

**Problem Statement:**
- **Watches** cost \$27.98 each (for ladies).
- **Beepers** cost \$23.46 each (for men).
- The number of watches is **3 times** the number of beepers.
- Total budget: \$537.

Let:
- \( b \) = number of beepers.
- \( w \) = number of watches = \( 3b \).
```


DeepSeek_new.ipynb
Saving failed since 13:10
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
Total Cost Equation:
\[
\text{Cost of watches} + \text{Cost of beepers} = 537
\]
\[
27.98w + 23.46b = 537
\]
Substitute \(w = 3b \):
\[
27.98(3b) + 23.46b = 537
\]
\[
83.94b + 23.46b = 537
\]
\[
107.40b = 537
\]
\[
b = \frac{537}{107.40} = 5
\]
Now, find \(w \):
\[
w = 3b = 3 \times 5 = 15
\]
Verification:
\[
27.98 \times 15 + 23.46 \times 5 = 419.70 + 117.30 = 537
\]
The calculation checks out.
Final Answer:
[15, 5]
GT: [[15.0, 5.0]]
[CoT] Q: A baseball player has a batting average of 0.320 with 192 hits in 600 bats . His average against left-handed pitching is 0.250 . Against right-handed pitching it is 0.350 . How man
A: Let's solve the problem step-by-step.
Given:
- Total hits = 192
- Total bats = 600
- Batting average against left-handed pitching = 0.250
- Batting average against right-handed pitching = 0.350
Let:
- \(x \) = number of bats against left-handed pitchers
- \(y \) = number of bats against right-handed pitchers


DeepSeek_new.ipynb
Saving failed since 13:10
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Run all
RAM
Disk
Step 1: Set up the equations:
1. Total bats:
\[
x + y = 600
\]
2. Total hits:
\[
0.250x + 0.350y = 192
\]
Step 2: Solve for \(y \) in terms of \(x \):
From the first equation:
\[
y = 600 - x
\]
Step 3: Substitute \(y \) into the second equation:
\[
0.250x + 0.350(600 - x) = 192
\]
\[
0.250x + 210 - 0.350x = 192
\]
\[
-0.100x + 210 = 192
\]
\[
-0.100x = -18
\]
\[
x = \frac{-18}{-0.100} = 180
\]
Final Answer:
The player batted 180 times against left-handed pitchers.
[180]
GT: [[180.0, 420.0]]
[CoT] Q: A child has 17.55 dollars worth of nickels and dimes in her piggy bank . There are a total of 207 coins . How many nickels does she have ? How many dimes does she have ?
A: Let's solve the problem step-by-step.
Problem Statement:
A child has $17.55 worth of nickels and dimes in her piggy bank. There are a total of 207 coins. How many nickels does she have? How many dimes does she have?
Solution:
1. **Define Variables:**
- Let \(n \) = number of nickels
- Let \(d \) = number of dimes



DeepSeek_new.ipynb — Saving failed since 13:10

File Edit View Insert Runtime Tools Help

Share Gemini

Commands + Code + Text Run all

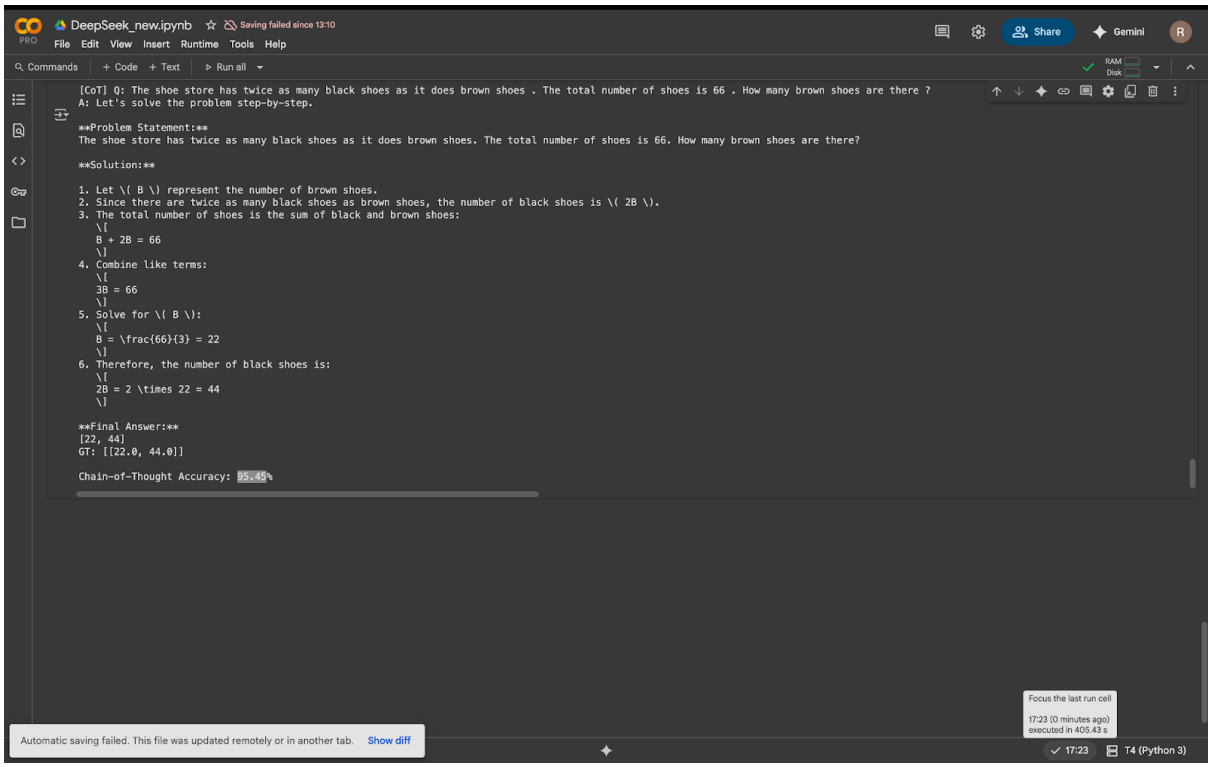
RAM Disk

- The total number of coins is 207:
  \[
  n + d = 207
  \]
- The total value of the coins is $17.55 (or 1755 cents):
  \[
  5n + 10d = 1755
  \]

3. **Simplify the Value Equation:**
   - Divide the entire equation by 5 to simplify:
     \[
     n + 2d = 351
     \]

4. **Solve the System of Equations:**
   - From the first equation, express \(n \) in terms of \(d \):
     \[
     n = 207 - d
     \]
   - Substitute this into the simplified value equation:
     \[
     (207 - d) + 2d = 351
     \]
     \[
     207 + d = 351
     \]
     \[
     d = 351 - 207
     \]
     \[
     d = 144
     \]
   - Now, substitute \(d = 144 \) back into the first equation to find \(n \):
     \[
     n = 207 - 144
     \]
     \[
     n = 63
     \]

5. **Verification:**
   - Check the total value:
     \[
     5(63) + 10(144) = 315 + 1440 = 1755 \text{ cents} = \$17.55
     \]
   - Check the total number of coins:
     \[
     63 + 144 = 207
     \]
   - Both conditions are satisfied.



DeepSeek_new.ipynb — Saving failed since 13:10

File Edit View Insert Runtime Tools Help

Share Gemini

Commands + Code + Text Run all

RAM Disk

[CoT] Q: The shoe store has twice as many black shoes as it does brown shoes . The total number of shoes is 66 . How many brown shoes are there ?
A: Let's solve the problem step-by-step.

Problem Statement:
The shoe store has twice as many black shoes as it does brown shoes. The total number of shoes is 66. How many brown shoes are there?

Solution:

1. Let \(B \) represent the number of brown shoes.
2. Since there are twice as many black shoes as brown shoes, the number of black shoes is \(2B \).
3. The total number of shoes is the sum of black and brown shoes:
   \[
   B + 2B = 66
   \]
4. Combine like terms:
   \[
   3B = 66
   \]
5. Solve for \(B \):
   \[
   B = \frac{66}{3} = 22
   \]
6. Therefore, the number of black shoes is:
   \[
   2B = 2 \times 22 = 44
   \]

Final Answer:
[22, 44]
GT: [[22.0, 44.0]]

Chain-of-Thought Accuracy: 95.45%

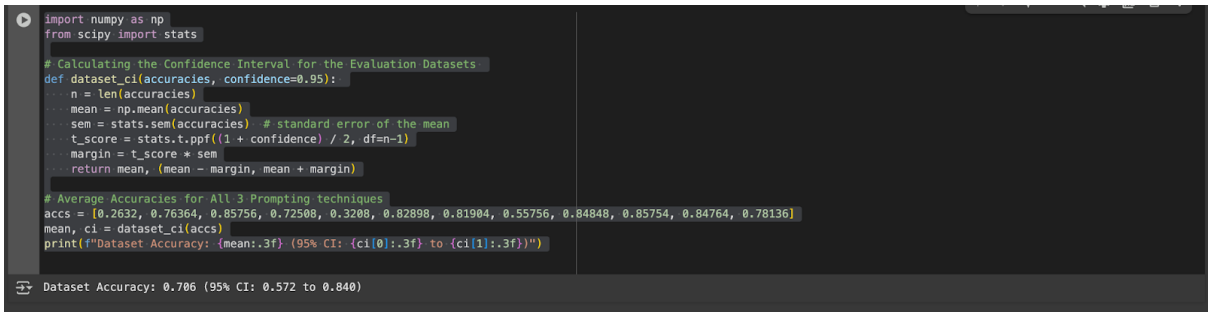
Automatic saving failed. This file was updated remotely or in another tab. Show diff

Focus the last run cell

17:23 (0 minutes ago) executed in 405.43 s

17:23 T4 (Python 3)

Python Script for Calculating the Confidence Interval for the Dataset based on Average Accuracy:



```
import numpy as np
from scipy import stats

# Calculating the Confidence Interval for the Evaluation Datasets
def dataset_ci(accuracies, confidence=0.95):
    n = len(accuracies)
    mean = np.mean(accuracies)
    sem = stats.sem(accuracies)  # standard error of the mean
    t_score = stats.t.ppf((1 + confidence) / 2, df=n-1)
    margin = t_score * sem
    return mean, (mean - margin, mean + margin)

# Average Accuracies for All 3 Prompting techniques
accs = [0.2632, 0.76364, 0.85756, 0.72508, 0.3208, 0.82898, 0.81904, 0.55756, 0.84848, 0.85754, 0.84764, 0.78136]
mean, ci = dataset_ci(accs)
print(f"Dataset Accuracy: {mean:.3f} (95% CI: {ci[0]:.3f} to {ci[1]:.3f})")
```

```
Dataset Accuracy: 0.706 (95% CI: 0.572 to 0.840)
```